TKR COLLEGE OF ENGINEERING AND TECHNOLOGY
AN AUTONOMOUS INSTITUTION
Accredited by NBA and NAAC with 'A' Grade.
(Sponsored by TKR Educational Society, Approved by AICTE, Affiliated to JNTU H)
Medbowli, Meerpet, Balapur, Hyderabad, Telangana – 500 097
Phone: 9100377790, email: info@tkrcet.ac.in, web site: www.tkrcet.ac.in

# AI-BASED FINANCIAL IDENTIFICATION BASED ON DEMOGRAPHY AND FINANCIAL FORECASTING

*An explainable machine-learning system for individual financial profiling and economic forecasting*

A project report submitted in partial fulfilment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY**

in

**COMPUTER SCIENCE AND ENGINEERING**

**Submitted by**

| | |
|---|---|
| **MOHD AHMED** | **23K91A05F7** |
| **MANNE UPENDER** | **23K91A05E6** |
| **PADALA RAJESH** | **23K91A05H7** |
| **MADAGOUNI HARSHAVARDHAN GOUD** | **23K91A05D5** |

Batch C9 (CSE-C)

Under the esteemed guidance of

**B SRIKANTH**

[DESIGNATION]

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**TKR COLLEGE OF ENGINEERING AND TECHNOLOGY (AUTONOMOUS)**

Hyderabad, Telangana

Academic year [ACADEMIC YEAR]

# CERTIFICATE

**TKR COLLEGE OF ENGINEERING AND TECHNOLOGY**
**AN AUTONOMOUS INSTITUTION**
Accredited by NBA and NAAC with 'A' Grade.
(Sponsored by TKR Educational Society, Approved by AICTE, Affiliated to JNTU H)
Medbowli, Meerpet, Balapur, Hyderabad, Telangana – 500 097
*Phone: 9100377790, email: info@tkrcet.ac.in, web site: www.tkrcet.ac.in*

This is to certify that the project report entitled **"AI-Based Financial Identification Based On Demography and Financial Forecasting"** is a bonafide record of the work carried out by **MOHD AHMED** (23K91A05F7), **MANNE UPENDER** (23K91A05E6), **PADALA RAJESH** (23K91A05H7), **MADAGOUNI HARSHAVARDHAN GOUD** (23K91A05D5), in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, during the academic year [ACADEMIC YEAR], under my supervision.

| **Project Guide** | **Head of the Department** | **Principal** |
|---|---|---|
| B SRIKANTH | [HOD NAME] | [PRINCIPAL NAME] |
| [DESIGNATION] | Department of Computer Science and Engineering | TKR College of Engineering and Technology (Autonomous) |

External Examiner: [NAME AND SIGNATURE]

Place: Hyderabad
Date: [DATE]

# DECLARATION

We hereby declare that the project report entitled **"AI-Based Financial Identification Based On Demography and Financial Forecasting"**, submitted to the Department of Computer Science and Engineering, TKR College of Engineering and Technology (Autonomous), in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work carried out by us under the guidance of B SRIKANTH, [DESIGNATION]. The results embodied in this report have not been submitted to any other university or institution for the award of any degree or diploma.

MOHD AHMED 23K91A05F7
MANNE UPENDER 23K91A05E6
PADALA RAJESH 23K91A05H7
MADAGOUNI HARSHAVARDHAN GOUD 23K91A05D5

Place: Hyderabad
Date: [DATE]

## ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, B SRIKANTH, [DESIGNATION], for valuable guidance, encouragement and constructive suggestions throughout this project.

We thank [HOD NAME], Head of the Department of Computer Science and Engineering, for providing the facilities and support needed to complete this work, and [PRINCIPAL NAME], Principal, TKR College of Engineering and Technology (Autonomous), for the encouragement given to us.

We thank the project coordinators and all faculty and staff members of the department for their help, and our families and friends for their constant support.

We also acknowledge the authors of the base paper, the maintainers of the UCI Machine Learning Repository whose Adult (Census Income) data we used, and the developers of the open-source software used in this project (scikit-learn, XGBoost, SHAP, statsmodels, Flask, React and Vite).

MOHD AHMED
MANNE UPENDER
PADALA RAJESH
MADAGOUNI HARSHAVARDHAN GOUD

# ABSTRACT

Judging an individual's financial position is usually done with a narrow set of financial parameters, a static score and no explanation, so non-linear interactions between demographic and financial factors are missed and the decision cannot be defended to the person concerned. This project builds FinSight AI, an explainable web application that turns twelve demographic and financial inputs (age, education, occupation, marital status, household role, country, weekly hours, investment gains and losses and others) into a financial profile (Emerging, Developing or Established), a Low, Moderate or High risk band, a financial-strength score, a per-person explanation and a 12-month economic forecast.

Four classifiers (Logistic Regression, Random Forest, Gradient Boosting and XGBoost) were trained on the public UCI Adult / Census Income data (48,842 records) inside leakage-safe scikit-learn pipelines and compared on a stratified 80/20 split with 5-fold cross-validation. XGBoost was selected: on 9,769 held-out records it reaches accuracy 87.8%, precision 79.2%, recall 66.6%, F1 0.724 and ROC-AUC 0.932 (cross-validated AUC 0.929 +/- 0.001). SHAP supplies global importance and live, plain-language contributions for every assessment, and a what-if screen compares a baseline with a changed scenario. A Holt-Winters model produces the 12-month forecast with a 95% band, but on a documented synthetic index, because the census data have no time dimension.

The system is served by a Flask JSON API and a React interface with a sample login. 9 automated tests and a 35-check smoke test pass, and a prediction takes a median of 69 ms on a desktop CPU. The target is income above 50K, not creditworthiness; sex, race and marital status are model inputs and no fairness audit was done; real macroeconomic indicators and transaction data are not integrated. The results are stated with these limits and the comparison with the base paper, which studies hospitals, is given as context and not as a ranking. FinSight AI is informational and is not financial advice.

**Keywords:** financial identification, demographic analytics, economic forecasting, machine learning, XGBoost, SHAP, explainable AI, Holt-Winters, Flask, React.

# TABLE OF CONTENTS

CERTIFICATE ..... i
DECLARATION ..... ii
ACKNOWLEDGEMENT ..... iii
ABSTRACT ..... iv
TABLE OF CONTENTS ..... v
LIST OF FIGURES ..... vii
LIST OF TABLES ..... ix
LIST OF ABBREVIATIONS ..... xi
CHAPTER 1: INTRODUCTION ..... 1
1.1 Domain overview ..... 1
1.2 Background and motivation ..... 1
1.3 Problem statement ..... 2
1.4 Need for the system ..... 2
1.5 Objectives ..... 2
1.6 Scope ..... 3
1.7 Contributions ..... 4
1.8 Limitations and assumptions ..... 4
1.9 Organisation of the report ..... 5
CHAPTER 2: LITERATURE SURVEY ..... 7
2.1 Overview ..... 7
2.2 Base paper analysis ..... 7
2.3 Related work by theme ..... 9
2.4 Datasets and algorithms used in the literature and in this project ..... 11
2.5 Strengths and limitations of existing work ..... 12
2.6 Research gap ..... 12
CHAPTER 3: FEASIBILITY STUDY ..... 14
3.1 Introduction ..... 14
3.2 Technical feasibility ..... 14
3.3 Economic feasibility ..... 15
3.4 Operational feasibility ..... 15
3.5 Schedule feasibility ..... 16
3.6 Advantages and constraints ..... 16
CHAPTER 4: SYSTEM REQUIREMENTS AND ANALYSIS ..... 18
4.1 Existing system and its limitations ..... 18
4.2 Proposed system and its advantages ..... 18
4.3 Functional requirements ..... 19
4.4 Non-functional requirements ..... 19
4.5 Hardware requirements ..... 20
4.6 Software requirements ..... 20
4.7 User roles ..... 21
4.8 Input and output specification ..... 21
4.9 Constraints ..... 22
CHAPTER 5: SYSTEM DESIGN ..... 23
5.1 Architecture ..... 23
5.2 High-level workflow ..... 24
5.3 Data flow diagrams ..... 24
5.4 UML diagrams ..... 27
5.5 Database and data design ..... 29
5.6 Module design ..... 30
5.7 User interface design ..... 31
5.8 Core algorithm and training pipeline ..... 33
CHAPTER 6: METHODOLOGY AND SOFTWARE IMPLEMENTATION ..... 35
6.1 Overall methodology ..... 35
6.2 Dataset ..... 36
6.3 Preprocessing and feature engineering ..... 36
6.4 Algorithm choice and justification ..... 38

6.5 Mathematical formulation ..... 38
6.6 Model training and selection ..... 39
6.7 Application architecture and what-if logic ..... 41
6.8 API design ..... 42
6.9 Security and error handling ..... 43
6.10 Installation and execution ..... 43
CHAPTER 7: SYSTEM TESTING ..... 45
7.1 Testing strategy ..... 45
7.2 Unit testing ..... 45
7.3 Integration testing ..... 46
7.4 Functional testing ..... 46
7.5 Security testing ..... 47
7.6 Validation and edge cases ..... 48
7.7 Performance testing ..... 49
7.8 User acceptance ..... 50
7.9 Limits of the testing done ..... 50
CHAPTER 8: RESULTS AND OUTPUT SCREENS ..... 52
8.1 Output screens ..... 52
8.2 Model performance ..... 56
8.3 Comparison with the base paper ..... 62
8.4 Sample inputs and outputs ..... 63
8.5 Success and failure analysis ..... 64
8.6 Limitations of the results ..... 65
8.7 Objectives versus outcomes ..... 66
CHAPTER 9: CONCLUSION ..... 67
9.1 Summary ..... 67
9.2 Objectives achieved ..... 67
9.3 Key findings ..... 67
9.4 Practical applications ..... 68
9.5 Known limitations ..... 68
CHAPTER 10: FUTURE SCOPE ..... 70
10.1 Improvements ..... 70
10.2 Additional features ..... 70
10.3 Scalability and deployment ..... 70
10.4 Further research ..... 71
REFERENCES ..... 72
APPENDIX A: IMPORTANT CODE ..... 75
A.1 Input validation (app/utils/validation.py) ..... 75
A.2 Cleaning and derived features (scripts/feature_engineering.py) ..... 75
A.3 Leakage-safe preprocessing (scripts/feature_engineering.py) ..... 76
A.4 Candidate models (scripts/models.py) ..... 76
A.5 Split, cross-validation and model selection (scripts/train.py) ..... 76
A.6 Forecasting (scripts/forecast.py) ..... 77
A.7 Profile mapping, prediction and live SHAP (app/services/predictor.py) ..... 77
APPENDIX B: INSTALLATION AND USER GUIDE ..... 79
B.1 Requirements ..... 79
B.2 Automatic start with run.bat ..... 79
B.3 Manual start ..... 79
B.4 Sample login ..... 79
B.5 User guide, screen by screen ..... 80
B.6 Troubleshooting ..... 80
APPENDIX C: ADDITIONAL SCREENS ..... 81

# LIST OF FIGURES

Figure 1: FinSight AI dashboard: serving model, test metrics, top SHAP drivers and the economic outlook ...2
Figure 2: Offline training and evaluation pipeline that produces the models and result files the application serves ...18
Figure 3: System architecture of FinSight AI ...23
Figure 4: High-level workflow of one assessment, from sign-in to exploration ...24
Figure 5: Data flow diagram, level 0 (context diagram) ...24
Figure 6: Data flow diagram, level 1 ...25
Figure 7: Data flow diagram, level 2: processes 2.0 and 3.0 expanded ...26
Figure 8: UML class diagram (modules drawn as classes) ...27
Figure 9: Sequence diagram: one financial assessment ...28
Figure 10: Use-case diagram ...28
Figure 11: Entity-relationship (data model) diagram ...29
Figure 12: Landing page of the React interface (light theme) ...31
Figure 13: The same landing page in the dark theme ...32
Figure 14: Sign-in page with the sample account ...32
Figure 15: Assessment form filled with a sample profile ...33
Figure 16: Flowchart of the core algorithm (POST /api/predict) ...33
Figure 17: The four classifiers compared on the held-out test split ...40
Figure 18: ROC-AUC of every cross-validation fold for each model ...40
Figure 19: Median and 95th-percentile response time of the API routes ...50
Figure 20: Validation error: age 7 is outside the permitted range of 16 to 100, so the field is outlined and the analysis is refused ...52
Figure 21: Result for the sample profile: Emerging profile, High risk band, financial-strength score 1.1%, with the SHAP factors that shaped it ...53
Figure 22: Result for a dual-income profile (age 34, Local-gov, Some-college, married, Sales, 42 hours): Developing profile, Moderate risk, score 46.8% ...53
Figure 23: Result for a mid-career manager (age 52, Private, Masters, married, Exec-managerial, gains 7,688, 50 hours): Established profile, Low risk, score 99.9% ...54
Figure 24: Explainability screen: global SHAP importance (left) and the SHAP summary of per-row effects (right) ...54
Figure 25: Forecast screen: 12-month Holt-Winters projection of the Household Financial Conditions Index with its 95% interval ...55
Figure 26: What-if screen: scenario gains of 80,000 and 50 weekly hours compared with the baseline profile ...55
Figure 27: Performance screen: model comparison table with the selected model highlighted, followed by the metrics, ROC and confusion-matrix views ...56
Figure 28: Confusion matrices of the four models on the test split (rows: actual, columns: predicted) ...57
Figure 29: ROC curves of the four models on the test split (generated by the training script) ...57
Figure 30: Precision, recall, F1 and accuracy of the selected model as the decision threshold moves ...58
Figure 31: Reliability diagram of the selected model on the test split ...59
Figure 32: Number of test rows per profile band and the share of each band that actually earns above 50K ...59
Figure 33: Twelve largest global SHAP importances of the selected model ...60
Figure 34: SHAP beeswarm plot: effect of each feature value on the prediction (generated by the training script) ...61
Figure 35: Holt-Winters forecast of the synthetic index with its 95% band ...61
Figure 36: ROC-AUC of the same four model families in the base paper and in this project (different data and protocols) ...62
Figure 37: About screen: the six-step description of how the system works, the technology list and the responsible-use statement ...81
Figure 38: Page-not-found screen shown for an unknown address inside the application, with buttons back to the dashboard and the landing page ...81
Figure 39: Result for a 52-year-old married executive with a Master's degree, gains of 7,688 and 50 working hours: profile *Established*, risk *Low*, strength 99.9% (upper part of the page, with the strengthening factors led by investment gains) ...82
Figure 40: Explainability screen (upper part): global importance bars for the top twelve features, the SHAP beeswarm summary and the original bar chart rendered at training time ...82

Figure 41: Forecast screen: horizon, trend, projected change (+3.46%) and residual standard deviation (1.659); the history-and-forecast chart with its 95% band; the interpretation text that names the series as synthetic; and the first six forecast months ..... 83
Figure 42: Performance screen (upper part): best model, dataset rows, train / test split, the four-model comparison table, the metrics-at-a-glance bars and the XGBoost confusion matrix with the ROC curves ..... 83

# LIST OF TABLES

Table 1: Abbreviations used in this report ..... xi
Table 2: Objectives and where they are addressed ..... 3
Table 3: Organisation of this report ..... 5
Table 4: Principal results reported in the base paper ..... 8
Table 5: Summary of the surveyed works ..... 10
Table 6: Algorithms in the base paper and in this project ..... 11
Table 7: Strengths and limitations of the surveyed approaches ..... 12
Table 8: The base paper and this project compared by setting ..... 12
Table 9: Technical feasibility: needs and evidence ..... 14
Table 10: Cost view of the project ..... 15
Table 11: Project phases in order ..... 16
Table 12: Limitations of the existing approaches ..... 18
Table 13: Functional requirements and where they are implemented ..... 19
Table 14: Non-functional requirements and the evidence ..... 19
Table 15: Hardware used ..... 20
Table 16: Software used and its version ..... 20
Table 17: User roles ..... 21
Table 18: Inputs of one assessment ..... 21
Table 19: Outputs of one assessment ..... 21
Table 20: Data model of FinSight AI (transient objects and persisted files) ..... 29
Table 21: Modules of FinSight AI and their responsibilities ..... 30
Table 22: Stages of the base-paper framework and how this project realises them ..... 35
Table 23: Facts about the classification dataset ..... 36
Table 24: Derived features added by add_derived_features() ..... 37
Table 25: Methods used and the reason for each ..... 38
Table 26: Hyperparameters of the four candidate classifiers ..... 39
Table 27: Test-split metrics and cross-validated ROC-AUC of the four models ..... 40
Table 28: Endpoints of FinSight AI ..... 42
Table 29: Test layers used ..... 45
Table 30: Unit tests ..... 45
Table 31: Integration and functional tests ..... 46
Table 32: Smoke-test checks (run against both ports) ..... 46
Table 33: Hostile and malformed input (POST /api/predict) ..... 47
Table 34: File-access probes (GET) ..... 47
Table 35: Missing and malformed requests (POST /api/predict) ..... 48
Table 36: Numeric boundaries and ranges (POST /api/predict) ..... 48
Table 37: What-if scenarios (POST /api/whatif) ..... 49
Table 38: API response time (milliseconds, 40 calls each) ..... 49
Table 39: Test-set metrics of the four classifiers ..... 56
Table 40: Confusion-matrix counts, specificity, PR-AUC and Brier score on the test split ..... 57
Table 41: Threshold analysis of the selected model (test split) ..... 58
Table 42: Calibration of the selected model: predicted probability against observed share above 50K (test split) ..... 58
Table 43: Profile bands on the test split ..... 59
Table 44: Top ten global SHAP importances (XGBoost, 500-row sample, log-odds scale) ..... 60
Table 45: Back-test of the forecasting method on the synthetic series ..... 61
Table 46: Base paper and this project compared (ROC-AUC) ..... 62
Table 47: What differs between the two studies ..... 62
Table 48: Sample profiles and the system's output ..... 63
Table 49: Strongest drivers reported for each sample (two strengthening, one weakening) ..... 63
Table 50: Most confident errors on the test split (FP: predicted above 50K, actually at most 50K; FN: the reverse) ..... 64
Table 51: Test-split outcomes by group (threshold 0.5) ..... 64
Table 52: Objectives versus outcomes ..... 66
Table 53: Objectives and their status ..... 67
Table 54: Software and hardware needed to run FinSight AI ..... 79

Table 55: Screens of the application and what to do on each ..... 80
Table 56: Common problems and fixes ..... 80

# LIST OF ABBREVIATIONS

**Table 1: Abbreviations used in this report**

| Abbreviation | Expansion |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| AUC | Area Under the Curve |
| CSS | Cascading Style Sheets |
| CSV | Comma-Separated Values |
| CV | Cross-Validation |
| DFD | Data Flow Diagram |
| ER | Entity-Relationship |
| ETS | Error, Trend, Seasonality (exponential smoothing) |
| FN / FP | False Negative / False Positive |
| GB | Gradient Boosting |
| HTTP | Hypertext Transfer Protocol |
| JSON | JavaScript Object Notation |
| LR | Logistic Regression |
| MAE | Mean Absolute Error |
| MAPE | Mean Absolute Percentage Error |
| ML | Machine Learning |
| PR | Precision-Recall |
| REST | Representational State Transfer |
| RF | Random Forest |
| RMSE | Root Mean Squared Error |
| ROC | Receiver Operating Characteristic |
| SHAP | SHapley Additive exPlanations |
| SPA | Single-Page Application |
| TN / TP | True Negative / True Positive |
| UCI | University of California, Irvine |
| UI | User Interface |
| UML | Unified Modeling Language |
| XAI | Explainable Artificial Intelligence |
| XGB | Extreme Gradient Boosting (XGBoost) |

# CHAPTER 1: INTRODUCTION

## 1.1 Domain overview

Financial identification is the task of describing where an individual stands financially: how strong the position is, how exposed it is to risk and where it is likely to move. Lenders, planners, policy makers and individuals all need such a picture, and the quality of the decisions that follow depends on how well it is drawn. The classical tools are statistical scores built from a few financial ratios. Altman's Z-score used five ratios in a linear discriminant function and Ohlson's logistic model relaxed its distributional assumptions [5], [6]; both showed that financial ratios carry real information about viability, and both belong to a generation of methods that assume a largely linear relationship between inputs and outcome.

Machine learning replaced that assumption with models that learn non-linear interactions directly from data. Random forests, gradient boosting and XGBoost [7], [8], [9] have repeatedly been reported to outperform traditional statistical models on bankruptcy and credit-risk problems [10], [11], and tree ensembles in particular remain strong on tabular data of moderate size [23]. At the same time, financial decisions are regulated and contestable, so a prediction that cannot be explained is of limited use; this has driven the adoption of SHAP-style attribution in finance [14], [15], [19]. The domain of this project is therefore Artificial Intelligence and Machine Learning applied to demographic analytics, financial profiling and economic forecasting.

## 1.2 Background and motivation

Individual financial circumstances are shaped by demographic context as much as by money figures. Age, education, occupation, hours of work, household role, region and the presence of investment income all move together, and they interact: the value of an additional year of education differs by age and by occupation. A score built on a narrow set of financial parameters ignores this context, so it can be inaccurate for the people it describes and unfair to those whose circumstances differ from the average. Forecasting adds a second dimension: knowing the present position matters less than knowing where conditions are heading.

The base paper for this project [1] builds an explainable machine-learning framework for organisational financial distress, instantiated on 165 Korean hospitals. It combines domain-adaptive feature engineering, a comparison of linear, ensemble and boosted classifiers, SHAP interpretation and domain-specific validation. Its authors state that the approach is designed to be adaptable to other settings with structured financial data. The motivation of this project is to carry that approach to the individual: to apply the same chain of ideas to demographic and financial attributes of people, to wrap it in a usable application, and to measure honestly what it achieves.

The result is FinSight AI, a web application that accepts twelve demographic and financial inputs, returns a financial profile, a risk band and a financial-strength score, explains the score with SHAP contributions in plain language, compares a baseline with a changed scenario and shows a 12-month economic forecast. Its dashboard is shown below.

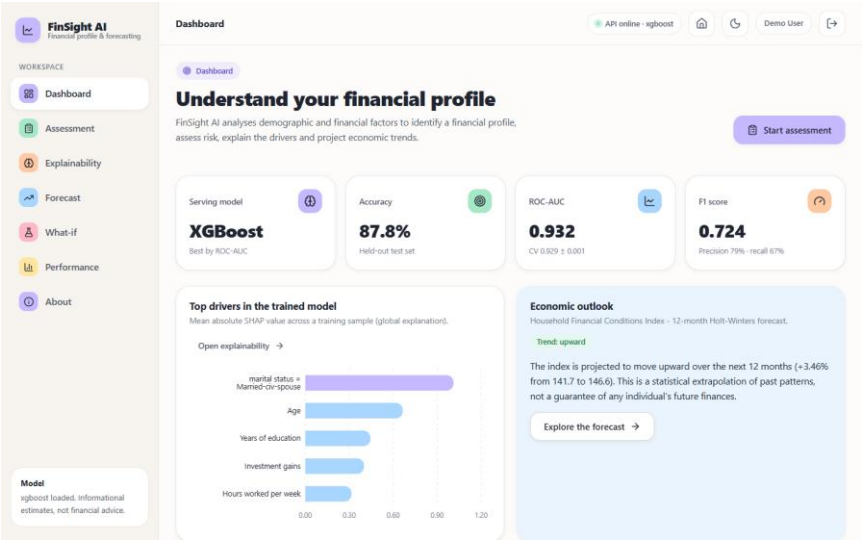



**Figure 1: FinSight AI dashboard: serving model, test metrics, top SHAP drivers and the economic outlook**

## 1.3 Problem statement

- Traditional statistical scoring misses non-linear interaction between demographic and financial factors.
- Assessment based on a limited set of parameters produces inaccurate and unfair outcomes.
- Static scoring describes the present position but does not forecast future risk.
- An unexplained financial decision cannot be defended to the customer or to a regulator.

The problem addressed here is: *given the demographic and financial attributes of one person, identify a financial profile and risk band with a calibrated strength score, show which factors drove the assessment and in which direction, let the user test what would change under a different scenario, and place the result beside a forecast of economic conditions, all through an interface a non-expert can use.*

## 1.4 Need for the system

There is a need for a system that combines demographic and financial data to assess financial position and to forecast economic conditions, with explainable output. Research code in this area usually stops at a notebook and an accuracy figure; commercial scoring products are closed and cannot be inspected. An open, reproducible implementation lets students, analysts and developers study the whole path from raw data to an explained assessment, repeat the measurements and extend the work. It is equally important that such a system states its limits clearly, because a number on a screen is easily mistaken for a verdict about a person.

## 1.5 Objectives

The project was proposed with six objectives. The table after the list gives each one with its outcome; the evidence is in Chapter 8.

1. Collect a dataset combining demographic attributes with financial and transactional parameters.
2. Preprocess the data, handle missing values and encode categorical demographic fields.

3. Train models to classify financial profile and estimate credit or distress risk.
4. Build a forecasting component that projects future financial position.
5. Add an explainability layer showing which factors drove each assessment.
6. Evaluate using accuracy, precision, recall and ROC-AUC, and deploy through a web interface.

**Table 2: Objectives and where they are addressed**

| No. | Objective (short) | Outcome | Where |
|---|---|---|---|
| 1 | Dataset of demographic and financial attributes | Met in part | Chapters 2, 6 |
| 2 | Preprocessing, missing values, encoding | Met | Chapter 6 |
| 3 | Classify financial profile, estimate risk | Met in part | Chapters 6, 8 |
| 4 | Forecasting component | Met in part | Chapters 6, 8 |
| 5 | Explainability layer | Met | Chapters 6, 8 |
| 6 | Evaluate and deploy through a web interface | Met | Chapters 5, 7, 8 |

## 1.6 Scope

**In scope.**

- Classification of one individual's demographic and financial profile (12 inputs) into an Emerging, Developing or Established profile with a Low, Moderate or High risk band and a financial-strength score.
- The preprocessing and feature-engineering chain in `scripts/feature_engineering.py`, applied identically at training and prediction time through a scikit-learn Pipeline.
- Four candidate classifiers (Logistic Regression, Random Forest, Gradient Boosting, XGBoost) evaluated by stratified 80/20 split and 5-fold cross-validation, with the best selected by ROC-AUC.
- SHAP explainability: global importance charts at training time and live local contributions for every assessment, with plain-language sentences.
- A 12-month Holt-Winters forecast of an economic index with a 95% band, and a what-if comparison of a baseline and a changed scenario.
- A Flask JSON API (port 8409), the original server-rendered pages, a React front end (port 5409) with a sample login, a dashboard, performance views and a responsive light / dark interface.
- A reproducible training pipeline, an automated test suite (pytest), a smoke test and the documentation set.

**Out of scope.**

- Credit decisions. The model predicts an income band from census-style attributes; it is not a credit score, a default or distress predictor, or a lending recommendation.
- Forecasting an individual's future finances. The forecast is of an aggregate economic index and is trained on a synthetic series.
- Real macroeconomic indicators (inflation, benchmark lending rates, GDP) named in the project abstract; they are not integrated.
- Transaction-level or bank data, income verification, identity checks and any personal-data storage.

- User accounts, real authentication, role management, audit logging and a relational database (the sample login is a browser-only demo gate).
- Fairness mitigation, causal inference, entity-grouped or temporal validation, and external validation on current data.
- Cloud deployment and horizontal scaling; the Docker files cover the original Flask pages only.

## 1.7 Contributions

The work is an application and an integration of established methods; its contributions are listed below, and Chapter 9 repeats them against the evidence.

1. **From organisations to individuals.** The explainable machine-learning framework of the base paper [1] (feature engineering, multi-model comparison, SHAP, validation) is carried over from hospital panel data to individual demographic and financial profiling on 48,842 census records.
2. **An end-to-end system, not only a model.** Validated form input, feature engineering, XGBoost prediction, profile and risk band, live SHAP explanation, what-if comparison, forecast and performance views are served through a Flask API and a React interface.
3. **Leakage-safe, reproducible evaluation.** Imputation, encoding and scaling live inside each Pipeline; four models are compared on one stratified split plus 5-fold CV; every metric is written to JSON and re-derived by a script.
4. **Explanations in plain language.** Per-assessment SHAP values become sentences, including a correct reading of one-hot features (the absence of a category), and the what-if screen is labelled as simulation, not causation.
5. **A forecasting component with honest provenance.** A Holt-Winters 12-month forecast with a 95% band is built on a documented synthetic index and its limits are stated wherever it appears.
6. **Evaluation beyond a single score.** Confusion matrices, a threshold sweep, calibration bins, observed income rates per profile band, API latency and automated test results are reported.

## 1.8 Limitations and assumptions

**Assumptions.**

- The 1994 US census extract (UCI Adult) is a usable stand-in for demographic and financial profiles; its income label is taken as ground truth.
- One row describes one person, so a random stratified split does not leak entities between training and test sets.
- Users enter values from the permitted option lists and ranges (age 16-100, hours 1-99, gains and losses 0-1,000,000).
- The census sampling weight `fnlwgt` has no meaning for a new user and is imputed with the training median at prediction time.
- The Household Financial Conditions Index used for forecasting is illustrative; its trend and seasonality are assumed to continue unchanged over the 12-month horizon.
- The system runs on a single local machine with Python 3.11 or newer and Node.js 18 or newer.

**Limitations.**

- **Old, US-specific data.** The Adult extract describes 1994 US adults. Its income distribution, occupations and wage levels do not describe current populations, and the countries in the form map to a handful of census categories.
- **Income is not creditworthiness.** The target is income above 50K. The report therefore claims a financial-profile signal only; it makes no claim about repayment behaviour or distress.
- **Synthetic forecast series.** The forecast component is validated only against its own generator (back-test MAPE below 1%), which says nothing about accuracy on real economic data.
- **Sensitive attributes in the model.** Sex, race and marital status are inputs and marital status is the most important SHAP feature. No fairness audit or mitigation was performed, so the model may reproduce historical bias.
- **Recall of the positive class is modest.** At the default 0.5 threshold the selected model finds about two thirds of the people with income above 50K; the threshold analysis shows how this trades against precision.
- **Model selected on the test split.** The best model was chosen by test ROC-AUC, with no separate validation set, no hyperparameter search and no significance test; the cross-validation ranking agrees with the test ranking.
- **Extrapolation in what-if scenarios.** Large changes (for example investment gains of 80,000) move the score by close to 100 points because the training data are sparse there; scenario output is a model simulation, not a causal prediction.
- **Demo-grade security and storage.** The login is client-side only, no data is persisted, the session secret has a development default, and the Docker image serves the Flask pages only.
- **Not financial advice.** Outputs are informational and must not be used as the sole basis for any financial decision.

To give a sense of scale: the selected model reaches an accuracy of 87.8% and a ROC-AUC of 0.932 on 9,769 held-out records, but it finds only 66.6% of the people who earn more than 50K at the default threshold, and every statement about it concerns income in a 1994 census extract, not creditworthiness today. The rest of the report keeps these limits in view.

## 1.9 Organisation of the report

**Table 3: Organisation of this report**

| Chapter | Content |
|---|---|
| 1 Introduction | Domain, motivation, problem, objectives, scope, contributions, limitations. |
| 2 Literature Survey | The base paper in depth, related work by theme, datasets and algorithms, research gap. |
| 3 Feasibility Study | Technical, economic, operational and schedule feasibility. |
| 4 System Requirements and Analysis | Existing and proposed system, functional and non-functional requirements, hardware, software, users. |
| 5 System Design | Architecture, workflow, data-flow diagrams, UML, data model, modules, API and UI design. |
| 6 Methodology and Software Implementation | Dataset, preprocessing, models, mathematics, training, forecasting, application, security, installation. |

| Chapter | Content |
|---|---|
| 7 System Testing | Strategy and results of unit, integration, functional, security, performance and edge-case tests. |
| 8 Results and Output Screens | Model performance, comparison with the base paper, output screens, failure analysis, objectives versus outcomes. |
| 9 Conclusion | Summary, achievements, findings, applications, known limitations. |
| 10 Future Scope | Improvements, features, deployment and research directions. |
| Appendices | A: important code; B: installation and user guide; C: additional screens. |

# CHAPTER 2: LITERATURE SURVEY

## 2.1 Overview

This chapter reviews the work on which the project rests. It starts with the base paper, which supplies the framework that the project adapts, and then groups the related work into four themes: machine learning for financial distress and credit risk, explainable artificial intelligence in finance, data quality and privacy in credit modelling, and forecasting. The batch literature survey lists five entries: the base paper, a 2025 study of sovereign credit risk forecasting, and three further entries recorded by title and year only. The base paper and the sovereign study were read and are discussed here; the other three entries [45], [46], [47] were recorded in the batch literature survey and could not be independently verified, so no claim is built on them. Two further credit-risk papers kept in the batch archive were read in full or in part and are used as related work, and the classical references are taken from the reference list of the base paper itself.

## 2.2 Base paper analysis

The base paper is: Y. Kim and E. J. Sun, "An Explainable Machine Learning Framework for Organizational Financial Distress Detection: Evidence From Korean Hospital Panel Data," IEEE Access, vol. 14, pp. 100337-100350, 2026, doi: 10.1109/ACCESS.2026.3709346.

### *Problem and contribution*

Kim and Sun [1] observe that research on financial-distress prediction concentrates on publicly traded corporations and stock-market signals, while non-market organisations such as hospitals, universities and public bodies are under-studied, and that predictive accuracy alone is not enough for high-stakes financial use because regulators and managers need to know why an organisation is predicted to be in distress. They present a four-stage framework and instantiate it on Korean hospitals. They are explicit that the framework assembles standard components and that its contribution lies in the domain-adaptive integration of those components.

### *The four stages*

1. **Domain-adaptive feature engineering.** Structured disclosures with 236 line items per hospital-year are reduced to 11 interpretable features: raw values are turned into ratios that are comparable across organisation sizes (for example labour cost over revenue), institutional characteristics are encoded as binary indicators, and time and size indices are added. Missing values are handled by listwise deletion when rare and ratios are winsorised at the 1st and 99th percentiles. Features with correlation above 0.85 are removed.
2. **Multi-model training.** Logistic Regression (L2, C = 1.0, standardised features), Random Forest (100 trees), Gradient Boosting (100 trees, depth 3, learning rate 0.1), XGBoost (100 trees, default parameters) and a small neural network (two hidden layers of 64 and 32 units, early stopping). Hyperparameters are kept at defaults for reproducibility, and class rebalancing is not applied when the minority class exceeds 30%.
3. **SHAP interpretation.** TreeExplainer provides global importance (mean absolute SHAP), directional effects through beeswarm plots, awareness of compositional collinearity among cost shares and a reconciliation of SHAP rankings with logistic-regression coefficients through a variance-versus-effect-size argument.

4. **Domain-specific validation.** Hospital-level Group K-Fold (primary), a temporal hold-out, standard stratified K-Fold (reference), a counterfactual subsidy analysis and panel fixed-effects regression.

### *Data*

The data are the mandatory public financial disclosures of Korean hospitals with 300 or more beds: a balanced panel of 165 hospitals observed from 2019 to 2023, giving 825 hospital-year observations. The target is a deficit indicator (net income below zero), positive for 41.2% of observations overall, with yearly deficit rates of 35.2%, 46.7%, 26.1%, 29.7% and 69.1% from 2019 to 2023. The jump in 2023 follows the withdrawal of pandemic subsidies and makes the panel a test of robustness to distribution shift.

### *Reported results*

**Table 4: Principal results reported in the base paper**

| Quantity | Value |
|---|---|
| ROC-AUC under hospital-level Group K-Fold: XGBoost / Gradient Boosting / Random Forest | 0.842 / 0.840 / 0.831 |
| ROC-AUC under Group K-Fold: Logistic Regression / small neural network | 0.776 / 0.772 |
| Modified Altman Z" score as a baseline | 0.680 |
| Temporal validation (train 2019-2022, test 2023); year-only baseline | 0.780; 0.659 |
| Forward prediction of next-year deficit (665 hospital-year pairs) | 0.772 |
| Rolling-origin AUC per split; mean | 0.731, 0.850, 0.808, 0.809; 0.800 |
| Random Forest under random CV versus Group K-Fold | 0.860 versus 0.831 |
| XGBoost after removing subsidy ratio and year; after also removing the three cost ratios | 0.794; 0.651 |
| XGBoost default versus best of a 25-configuration random search | 0.827 versus 0.834 |
| Mean \|SHAP\|: material cost ratio / labour cost ratio / university indicator | 0.777 / 0.729 / 0.537 |
| DeLong p-values: XGBoost vs Gradient Boosting / vs Random Forest / vs Logistic Regression | 0.396 / 0.546 / 0.028 |
| Counterfactual gap between observed and latent distress (2021); fixed-effects post-subsidy effect | 57.5 pp; 28.9 pp ($p < 0.001$) |

Three observations matter for this project. First, the three ensemble models are statistically indistinguishable from one another (all pairwise DeLong p-values above 0.39), and only XGBoost is significantly better than Logistic Regression, so the ordering among ensembles should not be over-read. Second, the gap between random and entity-grouped cross-validation (0.860 versus 0.831 for Random Forest) quantifies leakage when the same entity appears on both sides of a split; the paper therefore treats the grouped protocol as primary. Third, the authors state that SHAP explains the model's learned associations and not causal mechanisms, that attributions of collinear shares are unstable, and that their econometric analysis has limits, which is the stance this project also adopts.

### *Strengths and weaknesses of the base paper*

- Strengths: a clear, stage-wise framework; entity-level and temporal validation; a significance test between models; robustness checks (ablation, forward prediction, rolling origin, tuning); and an honest discussion of SHAP's limits.
- Weaknesses for the purposes of this project: the unit is an organisation, not a person; the panel is small (825 rows); the label is mechanically linked to some features; and there is no application layer, so a non-expert cannot use the framework.

### *What this project takes from it*

The project keeps the four-stage structure (engineered features, four of the five model families, SHAP, validation) and the discipline of stating the validation protocol next to every number. It does not reproduce the paper's temporal, counterfactual or econometric analyses, because the individual-level data used here have one row per person and no time dimension.

## 2.3 Related work by theme

### *Machine learning for financial distress and credit risk*

The base paper describes three generations of methods [1]. The first is linear: Altman's Z-score [5] and Ohlson's logistic model [6]. The second is ensemble learning: random forests [7], gradient boosting [8] and XGBoost [9]. Barboza et al. [10] compared such methods extensively and, as summarised in the base paper, found that ensembles outperform traditional statistical models for bankruptcy prediction; Zieba et al. [11] showed that ensemble boosted trees with synthetic feature generation improve predictive performance further. The third generation uses deep learning: Mai et al. [13] applied recurrent networks to textual disclosures, and Moscatelli et al. [12] combined traditional financial features with machine learning for corporate default forecasting. Deep models trade interpretability for flexibility and typically need more data than organisational settings supply. For tabular data of moderate size, Grinsztajn et al. [23] report that tree-based models still outperform deep learning, which supports the choice of tree ensembles here.

Abid [2] forecasts credit-default probabilities implied by credit-default-swap spreads for Egypt, Morocco and Saudi Arabia during periods of political crisis. Five machine-learning models (linear, ridge, lasso, XGBoost and kernel ridge regression) and four deep models (RNN, LSTM, BiLSTM and GRU) are compared; linear regression performs best and the RNN and GRU are competitive, and the forecast probabilities are mapped to implied ratings. Only the abstract of this study was available for this survey, so its data periods and error metrics are not discussed. Its relevance is the forecasting orientation and the observation that a simple model can be hard to beat.

### *Explainable AI in finance*

The need for interpretability in financial applications has produced a large literature [20]. Ribeiro et al. [16] introduced LIME, which approximates a model locally with an interpretable surrogate; Lundberg and Lee [14] grounded feature attribution in Shapley values from cooperative game theory, and Lundberg et al. [15] added TreeExplainer, which computes exact values for tree ensembles efficiently. Cerneviciene and Kabasinskas [19] reviewed explainable AI in finance, identified SHAP as the most widely adopted method and noted that few studies provide systematic end-to-end frameworks. Bussmann et al. [17] used Shapley values to explain credit-risk models for lending to small and medium enterprises, and Gramegna and Giudici [18] evaluated the discriminative power of SHAP and LIME in credit risk. Doshi-Velez and Kim [21] called for rigorous evaluation of interpretability claims, and Molnar et al. [22] separated inherently interpretable models from post-hoc explanation, arguing that both are needed. This project follows that advice: it reports a logistic-regression baseline with its coefficients beside SHAP, and it labels every explanation as an association.

### *Data quality and privacy in credit modelling*

Maurino [3] studies how missing values, noise, outliers and label errors in the training data affect ten commonly used credit-risk models on an open-source dataset, using the Pucktrick library to inject controlled corruption. Some imperfections (outliers, missing values, duplicates) can sometimes improve a given model, with a reported gain of up to 17% in F1 for specific models, while noise and mislabelling generally degrade performance; the effect depends on the model, the feature and the type of error. The study suggests extending the work to multiclass tasks, other datasets and fairness. The lesson for this project is to treat data preparation as part of the model: the Adult data contain missing markers in workclass, occupation and native country, and the pipelines impute them inside cross-validation.

Zhang et al. [4] address a different problem: credit-risk models improve when alternative data, such as mobile-phone or e-commerce records, are added to demographic, financial and credit-history data, but those data sit with external parties and sharing them infringes privacy. They propose PrivacyCredit, a privacy-preserving, model-confidential and lossless method based on vertical federated learning, and report that it matches the performance of the model trained on the insecure plaintext combination. The paper also notes that XGBoost is widely used for credit risk because of its interpretability and predictive performance. It is relevant here as a statement of what the project does not do: it uses only the data the user types in and stores nothing.

### *Forecasting*

Exponential smoothing with trend and seasonality was introduced by Winters [33] building on Holt's work [34]; textbook treatments and accuracy measures are given by Hyndman and Athanasopoulos [36] and Hyndman and Koehler [35]. The method is simple, has few parameters and produces forecasts with an interval, which suits an index with a stable trend and yearly seasonality. It is implemented in statsmodels [41]. The survey found no study that places such a forecast beside an individual-level explainable assessment.

### *Other entries recorded in the batch literature survey*

The batch literature survey also records a 2025 title on credit and financial cycle synchronisation and sovereign credit risk [45], a 2024 title on financial and credit risk research [46] and a 2025 conference title on a multimodal credit-risk forecasting system for financial text analysis [47]. These entries give only a title and a year, were not independently verified, and are not used for any factual claim in this report.

**Table 5: Summary of the surveyed works**

| Work | Approach | Data | Reported result | Limitation |
|---|---|---|---|---|
| Kim and Sun [1] (base paper) | Four-stage explainable ML: feature engineering, LR / RF / GB / XGBoost / MLP, SHAP, domain validation | 165 Korean hospitals, 825 hospital-year rows, 2019-2023 | XGBoost AUC 0.842 (hospital-level Group K-Fold); 0.780 on 2023 hold-out; Altman Z" 0.680 | Organisations, not individuals; small panel; SHAP is associational |
| Abid [2] | Five ML and four deep models forecasting CDS-implied default probability | Sovereign credit data for Egypt, Morocco and Saudi Arabia | Linear regression best; RNN and GRU competitive | Three countries; sovereign risk, not individual profiles |
| Maurino [3] | Effect of missing values, noise, outliers and label errors on 10 credit-risk models (Pucktrick library) | One open-source credit-risk dataset | Some errors can raise F1 (up to 17% for specific models); noise and mislabelling generally hurt | Binary task, one dataset; fairness not studied |

| Work | Approach | Data | Reported result | Limitation |
|---|---|---|---|---|
| Zhang et al. [4] | PrivacyCredit: privacy-preserving credit-risk learning with alternative data (vertical federated learning) | Real credit data linked with alternative data | Same performance as the insecure plaintext combination | Needs partner data; no demographic explanation layer |
| Barboza et al. [10] | Comparison of ensembles with statistical models for bankruptcy prediction | Listed firms | Ensembles outperform traditional statistical models (as summarised in the base paper) | Corporate setting |
| Bussmann et al. [17] | Shapley values to explain credit-risk models | SME lending data | Explanations for credit decisions | SME-level, not demographic profiling |
| Gramegna and Giudici [18] | SHAP and LIME compared for discriminative power in credit risk | Credit-risk data | Evaluates explanation methods | No end-to-end application |
| Grinsztajn et al. [23] | Benchmark of tree-based models against deep learning on tabular data | Many tabular datasets | Tree-based models still outperform deep models on typical tabular data | General benchmark, not finance-specific |

## 2.4 Datasets and algorithms used in the literature and in this project

The datasets in the surveyed work are organisational or sovereign: a Korean hospital panel in the base paper [1], sovereign credit data in Abid [2], an open-source credit-risk dataset in Maurino [3], and a credit dataset linked with alternative data in Zhang et al. [4]. None is a public file of individual demographic records. For the individual profile, this project uses the Adult (Census Income) data, which were extracted from the 1994 US census and introduced to the research community by Kohavi [31]; they are distributed by the UCI Machine Learning Repository [32]. The data hold 48,842 records (32,561 in the training file and 16,281 in the test file) with 14 attributes and a binary income label, of which 23.93% are in the higher-income class. Missing values are marked with a question mark in workclass (2,799 cells), occupation (2,809) and native country (857).

Because the Adult records have no time dimension, they cannot support forecasting. The forecasting component therefore uses a separate monthly series, the Household Financial Conditions Index, which is synthetic and documented as such (trend, yearly seasonality, noise and a mid-2020 downturn, 132 monthly values). It is used only to demonstrate the method, and every screen and document that shows it says so.

**Table 6: Algorithms in the base paper and in this project**

| Algorithm | Base paper | This project | Role in this project |
|---|---|---|---|
| Logistic Regression | Yes (L2, C = 1.0, standardised) | Yes (C = 1.0, standardised) | Interpretable baseline; statsmodels logit gives coefficients and p-values |
| Random Forest | Yes (100 trees) | Yes (300 trees) | Bagged-tree comparator |
| Gradient Boosting | Yes (100 trees, depth 3) | Yes (scikit-learn defaults) | Boosted-tree comparator |
| XGBoost | Yes (100 trees, defaults) | Yes (400 trees, depth 6, learning rate 0.05) | Selected model by ROC-AUC |
| Neural network (MLP) | Yes (64 and 32 units) | No | Not reproduced |
| SHAP TreeExplainer | Yes (global and local) | Yes (global at training, local per request) | Plain-language explanation |
| Group K-Fold, temporal hold-out | Yes | No (one row per person) | Replaced by stratified 5-fold CV and a stratified test split |
| Counterfactual and fixed-effects analysis | Yes | No | Replaced by a labelled what-if simulation |

| Algorithm | Base paper | This project | Role in this project |
|---|---|---|---|
| Holt-Winters exponential smoothing | No | Yes (additive trend and season, 12 months) | Forecast of the economic index |

## 2.5 Strengths and limitations of existing work

**Table 7: Strengths and limitations of the surveyed approaches**

| Theme | Strengths | Limitations |
|---|---|---|
| Statistical scores (Z-score, logit) | Transparent, cheap, long track record [5], [6] | Linear, few ratios; the base paper reports a Z" AUC of 0.680 on its panel [1] |
| Tree ensembles | Capture non-linear interactions; strong on tabular data [10], [23] | Opaque without an explanation layer; ensembles are statistically close to each other in the base paper |
| Deep models | Flexible; can use text and sequences [13], [2] | Need more data; weaker interpretability; no gain over linear regression in the sovereign study |
| SHAP and LIME | Local and global attribution [14], [15], [16] | Associational, not causal; unstable for collinear features [1], [21] |
| Data-quality studies | Show which errors hurt which models [3] | One dataset, binary task, fairness not studied |
| Privacy-preserving learning | Uses more data without exposing it [4] | Needs partners that hold alternative data; does not address explanation to the end user |

Across the themes, three weaknesses recur. The unit of analysis is the organisation or the country, not the person. The output is a score or a ranking and is not delivered to a user with an explanation. And the forecast, where one exists, is a separate exercise from the assessment. Existing work also rarely states the limits of the label: income, default and deficit are different things.

## 2.6 Research gap

Explainable machine learning for financial distress has been developed for organisations and sovereigns, while work on individuals concentrates on accuracy and uses a narrow set of financial parameters. None of the surveyed works combines individual demographic profiling, a per-person explanation, a forecast of economic conditions and a usable web interface in one reproducible system. This project fills that gap on public data.

The base paper states that its framework is designed to be adaptable to other settings with structured financial data [1]. Individual profiling is such a setting in which demographic context carries much of the signal, and it is the one chosen here. The gap is addressed in three steps: apply the framework to a public individual-level dataset with leakage-safe pipelines, explain each assessment in language a user can read, and serve the result with a forecast and a scenario tool in an application whose limits are stated on every screen.

**Table 8: The base paper and this project compared by setting**

| Aspect | Base paper | This project |
|---|---|---|
| Unit of analysis | Hospital (organisation) | Individual person |
| Data | 165 hospitals, 825 hospital-year rows, 2019-2023 | UCI Adult, 48,842 records (1994 census extract) |
| Target | Deficit (net income below zero), 41.2% positive | Income above 50K, 23.9% positive; used as a profile signal |
| Features | 11 engineered ratios and indicators | 12 user inputs, 26 columns after engineering |

| Aspect | Base paper | This project |
|---|---|---|
| Validation | Group K-Fold, temporal hold-out, stratified CV | Stratified 80/20 split and 5-fold stratified CV |
| Best model and score | XGBoost, AUC 0.842 (Group K-Fold) | XGBoost, AUC 0.932 (test split) |
| Explanation | SHAP global and local, counterfactual analysis | SHAP global and live local, plain language, what-if simulation |
| Forecast | Counterfactual trajectories | 12-month Holt-Winters forecast of a synthetic index |
| Delivery | Analysis code | Flask API and React web application |

The two columns describe different problems on different data, so the scores are not comparable; Chapter 8 repeats this caveat where the numbers are set side by side.

# CHAPTER 3: FEASIBILITY STUDY

## 3.1 Introduction

A feasibility study asks whether the proposed system can be built, whether it is worth building and whether it can be used and kept running. For a student project the answers are mostly technical and operational, because the economic side is dominated by free software and existing hardware. This chapter answers each question with evidence from the build: the software that was installed, the machine that was used and the measurements that were taken.

## 3.2 Technical feasibility

The system needs tabular data, a machine-learning toolkit, an explanation library, a time-series library, a web framework and a browser front end. Each is available as open-source software with a stable Python or JavaScript interface: scikit-learn [24] for pipelines and models, XGBoost [9] for the boosted trees, SHAP [14], [15] for explanations, statsmodels [41] for Holt-Winters smoothing and the logit statistics, Flask [42] for the API, and React [43] with Vite [44] for the interface.

**Table 9: Technical feasibility: needs and evidence**

| Need | Chosen solution | Evidence that it works |
|---|---|---|
| Public individual-level data with demographic and financial attributes | UCI Adult / Census Income, 48,842 records [32] | Downloaded once into data/raw; loads in the training script |
| Leakage-safe preprocessing and four models | scikit-learn Pipeline with ColumnTransformer; LR, RF, GB, XGBoost | All four trained; test ROC-AUC 0.914 to 0.932 |
| Per-request explanation | SHAP TreeExplainer on the selected XGBoost model | Live explanation is part of every prediction; median 69 ms per request |
| Forecast with an interval | Holt-Winters (statsmodels), 12 months | Back-test MAE 1.429 on the synthetic series |
| API and interface | Flask JSON API on 8409; React SPA on 5409 | 35 of 35 smoke checks and 9 automated tests pass |
| Reproducibility | random_state 42, metrics written to JSON, report_metrics.py re-derives them | Re-derived numbers equal the stored ones exactly |

The main technical risk was size and speed. The Random Forest candidate is large (its saved file is about 320 MB because it holds 300 unpruned trees), while the selected XGBoost pipeline is 1.2 MB and is loaded once at start-up. A prediction including its SHAP explanation takes a median of 69 ms and a what-if comparison, which runs the model twice, 136 ms, both on a CPU-only desktop. The risk was therefore judged acceptable. A second risk is version drift: the saved pipelines were written with older scikit-learn and XGBoost releases and emit warnings when loaded by the newer versions used for the final measurements; the metrics reproduce exactly, and retraining with `python scripts/train.py` regenerates the files.

### *Data feasibility*

The classification data are feasible because they are public, individual-level, large enough (48,842 records, 39,073 for training and 9,769 for testing) and rich in the attributes the abstract names: age, education, occupation, household role and location (through native country), plus capital gains and losses as the financial signal. Their weaknesses were known in advance and shaped the design. They are old; they have no time dimension, so a separate series is needed for

forecasting; they hold no transaction-level data; and a small share of workclass, occupation and country values is missing, which the pipelines impute. The census sampling weight has no meaning for a new user and is therefore imputed with the training median at prediction time. A fallback that generates a synthetic dataset with the same schema exists in the training script for offline machines; it was not used for any result reported here.

## 3.3 Economic feasibility

Every software component is open source and free of licence cost, and the dataset is public. The project needs no cloud service, no paid API and no specialised hardware. All development, training and measurement were done on one desktop (Intel Core i5-12500 (12th Gen), 15.7 GB of memory, Windows 11 Pro) with no graphics processor, which is possible because the tabular data and the tree models are small by deep-learning standards.

**Table 10: Cost view of the project**

| Item | Cost | Note |
|---|---|---|
| Python, scikit-learn, XGBoost, SHAP, statsmodels, pandas, NumPy | No licence fee | Open-source libraries |
| Flask, React, Vite, Recharts, Framer Motion and other front-end packages | No licence fee | Open-source packages installed with npm |
| Dataset | No fee | Public UCI Adult file; attribution to the UCI repository |
| Hardware | Existing desktop | No GPU, no server, no cloud account |
| Hosting | None | Runs on localhost only (127.0.0.1) |
| Effort | Student team time | Not costed |

The system is therefore economically feasible for its purpose. A production deployment would add costs that this project does not estimate: hosting, security review, model monitoring and legal review of the use of demographic attributes in financial decisions.

## 3.4 Operational feasibility

The people who would use the system are an individual who wants to understand a financial profile and an analyst who wants to inspect the model. Both are served by a browser interface that needs no installation beyond starting the two servers. Operational feasibility rests on five points.

- **Ease of use.** A single form of twelve fields with inline range hints and server-side validation; the result screen shows the profile, risk band, score and the factors behind it in plain language.
- **Ease of start-up.** `run.bat` creates the virtual environment, installs the requirements, trains the models if they are missing, builds the front end once and opens the application; the manual steps are in Appendix B.
- **Maintainability.** Retraining is one command; every metric is written to a JSON file that the interface reads, so a retrained model appears on the dashboard and performance screens without code changes.
- **Verifiability.** 9 pytest tests, a 35-check smoke test and a screenshot script can be run at any time to confirm that the system still works.

- **Responsible use.** Every analysis screen states that the output is informational, not financial advice, and the what-if screen states that it is a simulation and not a causal prediction.

Operational limits are equally clear. The login is a browser-only demo gate, nothing is stored about a person, and there is no administrator role, audit log or support process; a real service would need all three.

## 3.5 Schedule feasibility

The work was organised in the phases below. The order reflects the dependencies between them: the models cannot be explained before they are trained, and the interface cannot be built before the API exists. Calendar dates and durations are not recorded in the project artefacts and are not stated here.

**Table 11: Project phases in order**

| Phase | Main activities | Output | Status |
|---|---|---|---|
| 1 Problem identification and survey | Abstract, base paper, literature survey, objectives | Batch documents 2-6 | Completed |
| 2 Data and preprocessing | Download Adult data, cleaning, missing values, feature engineering | scripts/feature_engineering.py, data/ | Completed |
| 3 Modelling and validation | Four pipelines, stratified split, 5-fold CV, selection, SHAP artefacts | models/, artifacts/*.json | Completed |
| 4 Forecasting | Synthetic index, Holt-Winters model, 95% band | scripts/forecast.py, artifacts/forecast.json | Completed |
| 5 Flask application | Validation, prediction service, what-if, server-rendered pages, JSON API | app/ | Completed |
| 6 React front end | Landing, sample login, dashboard, assessment, result, explain, forecast, what-if, performance | frontend/ | Completed |
| 7 Testing and measurement | pytest, smoke test, latency, screenshot walk-through, report metrics | tests/, scripts/, artifacts/measured.json | Completed |
| 8 Documentation | Diagrams, figures, chapters, decks, report and paper | Batch root documents | Completed |

## 3.6 Advantages and constraints

**Advantages.**

- Combines demographic and financial attributes in a non-linear model instead of a handful of ratios.
- Explains every assessment, globally and per person, and states that the explanation is associational.
- Adds a forecast and a scenario tool to the assessment in one application.
- Runs entirely on a desktop, offline after installation, with no data leaving the machine.
- Reproducible: fixed seeds, scripted training, metrics regenerated by a script and checked by tests.

**Constraints.**

- The data are a 1994 US census extract and the label is income, so the output is a profile signal and not a credit decision.
- The forecast series is synthetic; real macroeconomic indicators are not integrated.
- Sex, race and marital status are model inputs and no fairness audit was performed.

- No database, no real authentication and no multi-user support.
- Single-machine, CPU-only measurements; no load or concurrency tests.

The study concludes that the system is technically, economically and operationally feasible as a research and demonstration prototype, and not feasible without further work as a decision tool for real lending.

# CHAPTER 4: SYSTEM REQUIREMENTS AND ANALYSIS

## 4.1 Existing system and its limitations

In the setting described by the problem statement, the existing practice is a traditional statistical score built from a small set of financial parameters. Its logic is fixed, it describes the present position only, and it does not say which factor moved the score. Research implementations of machine-learning alternatives, including the framework of the base paper [1], are analysis code for organisations: they stop at a metric table and a SHAP plot and offer no interface for an individual.

**Table 12: Limitations of the existing approaches**

| Limitation | Consequence |
|---|---|
| Narrow set of financial parameters | Demographic context (age, education, occupation, region, household role) is ignored; outcomes can be inaccurate and unfair |
| Linear or fixed scoring logic | Non-linear interactions between factors are missed |
| Static, present-only score | No view of where economic conditions are heading |
| No explanation | The result cannot be justified to the person concerned or to a regulator |
| Research code without an application | Nothing a non-specialist can open, try or re-run; no what-if capability |
| Silent about the label | Income, default and distress are treated as interchangeable |

## 4.2 Proposed system and its advantages

The proposed system, FinSight AI, takes twelve inputs, passes them through the same feature-engineering code that was used in training, scores them with the selected XGBoost pipeline and returns a financial-strength score, a profile (Emerging, Developing or Established) and a risk band (High, Moderate or Low). The profile and risk follow fixed thresholds on the score: 0.60 or above is Established with Low risk, 0.35 to 0.60 is Developing with Moderate risk and below 0.35 is Emerging with High risk. SHAP contributions for the individual case are converted into sentences, a what-if screen compares a baseline with a changed scenario, and a Holt-Winters forecast gives economic context.

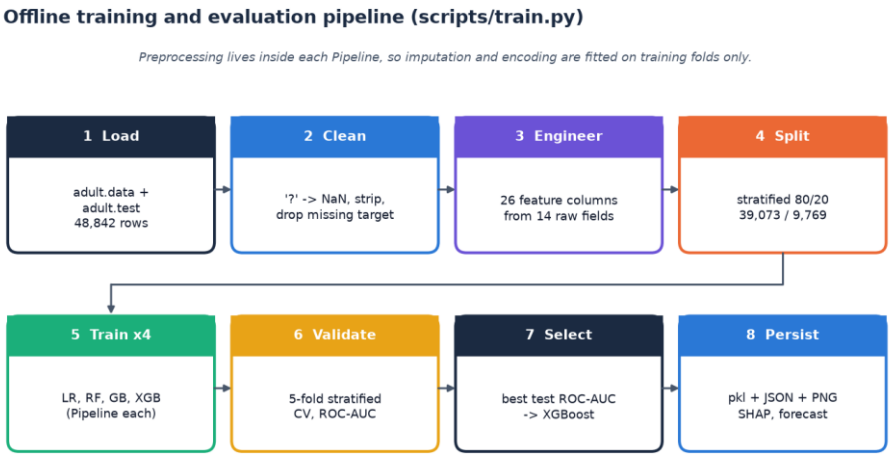



**Figure 2: Offline training and evaluation pipeline that produces the models and result files the application serves**

Its advantages over the existing approaches are the direct answers to the limitations in the table above: demographic and financial inputs together, a non-linear model selected on measured performance, a per-person explanation, a forecast, a scenario tool and a working interface. It keeps the discipline of the base paper in stating the validation protocol with every figure and in treating SHAP as association.

## 4.3 Functional requirements

**Table 13: Functional requirements and where they are implemented**

| ID | Requirement | Implementation |
|---|---|---|
| FR-1 | Let a user sign in with a sample account before using the analysis screens | /login and the guarded /app routes (client-side demo gate) |
| FR-2 | Collect twelve demographic and financial inputs with range hints | /app/assess; ranges from GET /api/options |
| FR-3 | Validate every field on the server and reject bad input with messages | app/utils/validation.py; HTTP 400 with an error list |
| FR-4 | Return profile, risk band, strength score and risk share for one person | POST /api/predict; /app/result |
| FR-5 | Explain each assessment with the strongest positive and negative factors in plain language | Live SHAP in services/predictor.py; result screen |
| FR-6 | Show global explainability (ranked features, importance charts) | GET /api/explain; /app/explain; /artifacts PNGs |
| FR-7 | Show a 12-month forecast with a 95% band and an interpretation | GET /api/forecast; /app/forecast |
| FR-8 | Compare a baseline with a scenario that changes gains, losses, hours or education | POST /api/whatif; /app/whatif |
| FR-9 | Show model comparison, validation protocol and statistical analysis | GET /api/performance; /app/performance |
| FR-10 | Show a dashboard of the serving model and headline metrics | GET /api/dashboard; /app |
| FR-11 | Report service health and model availability | GET /health |
| FR-12 | Offer light and dark themes and friendly 404 and error pages | Front-end theme toggle; NotFound page; Jinja error page |
| FR-13 | Keep the original server-rendered pages available | Flask routes /, /assess, /explain, /forecast, /whatif, /performance, /about |
| FR-14 | Retrain from the raw data and refresh every result file | python scripts/train.py |

## 4.4 Non-functional requirements

**Table 14: Non-functional requirements and the evidence**

| Quality | Requirement | Evidence |
|---|---|---|
| Performance | A prediction with its explanation should feel immediate | Median 68.9 ms, 95th percentile 116.9 ms for POST /api/predict (40 calls, 127.0.0.1, CPU only) |
| Accuracy | Rank profiles well and report the protocol | Test ROC-AUC 0.932, accuracy 87.8%, 5-fold CV AUC 0.929 +/- 0.001 |
| Explainability | Every assessment shows its drivers | Live SHAP contributions returned with every POST /api/predict |
| Reliability | Bad input must not crash the service | Invalid age, unknown categories and empty bodies return HTTP 400; covered by tests |
| Reproducibility | Same data and seed give the same numbers | random_state 42; re-derived metrics equal stored metrics |
| Maintainability | Small modules, tests, scripted checks | 9 pytest tests (all pass) and a 35-check smoke test |
| Security and privacy | No personal data kept; local-only service | Binds to 127.0.0.1; no storage of submitted profiles; demo login is not real authentication |

| Quality | Requirement | Evidence |
|---|---|---|
| Usability | Non-experts can complete an assessment | One form, inline hints, plain-language result; screens captured at 1440 x 900 |
| Portability | Run on a normal desktop | run.bat on Windows; manual steps for other systems; Docker image for the Flask pages |

Two qualities were not measured: behaviour under concurrent load and appearance on every browser and device. Chapter 7 says so where it reports testing.

## 4.5 Hardware requirements

Minimum requirements were not determined by experiment. The figures below describe the machine on which the system was built and measured, and are the only tested configuration.

**Table 15: Hardware used**

| Component | Specification |
|---|---|
| Processor | Intel Core i5-12500 (12th Gen) |
| Memory | 15.7 GB |
| Graphics processor | Not used |
| Disk | About 310 MB for the saved models (the Random Forest file alone is about 320 MB; the served model is 1.2 MB), 6 MB for the raw data and space for the Python and Node.js packages |
| Display | 1440 x 900 used for all screenshots |

## 4.6 Software requirements

**Table 16: Software used and its version**

| Software | Version | Purpose |
|---|---|---|
| Operating system | Windows 11 Pro | Development and measurement |
| Python | 3.11.9 | Training, API, scripts |
| scikit-learn | 1.9.1 | Pipelines, preprocessing, models, metrics |
| XGBoost | 3.2.0 | Selected classifier |
| SHAP | 0.51.0 | Global and local explanations |
| statsmodels | 0.15.0 | Holt-Winters forecast; logit statistics |
| Flask | 3.1.3 | JSON API and server-rendered pages |
| pandas, NumPy, joblib, matplotlib | See requirements.txt | Data handling, persistence, plots |
| Node.js | 24.19.0 | Front-end build and preview |
| React / React Router | ^19.3.1 / ^7.18.4 | Single-page application |
| Vite | ^8.3.5 | Build tool and preview server |
| Recharts, Framer Motion, GSAP, Lucide | ^3.10.1, ^14.5.0, ^3.15.0, ^1.61.0 | Charts, motion and icons |
| pytest, requests, Playwright | pytest and requests from requirements.txt; Playwright installed separately | Tests, smoke test, screenshot capture |
| Browser | Google Chrome | Screenshots and manual checks |

Front-end versions are the ranges declared in `frontend/package.json`; the Python packages are listed in `requirements.txt` as minimum versions: pandas>=2.2.0, numpy>=1.26.0, scikit-learn>=1.5.0, xgboost>=2.1.0, shap>=0.46.0, statsmodels>=0.14.0, Flask>=3.0.0, joblib>=1.4.0, matplotlib>=3.8.0, pytest>=8.0.0, requests>=2.31.0, fpdf2>=2.7.0.

## 4.7 User roles

**Table 17: User roles**

| Role | Description | Can do |
|---|---|---|
| User (individual or analyst) | Anyone who opens the web application and signs in with the sample account | Enter a profile, read the result and its explanation, run what-if scenarios, view the forecast, explainability and performance screens |
| Developer | A person who runs the scripts on the machine | Retrain the models, regenerate result files, run tests, smoke test and screenshot capture |

There is no administrator role and no per-user data: the sample account (a fixed e-mail and password shown on the sign-in screen) only gates the screens and is stored in the browser.

## 4.8 Input and output specification

The interface between the user and the model is fixed by the twelve inputs and a small set of outputs. Allowed values are exactly the categories present in the training data, and the same lists drive the form drop-downs (through GET /api/options) and the server-side check.

**Table 18: Inputs of one assessment**

| Input | Type | Allowed values |
|---|---|---|
| Age | Integer | 16 to 100 |
| Employment type (workclass) | Choice | 8 values, for example Private, Self-emp-not-inc, State-gov |
| Education | Choice | 16 levels from Preschool to Doctorate; mapped to years of education (1 to 16) |
| Marital status | Choice | 7 values, for example Married-civ-spouse, Never-married |
| Occupation | Choice | 14 values, for example Exec-managerial, Sales, Other-service |
| Household relationship | Choice | 6 values, for example Husband, Wife, Own-child |
| Race | Choice | 5 values |
| Sex | Choice | Female or Male |
| Country | Choice | 41 native-country values |
| Investment gains (annual, dollars) | Number | 0 to 1,000,000 |
| Investment losses (annual, dollars) | Number | 0 to 1,000,000 |
| Hours worked per week | Integer | 1 to 99 |

**Table 19: Outputs of one assessment**

| Output | Meaning |
|---|---|
| Financial-strength score (strength %) | The model's probability of the higher-income class, shown as a percentage |
| Risk share | 100 minus the strength score |
| Profile | Emerging (score below 0.35), Developing (0.35 to below 0.60) or Established (0.60 and above) |
| Risk band | High, Moderate or Low, paired with the profile |
| Strengthening and weakening factors | Up to five positive and five negative SHAP contributions for this person, each with a plain-language sentence |
| Insights | Short informational messages derived from the inputs, the score and the strongest drivers, ending with a disclaimer |
| Forecast context | The 12-month index forecast with a 95% band, shown beside the result |

Invalid input never reaches the model: a missing field, a value outside its set or range, or a non-numeric number produces HTTP 400 with one message per problem, and the interface shows the messages beside the fields.

## 4.9 Constraints

- **Data constraint.** The only individual-level data are the 1994 Adult records; the forecast uses a synthetic series.
- **Label constraint.** The target is income above 50K; the system makes no statement about repayment or distress.
- **Input constraint.** The twelve inputs and their allowed values are the ones present in the training data; other countries or occupations cannot be entered.
- **Interpretation constraint.** SHAP values and what-if results describe the model, not causes in the person's life.
- **Resource constraint.** CPU-only training on one desktop; no hyperparameter search was run.
- **Regulatory constraint.** The system is informational and not financial advice; use of sex, race and marital status in a real decision would need legal and ethical review.

# CHAPTER 5: SYSTEM DESIGN

## 5.1 Architecture

The design follows three rules: keep the model, the explanation and the user interface separate; load heavy objects once and reuse them; and write nothing about a person to disk. The original Flask project (server-rendered pages) is kept working, and a thin JSON layer and a React interface are added on top of the same service code, so there is one source of truth for every prediction.

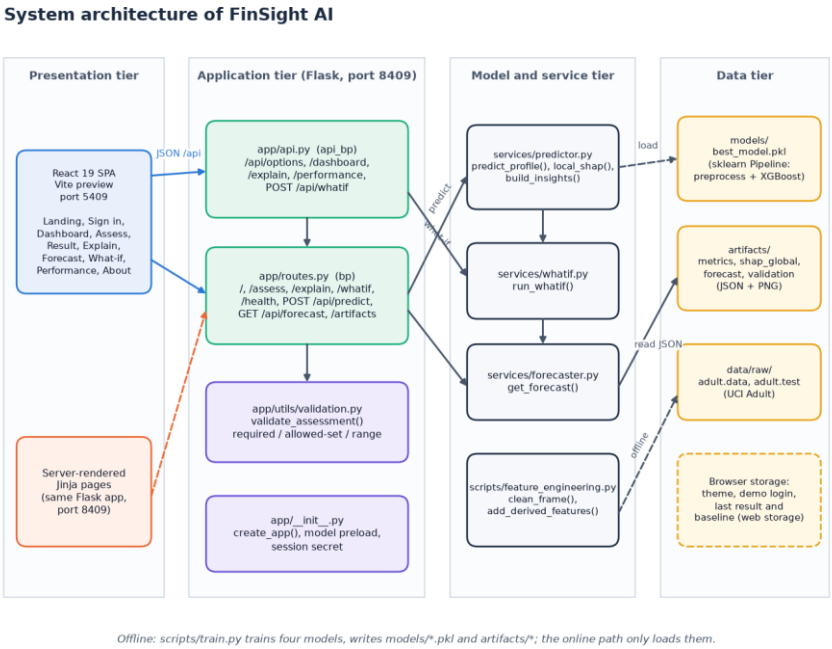



**Figure 3: System architecture of FinSight AI**

The system has four tiers. In the presentation tier the React single-page application (served by Vite on port 5409) and the server-rendered Jinja pages of the original Flask project are the two ways a user reaches the system. The application tier is the Flask service on port 8409: `app/api.py` holds the JSON routes written for the React front end, `app/routes.py` holds the page routes plus `/api/predict` and `/api/forecast`, `app/utils/validation.py` checks every input and `app/__init__.py` builds the application and preloads the model. The model and service tier contains `predictor.py` (prediction, SHAP and insights), `whatif.py`, `forecaster.py` and the shared `feature_engineering.py`. The data tier is made of files: the persisted pipeline in `models/`, the JSON and PNG artifacts, the raw census files and browser storage.

## 5.2 High-level workflow

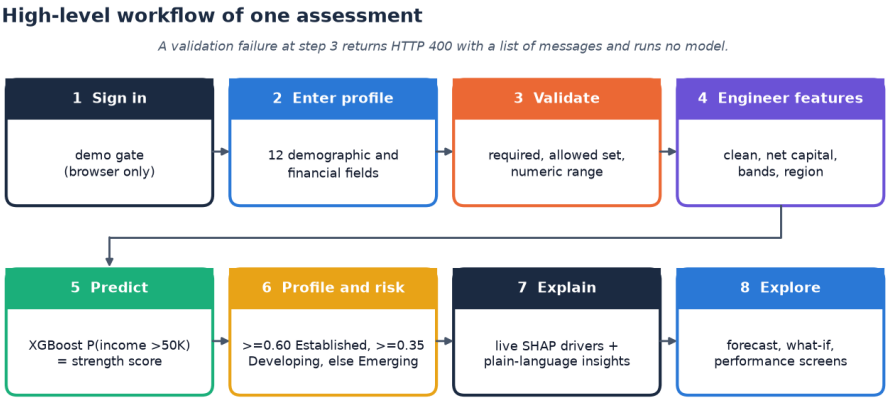



**Figure 4: High-level workflow of one assessment, from sign-in to exploration**

One assessment passes through eight steps. The user signs in through the demo gate and enters the twelve demographic and financial fields. The server validates that every field is present, belongs to its allowed set and lies in its numeric range; a failure ends the flow with HTTP 400 and a list of messages, and no model is run. Valid input is cleaned and extended with the derived features, and the saved XGBoost pipeline returns the probability of income above 50K, which is the financial-strength score. The probability is mapped to a profile and a risk band, and SHAP contributions plus rule-based insights explain it. The user can then open the forecast, the what-if comparison and the performance screens.

## 5.3 Data flow diagrams

The data flow diagrams follow Yourdon notation: circles are processes, rectangles with a bold border are external entities and open-ended rectangles are data stores.

***Level 0***

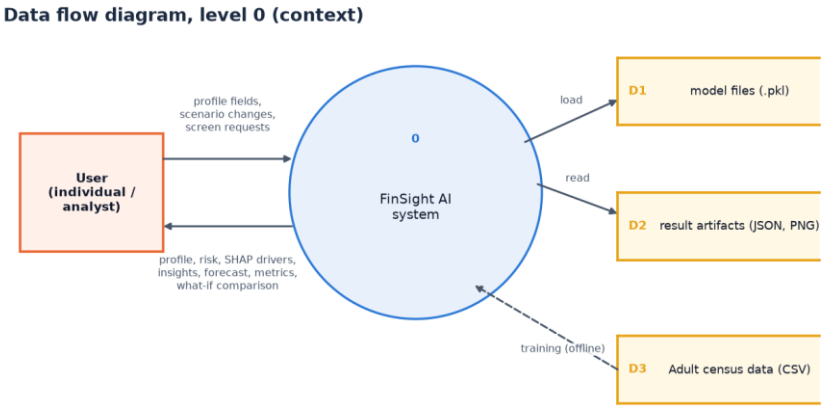



**Figure 5: Data flow diagram, level 0 (context diagram)**

The context diagram shows FinSight AI as one process with one external entity, the user. The user sends profile fields, scenario changes and requests for the other screens; the system returns the profile, risk band, SHAP drivers, insights, forecast, metrics and what-if comparison. Three

data stores surround the process. D1 holds the trained model files, which are loaded and never modified while the application runs. D2 holds the result artifacts (metrics, SHAP importance, forecast and validation reports as JSON and PNG), which are read. D3 is the Adult census data; it is touched only by the offline training run, drawn as a dashed arrow, which is the only place where D1 and D2 are written.

***Level 1***

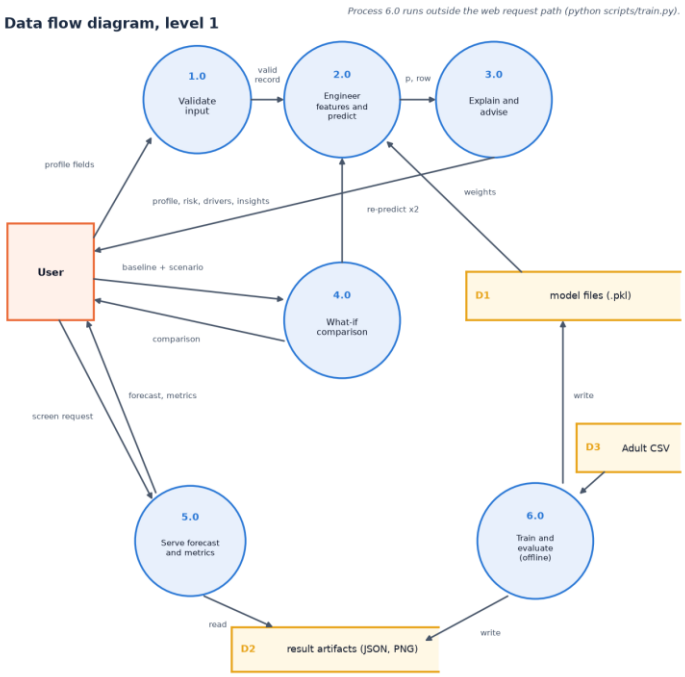



**Figure 6: Data flow diagram, level 1**

Level 1 splits the system into six processes. Validate input (1.0) turns the raw form values into a valid record or an error list. Engineer features and predict (2.0) builds the feature row and obtains the probability from the pipeline stored in D1. Explain and advise (3.0) computes SHAP contributions and plain-language insights and returns the profile, risk band, drivers and insights to the user. What-if comparison (4.0) receives a baseline and a scenario and calls process 2.0 twice. Serve forecast and metrics (5.0) answers screen requests by reading D2. Train and evaluate (6.0) runs offline: it reads the census data (D3) and writes the model files (D1) and the artifacts (D2).

***Level 2***

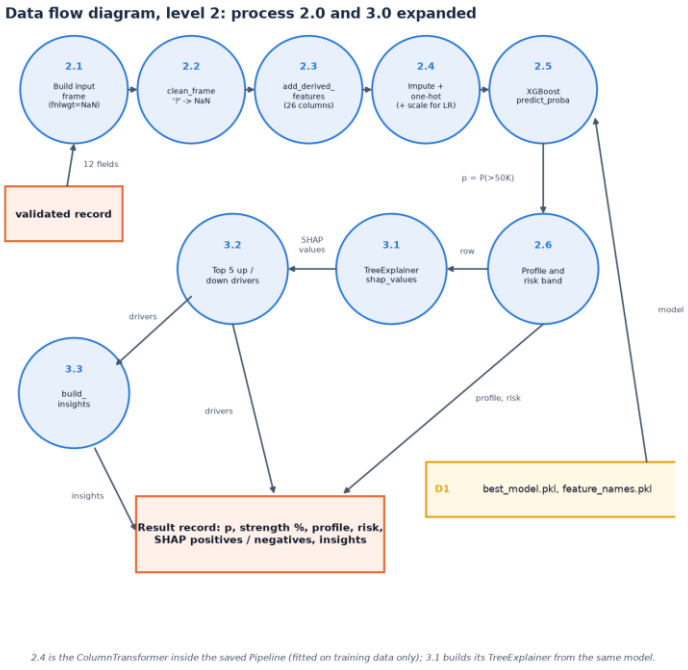



**Figure 7: Data flow diagram, level 2: processes 2.0 and 3.0 expanded**

Level 2 expands the prediction and explanation processes in the order the code executes them. The validated record becomes a one-row frame in which the census weight is left missing (2.1); `clean_frame` converts '?' markers to missing values (2.2); `add_derived_features` adds the net-capital, band, tier, region and stability columns, giving 26 columns (2.3); the preprocessing step of the saved pipeline imputes, one-hot encodes and, for logistic regression only, scales (2.4); XGBoost returns the probability (2.5). The probability becomes a profile and risk band (2.6). In parallel the same row goes to the TreeExplainer (3.1), the five strongest positive and negative contributions are selected (3.2) and the insight rules write the advice (3.3). All of it is assembled into one result record.

## 5.4 UML diagrams

### *Class diagram*

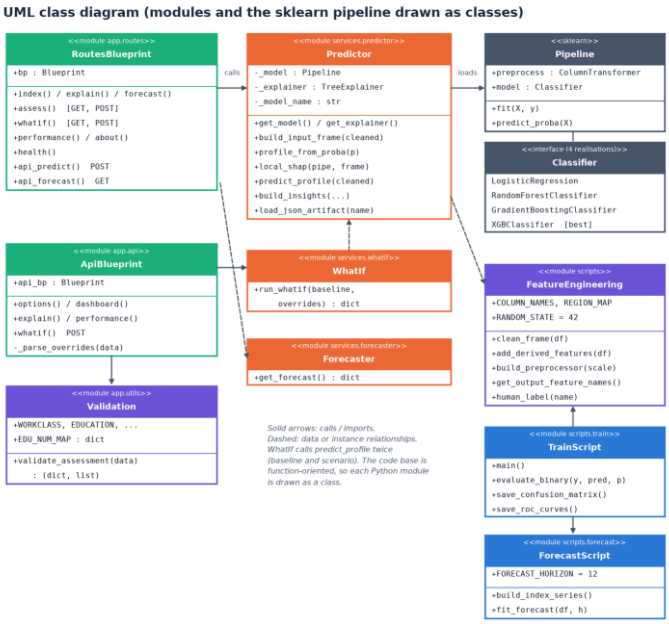



**Figure 8: UML class diagram (modules drawn as classes)**

The code base is function-oriented, so each Python module is drawn as a class stereotyped as a module, listing its constants and public functions. `RoutesBlueprint` and `ApiBlueprint` are the two Flask blueprints; both depend on `Validation` for input checking, and the routes call `Predictor`, which owns the cached model, the cached explainer and the prediction, SHAP and insight functions. `WhatIf` calls `predict_profile` twice, `Forecaster` reads the forecast artifact, and `FeatureEngineering` supplies cleaning, derived features and the preprocessor. A scikit-learn `Pipeline` combines the preprocessor with one of four classifier realisations, XGBoost being the selected one. `TrainScript` and `ForecastScript` are the offline modules.

### *Sequence diagram*

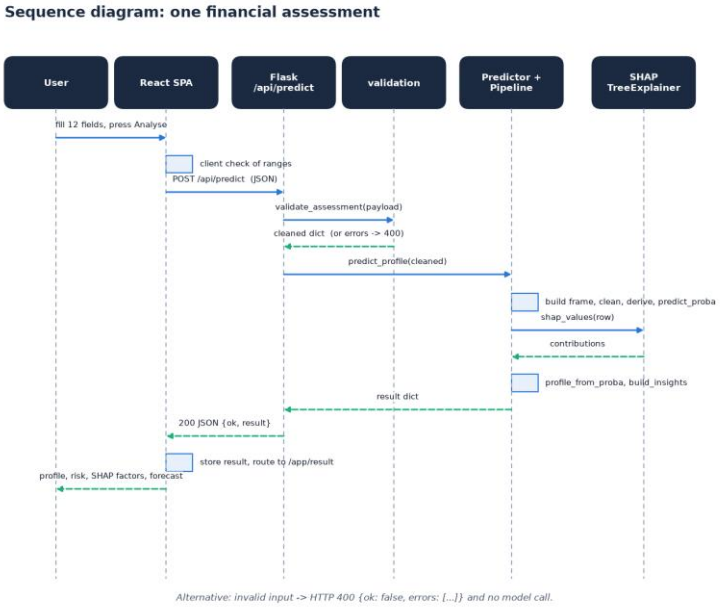



**Figure 9: Sequence diagram: one financial assessment**

The sequence follows one assessment. The user fills the twelve fields in the React page, which first checks the ranges in the browser and then posts the values as JSON to `/api/predict`. The route calls `validate_assessment`; on success it calls `predict_profile`, which builds the frame, derives the features and obtains the probability from the pipeline. It then asks the TreeExplainer for the SHAP values of that single row, maps the probability to a profile and band, and builds the insights. The result dictionary travels back as an HTTP 200 JSON body; the page stores it in the browser session and navigates to the result screen, which shows the profile, score bar, SHAP factors, insights and the forecast. Invalid input short-circuits with HTTP 400.

### *Use-case diagram*

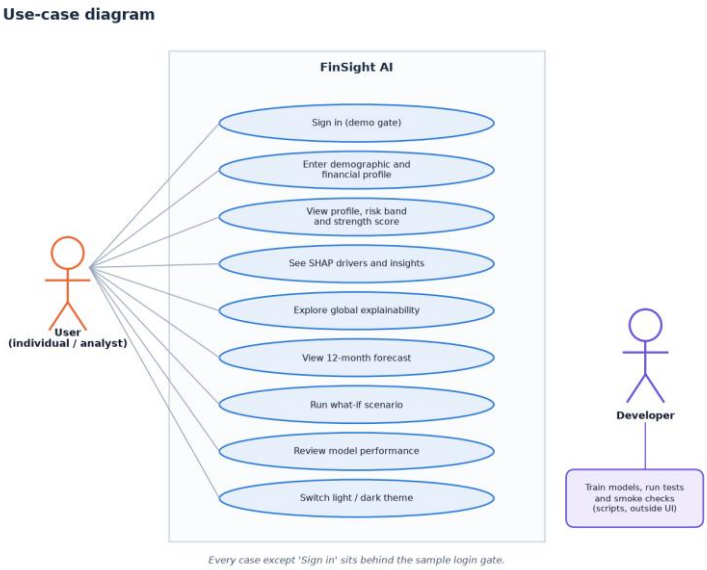



**Figure 10: Use-case diagram**

The user can sign in with the demo account, enter a demographic and financial profile, view the profile, risk band and strength score, read the SHAP drivers and insights, explore the global explainability charts, view the 12-month forecast, run a what-if scenario, review the model performance and switch between light and dark themes. Every case except signing in sits behind the sample login gate. The second actor, the developer, trains the models and runs the tests and smoke checks from the command line; those activities are outside the web interface and are drawn separately to show that the running application never retrains a model.

## 5.5 Database and data design

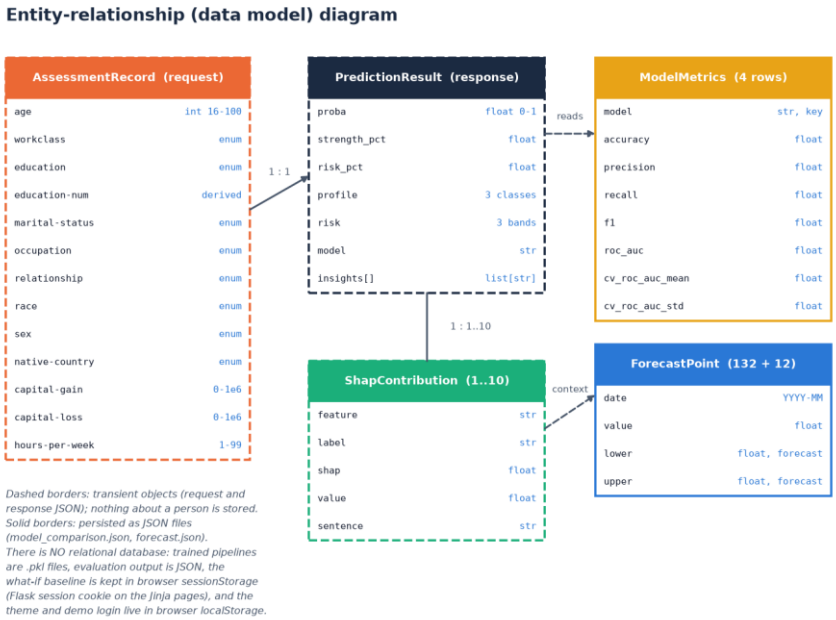



**Figure 11: Entity-relationship (data model) diagram**

FinSight AI has no relational database, so the diagram shows the data model instead of tables. Dashed boxes are transient objects that exist only inside one request: the assessment record (the twelve inputs, with education-num derived), the prediction result and the SHAP contributions, between one and ten per result. Solid boxes are persisted as JSON: the model-metrics rows (four models, with test and cross-validation scores) and the forecast points (132 history points and 12 forecast points). One request produces one result, which holds many SHAP items. Nothing about a person is written to disk; the what-if baseline lives only in browser or session storage.

The product has **no relational database**. The batch proposal mentioned MySQL, but nothing that FinSight AI handles needs to be stored: an assessment is computed from the request alone, the result is returned to the browser and then discarded by the server, and the only data that outlive a request are the trained pipelines and the evaluation artifacts, which are written once by the training script and then only read. Leaving out a database removes a whole class of risks (stored personal data, injection, schema migration) and keeps the system runnable with two commands. It also means there is no user history, no audit trail and no multi-user state; Chapter 10 lists this as future work.

**Table 20: Data model of FinSight AI (transient objects and persisted files)**

| Entity | Fields | Type | Rule | Where it lives |
|---|---|---|---|---|
| AssessmentRecord | age | integer | 16 to 100 | request body only |

| Entity | Fields | Type | Rule | Where it lives |
|---|---|---|---|---|
| AssessmentRecord | workclass, education, marital-status, occupation, relationship, race, sex, native-country | text (enumerations) | must belong to the allowed list in `validation.py` | request body only |
| AssessmentRecord | capital-gain, capital-loss | number | 0 to 1,000,000 | request body only |
| AssessmentRecord | hours-per-week | integer | 1 to 99 | request body only |
| AssessmentRecord | education-num | integer (derived) | looked up from `EDU_NUM_MAP`, 1 to 16 | added by the validator |
| PredictionResult | proba, strength_pct, risk_pct | float | 0 to 1; 0 to 100; 100 minus strength | response body, browser session |
| PredictionResult | profile, risk, model | text | Emerging / Developing / Established; Low / Moderate / High; model name | response body |
| PredictionResult | insights | list of text | 2 to 7 rule-generated sentences | response body |
| ShapContribution | feature, label, shap, value, sentence | text, text, float, float, text | up to 5 positive and 5 negative per result | response body |
| ModelMetrics | model, accuracy, precision, recall, f1, roc_auc, cv_roc_auc_mean, cv_roc_auc_std | text and floats | one row per model, four rows | `artifacts/model_comparison.json` |
| ForecastPoint | date, value, lower, upper | text and floats | 132 history points, 12 forecast points with a 95% band | `artifacts/forecast.json` |
| Pipeline | preprocess + model | scikit-learn object | one pickle per candidate and `best_model.pkl` | `models/*.pkl` |

All trained objects are stored with `joblib`. The pickles record the scikit-learn and XGBoost versions that wrote them, so loading them under a newer release prints a version warning; the metrics were re-derived with the newer release and match the stored values exactly, but a production deployment should re-train and pin versions.

## 5.6 Module design

Each module has one responsibility and the dependencies point in one direction: routes call services, services call the feature-engineering code and the saved pipeline, and nothing in the online path imports the training script. The table below lists the modules and the public functions they offer.

**Table 21: Modules of FinSight AI and their responsibilities**

| Module | Public functions | Responsibility |
|---|---|---|
| `app/__init__.py` | `create_app()` | Application factory: reads the secret key and port, preloads the model, registers both blueprints. |
| `app/routes.py` | `index`, `assess`, `explain`, `forecast`, `whatif`, `performance`, `about`, `health`, `api_predict`, `api_forecast`, error handlers | Server-rendered pages, health check, `/api/predict`, `/api/forecast`, whitelist of served artifact images. |
| `app/api.py` | `options`, `dashboard`, `explain`, `performance`, `whatif`, `_parse_overrides` | Thin read-only JSON layer for the React front end; adds `/api/whatif`. |
| `app/utils/validation.py` | `validate_assessment()`, option lists, `EDU_NUM_MAP` | Required, allowed-set and numeric-range checks; returns a cleaned dictionary or a list of messages. |
| `app/services/predictor.py` | `get_model`, `get_explainer`, `build_input_frame`, `profile_from_proba`, `local_shap`, `predict_profile`, `build_insights`, `load_json_artifact` | Prediction, profile and risk mapping, live SHAP, plain-language insights and artifact loading. |

| Module | Public functions | Responsibility |
|---|---|---|
| app/services/whatif.py | run_whatif() | Predicts a baseline and a changed scenario and reports the score change and the verdict. |
| app/services/forecaster.py | get_forecast() | Loads the forecast artifact and flags it unavailable when it is missing. |
| scripts/feature_engineering.py | clean_frame, add_derived_features, build_preprocessor, get_output_feature_names, human_label | Cleaning, 12 derived features, the leakage-safe ColumnTransformer and readable feature names. |
| scripts/models.py | get_candidate_models() | The four classifiers and their hyperparameters. |
| scripts/train.py | main, evaluate_binary, save_confusion_matrix, save_roc_curves | Offline training, cross-validation, selection, SHAP artifacts, plots, reports. |
| scripts/forecast.py | build_index_series, fit_forecast | Synthetic index series and the Holt-Winters forecast. |
| scripts/report_metrics.py | main | Re-derives confusion matrices, threshold sweep, calibration and the forecast back-test. |
| tests/, scripts/smoke_test.py | test_app.py, test_features.py | Nine automated tests and a 35-check live smoke test. |
| frontend/src/ | 11 pages, lib/, components/ | The React interface: routing, API client, theme, auth gate and charts. |

## 5.7 User interface design

The React interface follows a calm, pastel design kit: a warm off-white page background, white cards with soft shadows, six pastel accents (lavender, mint, peach, sky, rose and butter) used to tell sections apart, and Plus Jakarta Sans for headings with Inter for body text. All colours, radii, shadows and the sidebar and top-bar sizes are CSS custom properties on `:root`, and a dark theme overrides the same tokens under `data-theme="dark"`. The theme follows the operating system until the user presses the toggle, after which the choice is kept in browser storage.

The interface has eleven routes: a landing page, a sign-in page and, behind the sign-in gate, the dashboard, assessment, result, explainability, forecast, what-if, performance and about pages, plus a not-found page. Pages are loaded lazily, so the first load carries only the shell. A fixed sidebar with colour-coded icons gives navigation on screens of 900 pixels and wider; below that it is replaced by a bottom bar and a drawer, and the form and card grids collapse to one column below 1,000 pixels. Loading states use skeletons, results arrive with short spring animations, toasts confirm actions, errors are shown above the form with the offending field marked `aria-invalid`, and focus moves to the first invalid field.

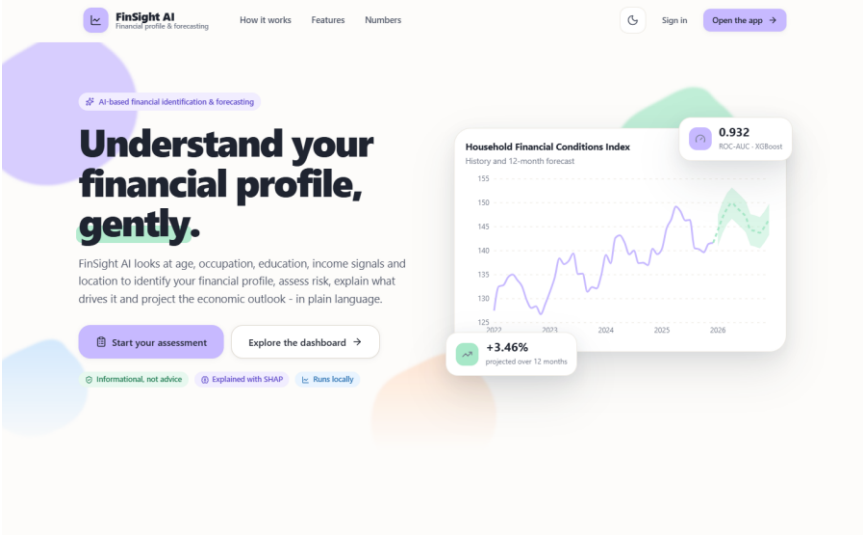



**Figure 12: Landing page of the React interface (light theme)**

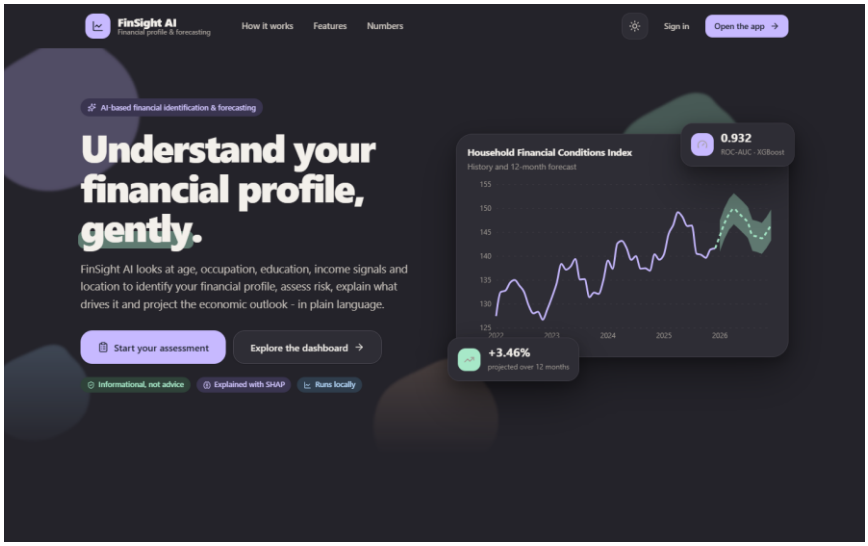



**Figure 13: The same landing page in the dark theme**

Signing in is a **demo gate**, not authentication. The login page offers a sample account (demo@finsight.ai) with a button that fills the credentials; the check runs in the browser, the user record is kept in browser storage and the guarded routes redirect signed-out visitors to the login page and return them to the page they asked for. The product stores no accounts, so there is nothing for a real login to protect; the gate exists so that the interface looks and behaves like a product, and Section 6.9 states this limit.

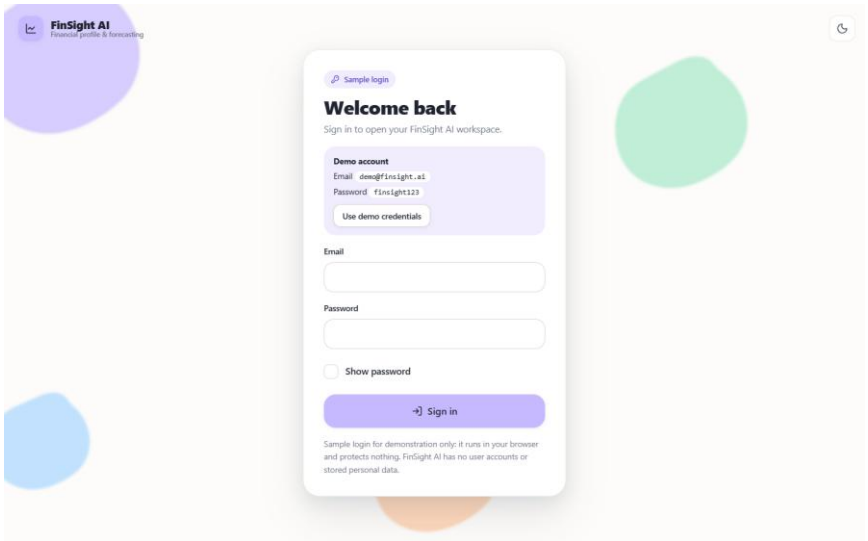



**Figure 14: Sign-in page with the sample account**

The assessment form groups the twelve inputs in a four-column grid with a label, a unit or range hint under each field, validation on submit and an option to fill a sample profile. Results are shown as a profile badge, a risk badge, a score bar, strengthening and weakening SHAP factors, actionable insights, an echo of the inputs and a mini forecast chart.

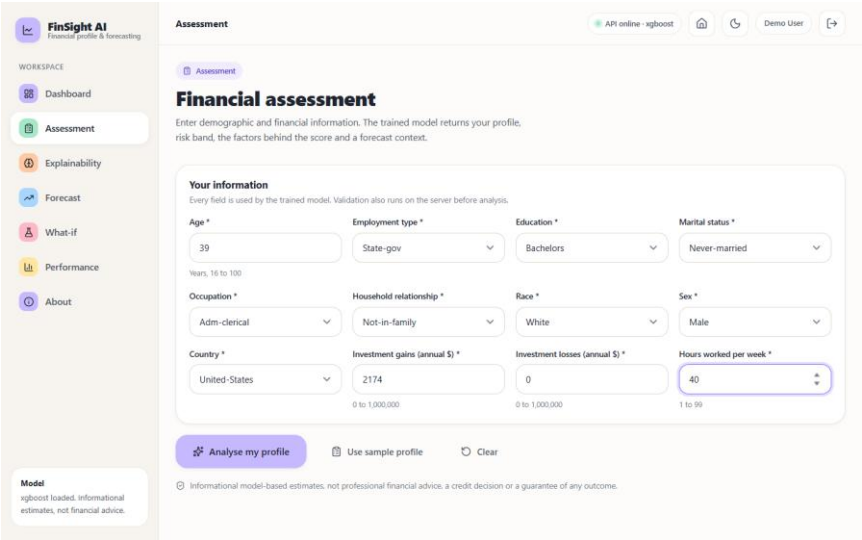



**Figure 15: Assessment form filled with a sample profile**

## 5.8 Core algorithm and training pipeline

The core algorithm is the prediction endpoint. Its decision logic is short on purpose: validate, build the frame, derive the features, call `predict_proba`, compare the probability with two fixed cut-offs, explain and return. The cut-offs 0.60 and 0.35 are design choices of this project, not learned values; on the test split they separate three groups whose observed share of income above 50K is 7.2%, 44.0% and 85.2% (Chapter 8).

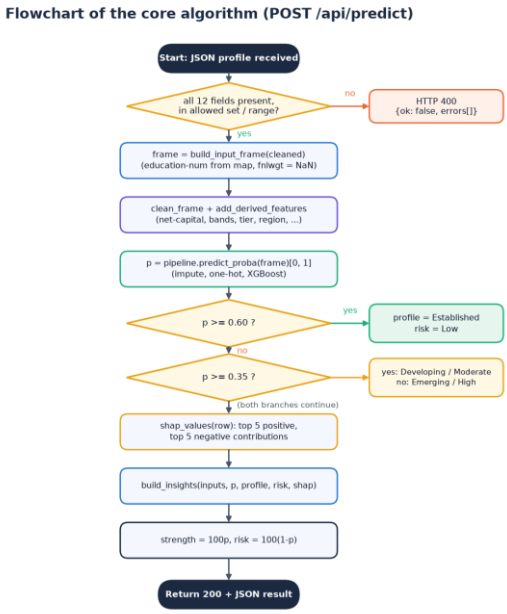



**Figure 16: Flowchart of the core algorithm (POST /api/predict)**

The flowchart gives the decision logic of the prediction endpoint. A JSON profile is received and validated; a missing, disallowed or out-of-range field returns HTTP 400 with the error list. Otherwise the input frame is built (education-num looked up from the education level, census weight left missing), the cleaning and feature-derivation functions run, and `predict_proba` returns the probability p of income above 50K. Two comparisons decide the label: p of at least 0.60 gives Established with Low risk, p of at least 0.35 gives Developing with Moderate risk,

and anything lower gives Emerging with High risk. The SHAP contributions and insights are then computed, strength is set to 100p and risk to 100(1-p), and a 200 response is returned.

Training is a separate offline script with eight steps. It loads `adult.data` and `adult.test` (48,842 rows), cleans them ('?' to missing, whitespace stripped, rows without a target dropped), engineers 26 feature columns from the 14 raw fields, and makes a stratified 80/20 split (39,073 training and 9,769 test rows). Each of four classifiers is wrapped in its own Pipeline together with its preprocessor, scored by 5-fold stratified cross-validation on the training split and then fitted and evaluated on the test split. The model with the highest test ROC-AUC is selected, and the pickles, JSON reports, charts, SHAP importance and forecast are persisted for the web application to load.

# CHAPTER 6: METHODOLOGY AND SOFTWARE IMPLEMENTATION

## 6.1 Overall methodology

The methodology adapts the four-stage explainable machine-learning framework of the base paper [1] (domain-adaptive feature engineering, multi-model training, SHAP interpretation and domain-specific validation) from organisations to individuals, and adds the parts that turn a model into a product: input validation, a forecast, a what-if comparison and a web interface. The work has two halves. An offline half prepares data, trains and evaluates models and writes artifacts; an online half loads those artifacts and answers one request at a time.

1. **Acquire** the UCI Adult / Census Income files (48,842 individual records) and a monthly economic index series for forecasting.
2. **Clean** the records: strip whitespace, turn '?' into missing values, coerce numeric columns and drop rows without a target.
3. **Engineer** twelve interpretable derived features (net capital, capital intensity, log scales, flags, age and hours bands, education tier, region, employment stability, capital position).
4. **Split** the rows with a stratified 80/20 split (random state 42) and keep the 20% for the final test.
5. **Train** four classifiers, each inside a scikit-learn Pipeline whose preprocessing is fitted on training data only.
6. **Validate** with 5-fold stratified cross-validation on the training split, then evaluate on the held-out test split.
7. **Select** the model with the highest test ROC-AUC and persist all pipelines, metrics, plots, SHAP importance and the forecast.
8. **Serve**: validate an input record, derive features, predict, map the probability to a profile and risk band, compute local SHAP contributions, write plain-language insights, and optionally compare a what-if scenario.
9. **Forecast** a 12-month horizon of the economic index with Holt-Winters exponential smoothing and a 95% band.

**Table 22: Stages of the base-paper framework and how this project realises them**

| Base-paper stage | Base paper (hospitals) | This project (individuals) |
|---|---|---|
| 1. Domain-adaptive feature engineering | 236 line items reduced to 11 ratio, scale and categorical features | 14 raw census fields extended to 26 columns with 12 derived features; 145 columns after one-hot encoding |
| 2. Multi-model training | LR, RF, GB, XGBoost, MLP at default settings | LR, RF, GB, XGBoost; fixed hyperparameters, no search; no neural baseline |
| 3. SHAP interpretation | Global importance and beeswarm; cost ratios dominate | TreeExplainer: global top-15 at training time and live local contributions for every request |
| 4. Domain-specific validation | Hospital-level Group K-Fold, temporal hold-out, counterfactuals, panel regression | Stratified 5-fold CV and 80/20 test split (one row per person); logistic statistics; what-if comparison |

## 6.2 Dataset

The classification data are the Adult (Census Income) extract [32], which originates in the 1994 US census database and was prepared for classification by Kohavi [31]. The project uses both published files, `adult.data` (32,561 rows) and `adult.test` (16,281 rows), joined into 48,842 individual records with 15 columns. The positive class, income above 50K per year, covers 23.93% of the rows. Missing values are written as '?' in three columns: workclass (2,799 cells), occupation (2,809) and native-country (857). The records are public and anonymous, and no personal data were collected for this project.

The income label is used as a **financial-profile signal**. It is not a credit score, a repayment record or a distress indicator, and the report never claims otherwise. This is the main difference from the base paper, whose label is a hospital deficit derived from net income [1], and the reason the comparison in Chapter 8 is given as context only.

**Table 23: Facts about the classification dataset**

| Item | Value |
|---|---|
| Source | UCI Machine Learning Repository, Adult (Census Income), 1994 US census |
| Files used | `data/raw/adult.data` (32,561 rows) and `data/raw/adult.test` (16,281 rows) |
| Records after joining | 48,842 rows, 15 columns (14 inputs and the target) |
| Target | income above 50K (positive) versus 50K or less; test labels carry a trailing '.' that the cleaner removes |
| Positive-class share | 23.93% |
| Training / test rows | 39,073 / 9,769 (stratified 80/20, random state 42) |
| Test class balance | 7,431 at 50K or less, 2,338 above 50K |
| Form option counts | workclass 8, education 16, marital status 7, occupation 14, relationship 6, race 5, sex 2, country 41 |
| Not available | transactions, balances, credit history, repayment records, time stamps |

The Adult data are cross-sectional: every person appears once and there is no date. They therefore cannot be used to forecast the future. A second, separate series is used for the forecasting component: a monthly **Household Financial Conditions Index** from January 2015 to December 2025 (132 points) generated by `build_index_series()` with a fixed seed. The index is the sum of a linear trend (100 plus 0.35 per month), a yearly seasonal term of amplitude 4, a six-month downturn of 7 points from March to August 2020 and Gaussian noise with standard deviation 1.6. The series is **synthetic**, documented as such in `data/README.md`, and chosen so that the forecasting method can be demonstrated and tested end to end; it carries no information about any real economy, and the abstract's real indicators (inflation, lending rates, GDP) are not integrated.

## 6.3 Preprocessing and feature engineering

Preprocessing is split in two places on purpose. Operations that need no fitted state (cleaning and derived features) are plain functions in `scripts/feature_engineering.py` and are applied to all rows before the split. Operations that learn from data (imputation values, category lists, scaling) live in a `ColumnTransformer` inside each Pipeline and are therefore fitted on training rows only.

`clean_frame` strips whitespace from text columns, converts '?' to a missing value, coerces `age`, `fnlwgt`, `education-num`, `capital-gain`, `capital-loss` and `hours-per-week` to numbers and removes the trailing '.' from test-file labels. `add_derived_features` then adds twelve columns, each computable from the raw fields of a new user:

**Table 24: Derived features added by add_derived_features()**

| Feature | Rule | Reason |
|---|---|---|
| net-capital | capital-gain minus capital-loss | single investment-position signal |
| capital-intensity | capital-gain plus capital-loss, at least 0 | overall investment activity |
| log-capital-gain, log-capital-loss | log(1 + value) | both are highly skewed; helps the linear model |
| has-capital-gain, has-capital-loss | 1 if the value is above 0, else 0 (as text) | interpretable yes / no indicators |
| hours-band | up to 29 Part-time; 30-40 Full-time; 41-60 Extended; above 60 Very-long | earning-effort proxy |
| age-band | up to 25; 26-35; 36-50; 51-65; above 65 | life-stage proxy |
| education-tier | education-num up to 9 Secondary-or-below; 10-12 Higher-secondary; 13 or more Tertiary | groups 16 levels into 3 |
| region | native-country mapped to North America, Latin America, Europe, Asia, Middle East or Other | regional economic category |
| employment-stability | government Stable-public; Self-emp-inc Business-owner; Self-emp-not-inc Self-employed; Private with 35 or more hours Stable-private, otherwise Variable-private; Without-pay or Never-worked Not-employed | employment security proxy |
| capital-position | net-capital below 0 Net-loss; 0 No-capital; 1 to 4,999 Moderate-gain; 5,000 or more High-gain | categorical view of net-capital |

The result has 26 columns: 10 numeric (`age`, `fnlwgt`, `education-num`, `capital-gain`, `capital-loss`, `hours-per-week`, `net-capital`, `capital-intensity` and the two log columns) and 16 categorical. In the ColumnTransformer, numeric columns receive median imputation (and standard scaling for logistic regression only, because trees are insensitive to scale) and categorical columns receive imputation with the constant "Unknown" followed by one-hot encoding with `handle_unknown="ignore"`, so a category never seen in training cannot crash a request. After encoding the fitted pipeline outputs **145 columns**.

```
def build_preprocessor(scale_numeric: bool) -> ColumnTransformer:
    """Build a leakage-safe ColumnTransformer.


    scale_numeric=True for the Logistic Regression pipeline;
    tree pipelines use scale_numeric=False.
    """
    if scale_numeric:
        num_pipe = Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ])
    else:
        num_pipe = Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
        ])
    cat_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
        ("onehot", _onehot()),
    ])
    return ColumnTransformer([
        ("num", num_pipe, NUMERIC_FEATURES),
        ("cat", cat_pipe, CATEGORICAL_FEATURES),
    # ... (listing shortened; full source in the repository)
```

The census sampling weight `fnlwgt` describes how many people a record represents and has no meaning for a new user. It is kept as a training feature, as in the original data, and left missing at prediction time, where the pipeline imputes the training median; its small SHAP importance (0.101) is reported rather than hidden.

## 6.4 Algorithm choice and justification

Four classifiers span the range used by the base paper [1]: a linear baseline, a bagging ensemble and two boosting ensembles. Tree ensembles are the natural choice for mixed, tabular data with interactions and skewed numeric columns, and Grinsztajn et al. [23] show that they still outperform deep networks on typical tabular data, which is why no neural network was trained. Explainability, the fifth objective, favours models for which exact attributions exist, which led to SHAP.

**Table 25: Methods used and the reason for each**

| Method | Role | Why chosen | Known limitation |
|---|---|---|---|
| Logistic regression [27] | Interpretable baseline (scaled inputs) | Coefficients readable; performance floor | Cannot represent interactions |
| Random forest [7] | Bagged-tree comparator | Robust, little tuning, captures non-linearity | Less accurate than boosting here; large model |
| Gradient boosting [8] | Boosted-tree comparator (scikit-learn) | Sequential error correction; strong on tabular data | Fewer regularisation options than XGBoost |
| XGBoost [9] | Primary classifier | Regularised boosting; best ROC-AUC in this study; exact TreeExplainer support | Needs care with extrapolation; many hyperparameters (not tuned) |
| SHAP TreeExplainer [14], [15] | Global and local explanation | Shapley-value attributions, exact and fast for trees; consistent with the base paper | Associational, not causal; correlated features share credit |
| Holt-Winters smoothing [33], [34] | 12-month forecast | Simple, interpretable trend and seasonality; fits a single monthly series | Assumes the pattern continues; interval is constant in width |
| Stratified K-fold [26] | Validation | Keeps the 24% positive share in every fold | Not entity-aware (unnecessary: one row per person) |

SHAP was preferred to LIME [16] for the reasons the base paper gives: Shapley values satisfy efficiency, symmetry and additivity, TreeExplainer computes them exactly for tree models, and one method supplies both the global ranking and the per-person explanation; comparisons of the two in credit risk [18] and applications to credit decisions [17] point the same way. The base paper's caution also applies here: SHAP attributions describe what the model relies on, not what would happen if a person changed a value.

## 6.5 Mathematical formulation

**Logistic regression** models the log-odds of the positive class as a linear function of the encoded and scaled feature vector x:

$$P(y = 1 \mid x) = 1 / (1 + \exp( -(\beta 0 + \beta^{T} x) )) \quad (1)$$

**Gradient boosting** builds an additive model of M shallow trees, each fitted to the gradient of the loss at the current prediction, with learning rate ν. **XGBoost** adds an explicit regularisation term to the training objective, where T is the number of leaves of a tree and w its leaf weights:

$$F(x) = \Sigma_{m=1..M} \nu \cdot f_m(x), \quad P(y = 1 \mid x) = 1 / (1 + \exp(-F(x))) \quad (2)$$

$$L = \Sigma_i l(y_i, \hat{y}_i) + \Sigma_m \Omega(f_m), \quad \Omega(f) = \gamma \cdot T + \tfrac{1}{2} \cdot \lambda \cdot \|w\|^2 \quad (3)$$

In this project λ = 1 (the library default L2 weight written explicitly), the learning rate is 0.05, 400 trees of depth at most 6 are grown, and each tree sees 90% of the rows and 90% of the columns.

**Shapley values** attribute a prediction to features. For a model f with feature set N of size M, the contribution of feature i is the weighted average of its marginal effect over all feature subsets S:

$$\varphi_i = \Sigma_{S \subseteq N \setminus \{i\}} \; [\,|S|!\,(M - |S| - 1)! \,/\, M!\,] \cdot [\, f(S \cup \{i\}) - f(S)\,] \qquad (4)$$

$$f(x) = \varphi_0 + \Sigma_{i=1..M} \varphi_i \quad \text{(local accuracy; for XGBoost the sum is in log-odds)} \qquad (5)$$

For the selected model the base value φ_0 is -1.1765 in log-odds, about a 24% probability, which is the share of positive records. The global importance of a feature is the mean of |φ_i| over a sample of 500 training rows; the local explanation of one request uses the five largest positive and five largest negative φ_i.

**Profile mapping.** The predicted probability p of income above 50K is the financial-strength score. It is converted to a profile and a risk band by two fixed cut-offs chosen for this project:

$$(\text{profile, risk}) = (\text{Established, Low}) \text{ if } p \geq 0.60;\;\; (\text{Developing, Moderate}) \text{ if } 0.35 \leq p < 0.60;\;\; (\text{Emerging, High}) \text{ otherwise} \qquad (6)$$

$$\text{strength} = 100 \cdot p, \quad \text{risk score} = 100 \cdot (1 - p) \qquad (7)$$

**Holt-Winters (additive trend and additive seasonality)** keeps a level ℓ, a trend b and m = 12 seasonal terms s, updated with smoothing weights α, β and γ that are estimated by optimisation:

$$\ell_t = \alpha\,(y_t - s_{t-m}) + (1 - \alpha)(\ell_{t-1} + b_{t-1}) \qquad (8)$$

$$b_t = \beta\,(\ell_t - \ell_{t-1}) + (1 - \beta)\, b_{t-1} \qquad (9)$$

$$s_t = \gamma\,(y_t - \ell_{t-1} - b_{t-1}) + (1 - \gamma)\, s_{t-m} \qquad (10)$$

$$\hat{y}_{t+h} = \ell_t + h \cdot b_t + s_{t+h-m(k+1)}, \quad k = \lfloor (h - 1)/m \rfloor \qquad (11)$$

The 95% band used by the application is $\hat{y} \pm 1.96 \cdot \sigma$, where σ is the standard deviation of the in-sample residuals (1.659). It is an approximation with constant width, not a full prediction interval whose width grows with the horizon [36]. The synthetic series itself is $y_t = 100 + 0.35t + 4\sin(2\pi(\text{month}_t - 1)/12) + \text{shock}_t + \varepsilon_t$ with $\varepsilon_t \sim N(0, 1.6^2)$.

**Evaluation metrics** are computed from the confusion matrix (TP, FP, TN, FN) of the positive class, income above 50K:

$$\text{accuracy} = (TP + TN) / (TP + TN + FP + FN);\;\; \text{precision} = TP / (TP + FP);\;\; \text{recall} = TP / (TP + FN);\;\; F1 = 2 \cdot \text{precision} \cdot \text{recall} / (\text{precision} + \text{recall}) \qquad (12)$$

$$\text{ROC-AUC} = P(\,\text{score(positive)} > \text{score(negative)}\,),\;\; \text{the area under the TPR-versus-FPR curve} \qquad (13)$$

As a check, the selected model's test confusion matrix is TN 7,021, FP 410, FN 780, TP 1,558, so precision is 1,558 / 1,968 = 0.7917, recall is 1,558 / 2,338 = 0.6664 and accuracy is 8,579 / 9,769 = 0.8782, matching the stored report. ROC-AUC [28], [29] equals the probability that a randomly chosen positive record receives a higher score than a randomly chosen negative one.

## 6.6 Model training and selection

Training is performed by `python scripts/train.py`. The four candidates are defined in `scripts/models.py` with fixed hyperparameters and `random_state = 42`; no grid or random search was run, which follows the base paper's choice of defaults for reproducibility [1] but means the models may be under-tuned.

**Table 26: Hyperparameters of the four candidate classifiers**

| Model | Settings |
|---|---|
| Logistic Regression | C = 1.0, max_iter 2000; numeric inputs standardised |
| Random Forest | 300 trees, unlimited depth, min_samples_split 2, all cores |

| Model | Settings |
|---|---|
| Gradient Boosting | scikit-learn defaults (100 trees, depth 3, learning rate 0.1) |
| XGBoost | 400 trees, max_depth 6, learning_rate 0.05, subsample 0.9, colsample_bytree 0.9, reg_lambda 1.0, logloss |

Each candidate is wrapped as `Pipeline([preprocess, model])`. The 5-fold stratified cross-validation (shuffled, seed 42) is run on the 39,073 training rows with ROC-AUC as the score; the pipeline is then fitted on all training rows and evaluated once on the 9,769 test rows at the 0.5 threshold. The best model is the one with the highest **test** ROC-AUC. Selecting on the test split slightly favours the winner and no separate validation set was held out; the cross-validation ranking is identical, which reduces but does not remove that concern. The time taken to train was not recorded.

**Table 27: Test-split metrics and cross-validated ROC-AUC of the four models**

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC | CV ROC-AUC (mean +/- sd) |
|---|---|---|---|---|---|---|
| Logistic Regression | 0.8599 | 0.7568 | 0.6108 | 0.6760 | 0.9140 | 0.9133 +/- 0.0013 |
| Random Forest | 0.8597 | 0.7402 | 0.6373 | 0.6849 | 0.9100 | 0.9079 +/- 0.0016 |
| Gradient Boosting | 0.8697 | 0.7915 | 0.6185 | 0.6944 | 0.9247 | 0.9204 +/- 0.0022 |
| **XGBoost** | 0.8782 | 0.7917 | 0.6664 | 0.7236 | 0.9319 | 0.9293 +/- 0.0011 |

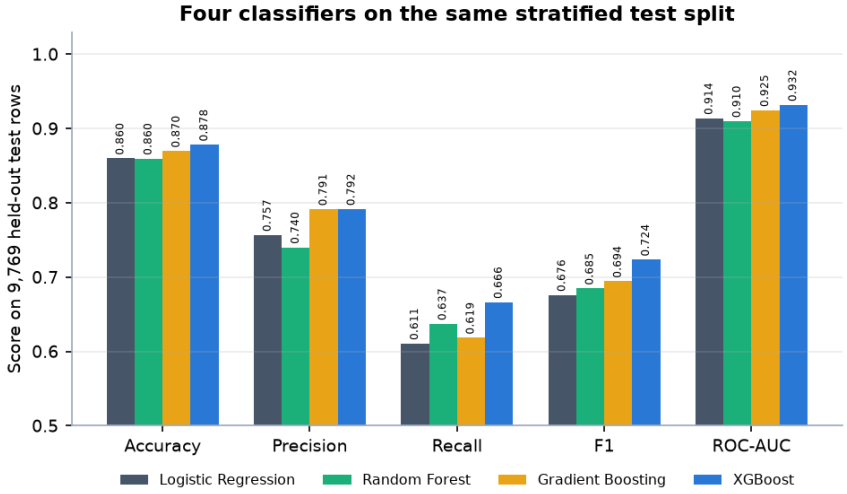



**Figure 17: The four classifiers compared on the held-out test split**

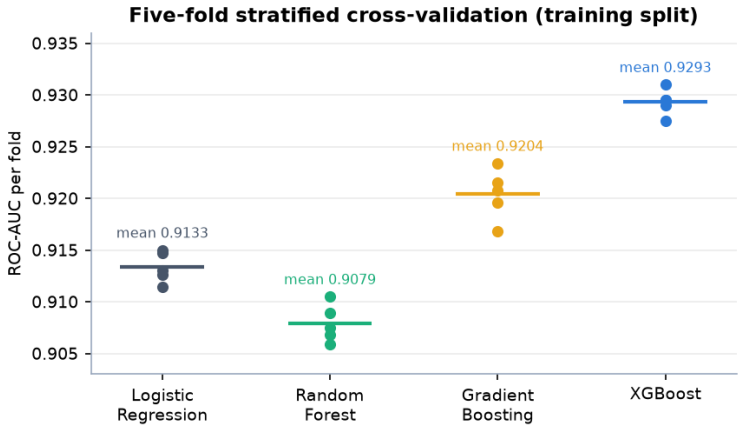



**Figure 18: ROC-AUC of every cross-validation fold for each model**

XGBoost leads on every metric, with Gradient Boosting a close second (precision 0.7915 against 0.7917); the spread between folds is at most 0.0035 AUC for XGBoost, so the ranking is stable across folds. Chapter 8 gives the confusion matrices and the threshold analysis.

## 6.7 Application architecture and what-if logic

`create_app()` builds the Flask application, reads the session secret from the environment, preloads the persisted pipeline so that the first request is as fast as later ones, and registers the two blueprints. A failed preload is logged as a warning and the application still starts; `/health` then reports `degraded` and prediction routes return an explanatory error until the model is trained.

`predict_profile(cleaned)` is the heart of the service. It builds a one-row frame from the validated record, applies `clean_frame` and `add_derived_features`, calls `predict_proba` on the cached pipeline, maps the probability with `profile_from_proba`, asks `local_shap` for the contributions and `build_insights` for the advice, and returns one dictionary. The TreeExplainer is built lazily on first use with `feature_perturbation="tree_path_dependent"`, which needs no background data and keeps each request fast.

```
def profile_from_proba(p: float) -> tuple:
    """Map P(income>50K) to (profile, risk_level)."""
    if p >= 0.60:
        return "Established", "Low"
    if p >= 0.35:
        return "Developing", "Moderate"
    return "Emerging", "High"
```

`local_shap` transforms the row with the pipeline's preprocessor, computes SHAP values for the single row, keeps the eight most negative and eight most positive, and returns the five strongest of each sign. Each item carries a human-readable label and a sentence. One-hot columns need care: for a feature such as `occupation = Sales` a value of 0 means the category does not apply, so the sentence reads "absence of occupation = Sales ..." instead of claiming the opposite. Failures are caught and returned as a note, so an explanation problem never blocks the prediction.

`build_insights` is rule-based and independent of SHAP: one message per profile, extra messages when weekly hours are below 35, when gains or losses are reported or when education is at or below high-school level, a sentence quoting up to four SHAP drivers and always a closing disclaimer. Because the rules look only at the inputs, an insight can disagree with the SHAP sign for the same person (for a never-married 39-year-old with gains of 2,174 the rule says gains are a positive signal while SHAP assigns them a contribution of -1.63, because the rule is fixed text keyed on the input and SHAP reflects what the model learned). This is a known weakness of mixing two explanation sources and is listed as future work.

**What-if comparison.** `run_whatif(baseline, overrides)` predicts the baseline, copies the record, replaces the overridden fields (investment gains, investment losses, weekly hours, education with its numeric equivalent), predicts again and reports the change in probability. A change above +0.02 is described as an improvement, below -0.02 as a decline and anything in between as roughly unchanged; the response also lists the changed fields and whether the profile or risk band changed. The verdict always ends with the warning that this is a model simulation, not a causal prediction. At least one scenario field must be supplied, and the scenario values obey the same ranges as the form. The React pages keep the baseline in the browser session; the original Flask pages keep it in the Flask session cookie.

**Forecast service.** `get_forecast()` loads `artifacts/forecast.json` (history, fitted values, 12 forecast points with lower and upper bounds, direction, percentage change and an interpretation sentence) and returns an `available: false` structure if the file is missing, so the interface degrades gracefully.

**Front end.** The React application talks to the API through one small client (`lib/api.js`) that wraps `fetch`, converts failures into an `ApiError` carrying the server's message list, and exposes one function per endpoint. Pages load data with a `useAsync` hook and show a skeleton, an error message or the content. Charts are drawn with Recharts and follow the active theme. In development and preview, Vite proxies `/api`, `/artifacts` and `/health` to port 8409, so the browser talks to one origin and no cross-origin configuration is needed.

## 6.8 API design

All routes are served by Flask on port 8409 (and through the Vite proxy on 5409). JSON routes return `{ok, result}` on success and `{ok: false, errors: [...]}` with status 400 or 500 on failure. The page routes return HTML and are kept for the original interface.

**Table 28: Endpoints of FinSight AI**

| Method and path | Request | Response | Errors |
|---|---|---|---|
| POST `/api/predict` | JSON with the 12 assessment fields | `{ok, result}`: proba, strength_pct, risk_pct, profile, risk, model, inputs, shap, insights | 400 validation list; 500 model missing or failure |
| POST `/api/whatif` | the 12 fields plus `scenario_capital_gain`, `scenario_capital_loss`, `scenario_hours`, `scenario_education` (at least one) | `{ok, result}`: baseline, scenario, changed, delta, delta_pct, verdict, profile_changed, risk_changed | 400 for bad input, no scenario field or out-of-range value; 500 |
| GET `/api/forecast` | none | history, fitted, 12 forecast points with bounds, method, trend, interpretation | always 200; `available: false` if missing |
| GET `/api/options` | none | option lists for the seven dropdowns and the numeric limits | - |
| GET `/api/dashboard` | none | model name, metrics, top-5 SHAP, outlook, training summary | - |
| GET `/api/explain` | none | global SHAP importance (top 15) | - |
| GET `/api/performance` | none | model comparison, validation report, statistics, training summary | - |
| GET `/health` | none | status ok or degraded, model name, model_loaded, forecast_available | - |
| GET `/artifacts/<name>` | one of four PNG names | confusion matrix, ROC, SHAP bar or beeswarm image | 404 for any other name |
| GET, POST `/`, `/assess`, `/explain`, `/forecast`, `/whatif`, `/performance`, `/about` | form fields for the two POST pages | server-rendered HTML pages | 400 / 500 re-render the form with messages; 404 page |

Example of a successful `/api/predict` response for the first record of the Adult file (a 39-year-old state employee with a bachelor's degree, never married, gains of 2,174, 40 hours per week); the arrays are abbreviated:

```
{"ok": true,
 "result": {"proba": 0.0109, "strength_pct": 1.1, "risk_pct": 98.9,
            "profile": "Emerging", "risk": "High", "model": "xgboost",
            "inputs": {...12 fields + education-num...},
            "shap": {"available": true, "expected_value": -1.1765,
                     "positives": [{"feature": "education-num", "label": "Years of education",
                                    "shap": 0.545, "value": 13.0, "sentence": "..."}, ...],
                     "negatives": [{"feature": "capital-gain", "shap": -1.6264, ...}, ...]},
            "insights": ["...", "..."]}}
```

Example of a failed request with the age set to -5: HTTP 400 and `{"ok": false, "errors": ["Age must be between 16 and 100."]}`.

## 6.9 Security and error handling

The application is a local research prototype and its security is deliberately modest; the measures that exist are listed together with their limits.

- **Input validation.** `validate_assessment` requires every field, restricts the seven categorical fields to fixed allowed lists and checks the numeric fields for type and range (age 16-100, hours 1-99, gains and losses 0-1,000,000). Because values are only ever compared with allowed lists or converted to numbers and the application has no database, shell call or template that echoes raw input into code, injection through the form is not a realistic path.
- **Error paths.** Invalid input returns HTTP 400 with every message at once; a missing model file or a failure inside prediction returns 500 with a safe message while the stack trace goes to the log; unknown URLs and server errors render a friendly error page; the React client shows the first message in a toast and the full list above the form.
- **What is exposed.** `/artifacts/<name>` serves only four whitelisted PNG names; the process binds to 127.0.0.1, so it is reachable only from the same machine; Flask debug mode is off.
- **No personal data is stored.** Inputs are used for one computation; the what-if baseline is held in the browser session (React) or the signed Flask session cookie (original pages).
- **Demo login.** The sign-in gate is client-side and its credentials are shown on the login page. It is not authentication and protects nothing; real accounts, hashed passwords and server-side sessions are future work.
- **Secrets.** The Flask secret key is read from `FINSIGHT_SECRET_KEY` and falls back to a fixed development value, which must be replaced in any shared deployment.
- **Transport and abuse.** Traffic is plain HTTP on localhost, there is no rate limiting, and the Flask development server is not meant for production; a WSGI server and a reverse proxy with TLS would be required.

## 6.10 Installation and execution

The project needs Python 3.11 or newer and Node.js 18 or newer. The measured environment was Windows 11 Pro, Intel Core i5-12500 (12th Gen), 15.7 GB RAM, Python 3.11.9, scikit-learn 1.9.1, XGBoost 3.2.0, SHAP 0.51.0, statsmodels 0.15.0, Flask 3.1.3 and Node 24.19.0; no GPU is used. `requirements.txt` lists 12 packages (pandas, numpy, scikit-learn, xgboost, shap, statsmodels, Flask, joblib, matplotlib, pytest, requests and fpdf2).

**Automatic.** Double-clicking `run.bat` reads the ports from `.env` (API 8409, front end 5409), creates `.venv`, installs the requirements, trains the models if `models/best_model.pkl` is missing, runs `npm install` and `npm run build` on the first run, starts the API in a second window, opens the browser and serves the built front end with `npm run preview`.

**Manual.**

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python scripts\train.py        # only if models\best_model.pkl is missing
python run.py                   # API on http://localhost:8409
```

```
cd frontend
npm install
npm run dev                     # http://localhost:5409 (proxies /api to 8409)
# production-style:  npm run build  then  npm run preview
```

Sign in with the sample account `demo@finsight.ai` and password `finsight123` (the login page has a button that fills them). The original server-rendered pages remain available at http://localhost:8409.

**Checks.** `pytest` runs 9 tests (seven in `tests/test_app.py` covering health, page loading, input-dependent predictions, validation errors, the JSON API, the what-if route and artifact presence, and two in `tests/test_features.py` covering the derived features and missing-value handling) in about 9 seconds. With both servers running, `python scripts/smoke_test.py` performs 35 live checks of the API and the proxy, and `python scripts/capture_screens.py` walks every screen with Playwright and fails on any console error. A single prediction takes a median of 68.9 ms (95th percentile 116.9 ms) and a what-if comparison 136.1 ms when measured on 127.0.0.1 over 40 calls.

# CHAPTER 7: SYSTEM TESTING

## 7.1 Testing strategy

FinSight AI is a small system whose value rests on three properties: the model returns different, plausible answers for different people; invalid input is refused before it reaches the model; and the explanations and forecast shown on screen are the ones the code produced. The testing strategy follows from this. Small deterministic pieces (feature engineering, the preprocessing pipeline) are covered by unit tests; the Flask application is exercised through its test client with the real trained model; the running stack is checked end to end by a smoke test that talks to the live servers; and hostile or malformed inputs are probed against the live API by a dedicated script written for this chapter.

All results in this chapter were produced on 2026-10-22 on the development machine (Intel Core i5-12500 (12th Gen), 15.7 GB RAM, Windows 11 Pro), using Python 3.11.9, scikit-learn 1.9.1, XGBoost 3.2.0 and Flask 3.1.3. The persisted model files were written by an older scikit-learn and XGBoost, which emit version warnings when loaded; the model nevertheless reproduces the stored metrics exactly (Section 8.2), so the warnings were treated as cosmetic and noted here for completeness.

**Table 29: Test layers used**

| Layer | Tool | What it checks | Result |
|---|---|---|---|
| Unit | pytest | Derived-feature arithmetic, missing-value handling by the preprocessing pipeline | 2 tests passed |
| Integration and functional | pytest + Flask test client | All pages, validation, prediction through the real model, JSON API, what-if, artifact files | 7 tests passed |
| System smoke | scripts/smoke_test.py (requests) | The running API on port 8409 and the Vite proxy on port 5409: every JSON route, artifact images, predictions, validation, what-if, SPA deep links | 35/35 checks passed |
| Edge and security probe | scripts/edge_cases.py (requests) | 29 malformed, boundary and hostile requests to /api/predict and /api/whatif and 7 path probes | Section 7.5 and 7.6 |
| Performance | Python timing loops | Median and 95th-percentile response time of five routes, 40 calls each | Section 7.7 |

The automated suite contains **9 pytest tests**; all 9 passed in 9.4 seconds, with 0 failures.

## 7.2 Unit testing

Unit tests live in `tests/test_features.py` and target `scripts/feature_engineering.py`, the module that turns a raw record into the 26-column frame the model expects. Because the same functions run at training time and at prediction time, a mistake here would silently corrupt every prediction, so the arithmetic is pinned down with exact expectations.

**Table 30: Unit tests**

| Test | Input | What it asserts |
|---|---|---|
| `test_derived_features_computable` | One complete record: age 40, Private, Bachelors, married, Exec-managerial, gains 5,000, 45 hours, United-States | net-capital equals 5,000; region is 'North America'; education-tier is 'Tertiary'; age-band is '36-50'; employment-stability is 'Stable-private' |

| Test | Input | What it asserts |
|---|---|---|
| `test_missing_values_handled_by_pipeline` | Two records, the first with age missing and '?' in workclass, marital status, occupation and country | `clean_frame` and `add_derived_features` run, the scaled preprocessor fits and transforms, the output has two rows and contains no missing value |

The second test matters because the real Adult data use the character '?' for missing values (2,799 workclass, 2,809 occupation and 857 country entries in the raw file) and a user may also leave a field empty; the test confirms that the imputation steps inside the Pipeline absorb both without error.

## 7.3 Integration testing

`tests/test_app.py` builds the real Flask application with `create_app()`, so the trained model, the SHAP explainer and the forecast artifact are loaded exactly as in production. It contains seven tests.

**Table 31: Integration and functional tests**

| Test | What it asserts |
|---|---|
| `test_health` | GET /health returns 200 and a status of 'ok' or 'degraded' |
| `test_pages_load` | The seven server-rendered pages (/, /assess, /explain, /forecast, /whatif, /performance, /about) all return 200 |
| `test_valid_prediction_not_hardcoded` | A valid assessment returns the 'Financial-strength score' page; a second, very different profile (age 60, Doctorate, gains 50,000, 60 hours) produces a different page, proving the output is not fixed text |
| `test_invalid_input_rejected` | An empty form, a negative age and an unknown workclass each return 400 |
| `test_api_predict_and_forecast` | POST /api/predict returns ok with a probability between 0 and 1 and live SHAP available; GET /api/forecast reports available data with 12 forecast points |
| `test_whatif_changes_score` | A what-if post with scenario gains 80,000 and hours 50 returns 200 and the 'Comparison' section |
| `test_artifacts_present_and_real` | Four models in the comparison file with a valid best model; more than five SHAP features; 12 forecast points over more than 100 history points |

The smoke test adds the layer the unit tests cannot reach: the processes that actually run. It is described in the next section.

## 7.4 Functional testing

`scripts/smoke_test.py` runs 35 checks against the live servers. The same 16 API checks are run twice, once directly against Flask on port 8409 and once through the Vite proxy on port 5409 (the path the browser uses), and three more checks confirm that the single-page application is served at /, /app and the deep link /app/forecast.

**Table 32: Smoke-test checks (run against both ports)**

| Group | Checks | What is verified |
|---|---|---|
| Service health | 1 | /health reports a loaded model |
| Read-only JSON routes | 5 | /api/options lists 16 education levels and numeric limits; /api/dashboard has ROC-AUC and five top SHAP drivers; /api/explain; /api/performance has four models; /api/forecast has 12 points |
| Artifact images | 4 | confusion_matrix.png, roc_curve.png, shap_bar.png and shap_beeswarm.png are served as image/png |

| Group | Checks | What is verified |
|---|---|---|
| Prediction | 3 | A valid profile returns a probability and live SHAP; a very different profile gives a different probability; age -5 returns 400 with messages |
| What-if | 3 | Gains 80,000 and hours 50 produce a comparison with two changed fields; a request with no change returns 400; hours 500 returns 400 |
| Single-page app (port 5409 only) | 3 | /, /app and /app/forecast each return the application shell |

All 35 of 35 checks passed. Functional behaviour of the interface was additionally exercised by the browser walk-through in `scripts/capture_screens.py`, which logs in with the sample account, fills the assessment form, analyses a profile, runs a what-if comparison and visits every screen, producing the screenshots of Chapter 8; a further script, `scripts/capture_extra_screens.py`, produced the validation-error, Established-profile and Developing-profile captures. These walks confirm that the screens render with real model output but they are not assertion-based tests.

## 7.5 Security testing

FinSight AI stores no personal data and has no real authentication, so its security surface is small: a JSON API that accepts user input, a route that serves four image files, and, on the original Flask pages only, a session cookie that carries the what-if baseline. This section records what was actually tested.

**Input handling.** The validation module checks every field against an allow-list or a numeric range before any model code runs. 29 requests were sent to the live API; 22 were refused with HTTP 400 and 7 were accepted, each accepted case being a valid or boundary input or an ignored extra field. After the whole probe the service still answered /health with status 'ok' and the model loaded, so none of the inputs crashed or degraded it.

**Table 33: Hostile and malformed input (POST /api/predict)**

| ID | Case | HTTP | Response |
|---|---|---|---|
| E19 | Script text in a field (occupation) | 400 | Occupation has an invalid value. |
| E20 | SQL-style text in a field (sex) | 400 | Sex has an invalid value. |
| E21 | Extra unknown field added (is_admin) | 200 | p 0.0109, Emerging / High |
| E22 | Very long string (10,000 characters) in race | 400 | Race has an invalid value. |
| E17 | Unknown workclass ('Hacker') | 400 | Employment type has an invalid value. |
| E18 | Unknown country ('Atlantis') | 400 | Country has an invalid value. |

Script text and SQL-style text in a categorical field are rejected because they are not in the allowed set, so they are never echoed into a page or passed to any query (there is no database to query). An unknown extra field (`is_admin`) is silently ignored: the validator copies only the twelve known keys, so the request cannot smuggle extra attributes into the model frame. A 10,000-character string is rejected by the same allow-list check.

**Table 34: File-access probes (GET)**

| ID | Case | HTTP | Content type |
|---|---|---|---|
| G01 | Allowed artifact image | 200 | image/png |
| G02 | Artifact not on the allow-list (metrics.json) | 404 | text/html |
| G03 | Model file requested through the artifact route | 404 | text/html |
| G04 | Encoded path traversal to models/best_model.pkl | 404 | text/html |
| G05 | Direct path to the model file | 404 | text/html |
| G06 | Environment file | 404 | text/html |
| G07 | Unknown page | 404 | text/html |

The artifact route serves only the four image names on its allow-list. Requests for a JSON artifact, the model pickle (directly and with an encoded `..` traversal) and `.env` all return 404; only the allowed image returns 200.

**What this does not establish.** This is a black-box probe of one input route and one file route, not a penetration test. Authentication was not tested because the sample login is a browser-only demo gate (`demo@finsight.ai` / `finsight123`, stored in client code) and gives no protection to the API, which is open on the local machine. The Flask secret key defaults to a development value when `FINSIGHT_SECRET_KEY` is not set, so session cookies are not trustworthy in a shared deployment. Cross-site scripting through the React interface was not tested in a browser; the interface renders only server-validated values and React escapes text by default, but that is a property of the framework and was not verified here. These are listed again as limitations in Chapter 8.

## 7.6 Validation and edge cases

The edge-case script `scripts/edge_cases.py` sends each case below to the running API and records the real status and message; the tables are generated from `artifacts/edge_cases.json`, so they cannot drift from what the server did. The first table covers missing and malformed requests.

**Table 35: Missing and malformed requests (POST /api/predict)**

| ID | Case | HTTP | Response |
|---|---|---|---|
| E01 | Missing field (workclass omitted) | 400 | Employment type is required. |
| E02 | Empty JSON object {} | 400 | Age is required.; Employment type is required.; Education is required.; Marital status is required.; Occupation is re... |
| E03 | Empty request body (no content) | 400 | Age is required.; Employment type is required.; Education is required.; Marital status is required.; Occupation is re... |
| E04 | Body is not JSON (plain text) | 400 | Age is required.; Employment type is required.; Education is required.; Marital status is required.; Occupation is re... |
| E05 | Malformed JSON (truncated) | 400 | Age is required.; Employment type is required.; Education is required.; Marital status is required.; Occupation is re... |

An empty body, a plain-text body and truncated JSON all return the same list of 'is required' messages. This is safe but slightly misleading: the route reads the body with `get_json(silent=True)` and treats anything that is not valid JSON as an empty object, so the caller is told every field is missing instead of being told the body was not JSON. It is recorded as a minor usability finding, not fixed, because it affects no valid client.

**Table 36: Numeric boundaries and ranges (POST /api/predict)**

| ID | Case | HTTP | Response |
|---|---|---|---|
| E06 | Age 15 (below minimum 16) | 400 | Age must be between 16 and 100. |
| E07 | Age 101 (above maximum 100) | 400 | Age must be between 16 and 100. |
| E08 | Age 16 (lower boundary) | 200 | p 0.0004, Emerging / High |
| E09 | Age 100 (upper boundary) | 200 | p 0.0088, Emerging / High |
| E10 | Hours 0 (below minimum 1) | 400 | Hours worked per week must be between 1 and 99. |
| E11 | Hours 100 (above maximum 99) | 400 | Hours worked per week must be between 1 and 99. |
| E12 | Negative investment gains (-5) | 400 | Investment gains must be between 0 and 1000000. |
| E13 | Negative investment losses (-1) | 400 | Investment losses must be between 0 and 1000000. |
| E14 | Investment gains 1,000,000 (upper boundary) | 200 | p 0.9935, Established / Low |
| E15 | Investment gains 1,000,001 (above maximum) | 400 | Investment gains must be between 0 and 1000000. |
| E16 | Age given as text ('abc') | 400 | Age must be a number. |

Every value one step outside its range is refused with a message naming the field and the permitted range, and every value on the boundary is accepted: ages 16 and 100, and investment gains of exactly 1,000,000. The boundary results are worth reading as predictions: a 16-year-old and a 100-year-old with the same other attributes score as Emerging with a very low strength score, while gains of 1,000,000 push the score to the Established band. Because the training data contain very few such rows, these boundary predictions are extrapolations; they show that the system does not crash at the limits, not that its scores there are reliable.

**Table 37: What-if scenarios (POST /api/whatif)**

| ID | Case | HTTP | Response |
|---|---|---|---|
| E23 | What-if with no scenario field changed | 400 | Change at least one scenario field to compare. |
| E24 | What-if with gains 1,000,001 (above maximum) | 400 | Scenario investment gains out of range. |
| E25 | What-if with hours 500 | 400 | Scenario hours out of range. |
| E26 | What-if with unknown education | 400 | Scenario education is invalid. |
| E27 | What-if with valid change (hours 40 to 50) | 200 | delta +1.68 pp, Emerging -> Emerging |
| E28 | What-if with same value as baseline (hours 40) | 200 | delta +0.00 pp, Emerging -> Emerging |
| E29 | What-if with extreme gains 1,000,000 | 200 | delta +98.26 pp, Emerging -> Established |

A what-if request with no scenario field returns 400 and a clear message, and every out-of-range scenario value is refused. Two outcomes deserve comment. A scenario that sets hours to the baseline value (40) is accepted and returns a change of 0.00 percentage points: the rule is 'at least one scenario field must be filled in', not 'at least one must differ'. And the extreme scenario of gains 1,000,000 changes the score by close to 100 points, the same extrapolation effect discussed above.

## 7.7 Performance testing

Response times were measured on the development machine against 127.0.0.1 (using the loopback address avoids a two-second IPv6 fallback delay seen on this machine with the name 'localhost'). Each route was called 40 times in sequence by one client, so these are single-user latencies and not capacity figures.

**Table 38: API response time (milliseconds, 40 calls each)**

| Route | Median | 95th percentile |
|---|---|---|
| POST /api/predict | 68.9 | 116.9 |
| POST /api/whatif | 136.1 | 193.7 |
| GET /api/forecast | 7.3 | 27.1 |
| GET /api/performance | 9.1 | 26.1 |
| GET /health | 6.7 | 19.5 |
| POST /api/predict via port 5409 proxy | 71.8 | 165.8 |

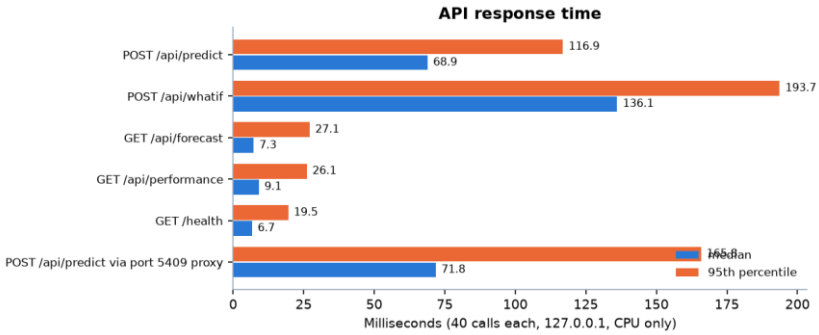



**Figure 19: Median and 95th-percentile response time of the API routes**

A prediction, which includes feature engineering, XGBoost inference and a live SHAP explanation, takes a median of 68.9 ms and 95 percent of calls finish within 116.9 ms. A what-if request runs the prediction twice and takes 136.1 ms at the median, about double, as expected. Read-only routes that return a stored JSON artifact take under 10 ms at the median. Going through the Vite proxy adds little (71.8 ms median for a prediction). In-process, `predict_profile` alone has a median of 52.4 ms and a 95th percentile of 86.7 ms (30 calls), so most of the HTTP time is the model and SHAP, not the web layer.

All of these are comfortably interactive for a single user. They say nothing about behaviour under concurrent load, which was not measured.

## 7.8 User acceptance

No formal user-acceptance test with outside participants was carried out. What exists is the project team's own acceptance walk-through: the team logged in with the sample account, entered several profiles of different kinds (Section 8.4), compared a baseline and a scenario, read the explanations, and checked the forecast and performance screens against the files in `artifacts/`. The criteria applied were those of the objectives in Chapter 1: the screen shows a profile, a risk band, the reasons, a forecast and the model's measured performance, in plain language, with a disclaimer on every analysis. All of these were satisfied, but a walk-through by the people who built the system is evidence that the features work, not evidence that outsiders find them clear.

Two usability observations came out of the walk-through and are reported as they are. The validation banner for a rejected assessment says 'Please fix the highlighted fields and try again' and marks the field in red (see the validation-error screen in Chapter 8), which is adequate for a single bad field. And the extreme what-if scores in the 90s show how easily a non-expert could over-read a scenario; the interface labels every such result as a model simulation, not a causal prediction, and that label is the only protection.

## 7.9 Limits of the testing done

The testing above supports the claim that the system works as designed on the machine it was built on. It does not support stronger claims, and the following were not done:

- **No load or concurrency test.** All timings are single-user. Flask is run with its development server, which is not designed for concurrent production traffic.
- **No penetration test or dependency audit.** The probe in Section 7.5 is a black-box check of two routes. Authentication, session handling under attack and cross-site scripting were not tested.

- **No formal user-acceptance study.** There were no outside participants and no questionnaire.
- **No browser or device matrix.** The only browser captures are the 1440 x 900 screenshots taken with Chrome through Playwright. Behaviour at phone or tablet widths, in other browsers and with assistive technology was not tested.
- **No front-end unit tests.** The React code has no component tests; it is checked only through the browser walk-throughs and the smoke test of what the server returns.
- **No model-regression test against new data.** The tests check that the model returns plausible, input-dependent output and that the stored artifacts are internally consistent; they do not check predictive accuracy, which is evaluated separately in Chapter 8 on a held-out split.
- **No test of the training script's failure paths** (for example a failed download, which falls back to synthetic data), and no test run on any other operating system.

These gaps are acceptable for a final-year prototype whose purpose is to demonstrate an explainable pipeline, and they are the first items to address before any wider use.

# CHAPTER 8: RESULTS AND OUTPUT SCREENS

## 8.1 Output screens

The screens below are captures of the running application at 1440 x 900 pixels, taken in Chrome by `scripts/capture_screens.py` and `scripts/capture_extra_screens.py` against the live servers (API on port 8409, React front end on port 5409). Every screen shows real model output computed from the profile typed into the form; none is a mock-up. Some captures include the green 'Analysis complete' notification, which is part of the interface.

The dashboard is the first screen after sign-in. Its four tiles are filled from `/api/dashboard`: the serving model (XGBoost), accuracy 87.8% on the held-out test set, ROC-AUC 0.932 (cross-validated 0.929 +/- 0.001) and F1 0.724 with precision 79% and recall 67%. Beneath them are a bar chart of the five largest mean absolute SHAP values and a short statement of the economic outlook, with buttons to start an assessment or open the forecast.

The form collects the twelve inputs, each with its allowed range or option list shown under the field. The values in the capture are the first record of the Adult data set. 'Use sample profile' fills them in one click and 'Clear' empties them. Validation runs in the browser and again on the server; the next figure shows a rejected entry.

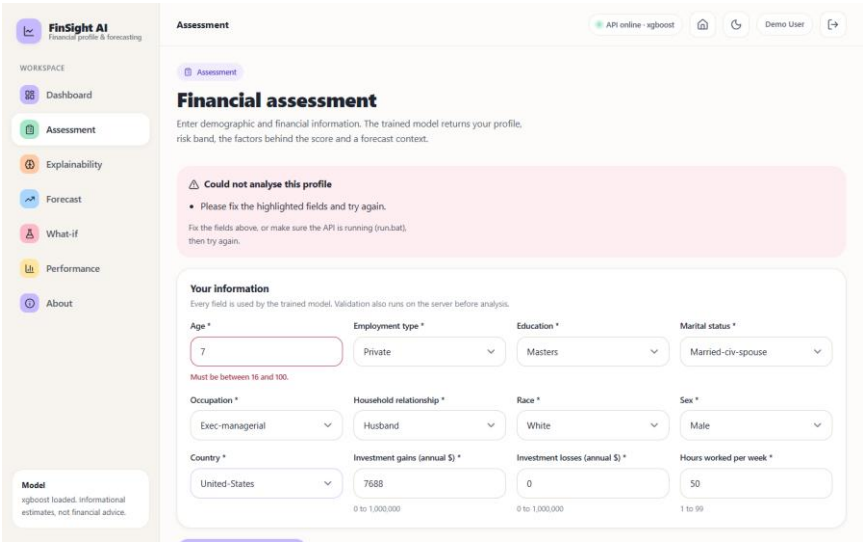



**Figure 20: Validation error: age 7 is outside the permitted range of 16 to 100, so the field is outlined and the analysis is refused**

When the server returns HTTP 400 the page shows a pink banner and marks the offending field with its allowed range ('Must be between 16 and 100.'). No model is run and no result is shown. This is the behaviour the edge-case tests of Chapter 7 checked at the API level.

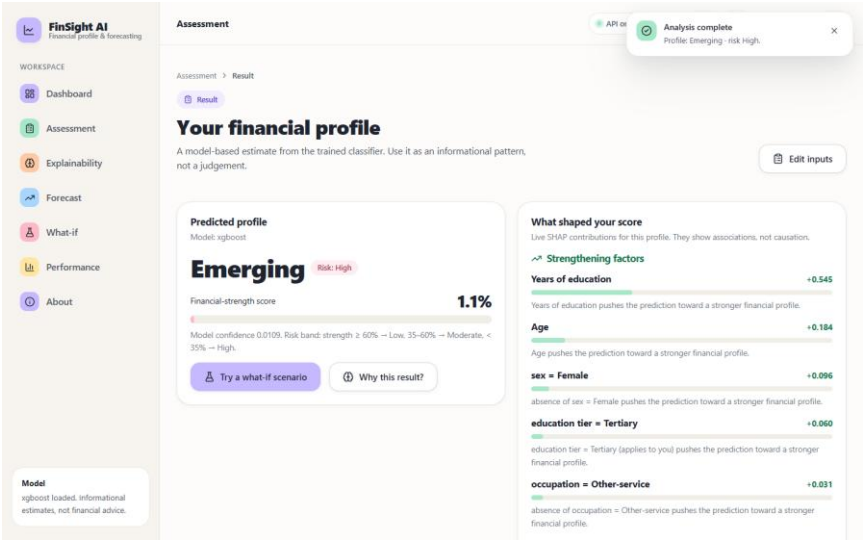



**Figure 21: Result for the sample profile: Emerging profile, High risk band, financial-strength score 1.1%, with the SHAP factors that shaped it**

The result screen shows the predicted profile, the risk band, the financial-strength score (the model's probability of income above 50K, here 0.0109) and the live SHAP contributions for this one person, split into strengthening and weakening factors with a plain-language sentence for each. For this record years of education (+0.545) and age (+0.184) strengthen the prediction, while its investment-gains value of 2,174 is the largest weakening factor (-1.626) presumably because larger gains accompany high incomes in the training data. The text below the score states the risk-band rule: strength of at least 60% is Low risk, 35% to 60% Moderate and below 35% High.

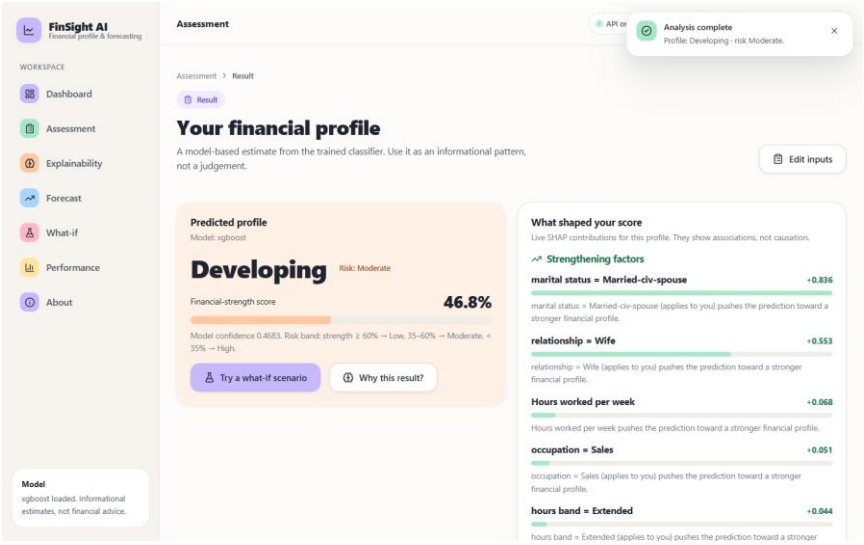



**Figure 22: Result for a dual-income profile (age 34, Local-gov, Some-college, married, Sales, 42 hours): Developing profile, Moderate risk, score 46.8%**

A profile between the two thresholds. Being married to a civilian spouse (+0.836) and the household role 'Wife' (+0.553) are the strongest strengthening factors; the screen makes clear that the score of 46.8% sits inside the Developing band, which in the test split corresponds to a population in which 44.0% actually earn more than 50K (Section 8.2).

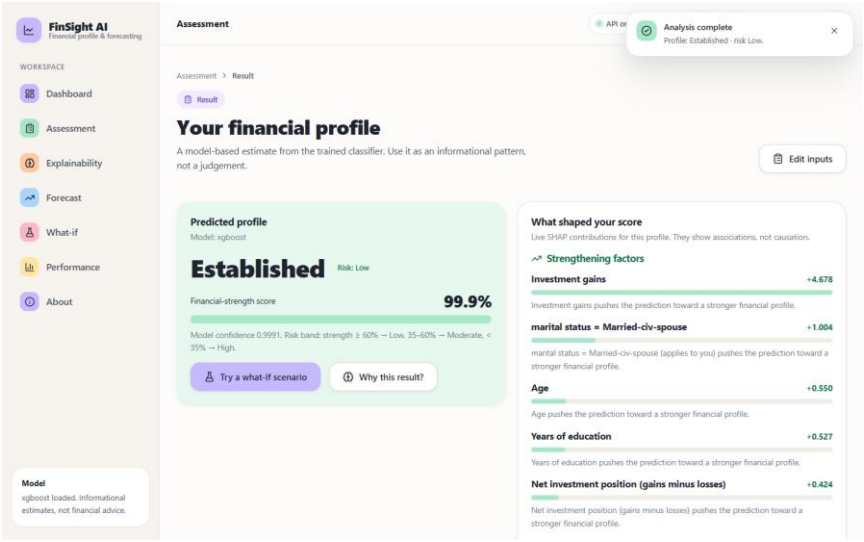



**Figure 23: Result for a mid-career manager (age 52, Private, Masters, married, Exec-managerial, gains 7,688, 50 hours): Established profile, Low risk, score 99.9%**

The third band. Investment gains (+4.678) dominate, followed by marital status (+1.004), age (+0.550) and years of education (+0.527). The full page continues with the actionable insights, an echo of the inputs and a miniature of the forecast; the full-page capture is stored as `docs/screenshots/15_result-established-full.png`.

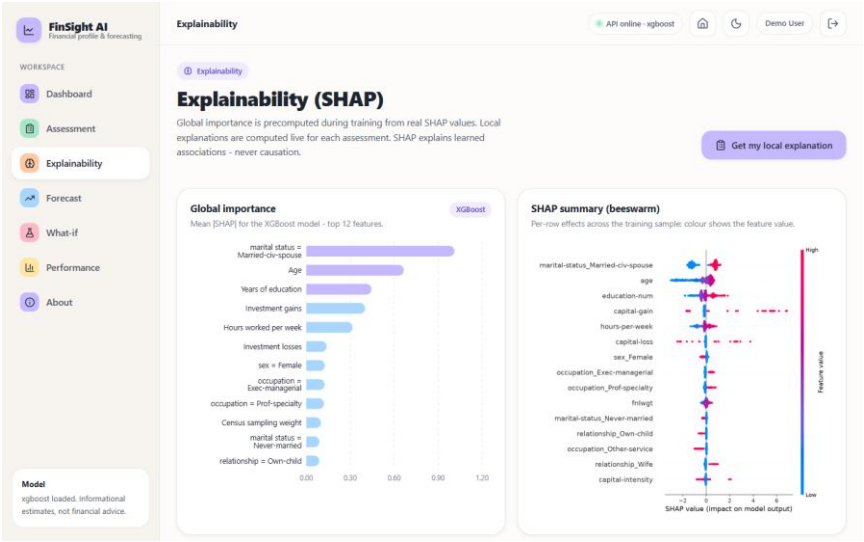



**Figure 24: Explainability screen: global SHAP importance (left) and the SHAP summary of per-row effects (right)**

The explainability screen shows the global picture computed at training time. 'marital status = Married-civ-spouse' has the largest mean absolute SHAP value (1.012), followed by age (0.665), years of education (0.444), investment gains (0.401) and hours per week (0.315). In the beeswarm on the right each dot is one row; the colour is the feature value, so red capital-gain dots far to the right show that large gains push the prediction up. A banner states that SHAP explains learned associations, never causation.

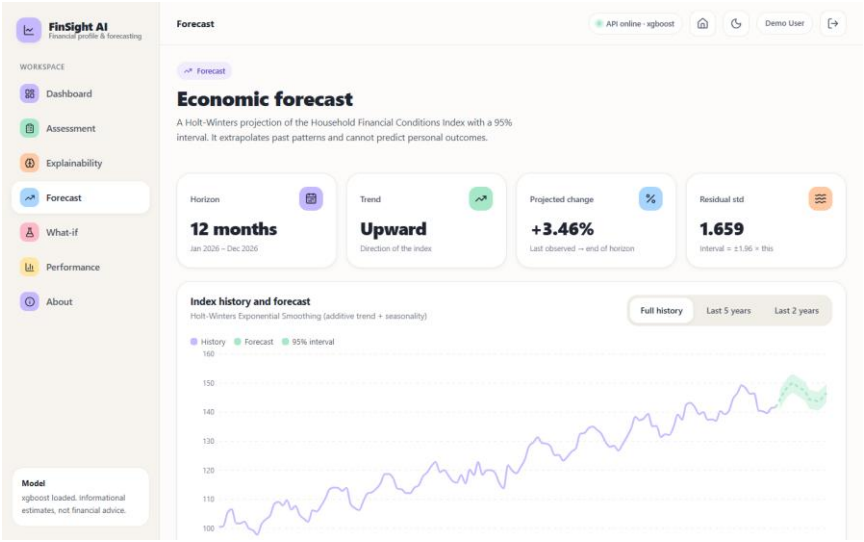



**Figure 25: Forecast screen: 12-month Holt-Winters projection of the Household Financial Conditions Index with its 95% interval**

Four tiles give the horizon (12 months, January to December 2026), the trend direction (upward), the projected change (+3.46%, from 141.7 to 146.6) and the residual standard deviation (1.659), from which the interval of plus or minus 1.96 standard deviations is drawn. The chart can be switched between the full history, the last five years and the last two years. The subtitle says the projection 'extrapolates past patterns and cannot predict personal outcomes'; the series it extrapolates is synthetic, as Section 8.2 explains.

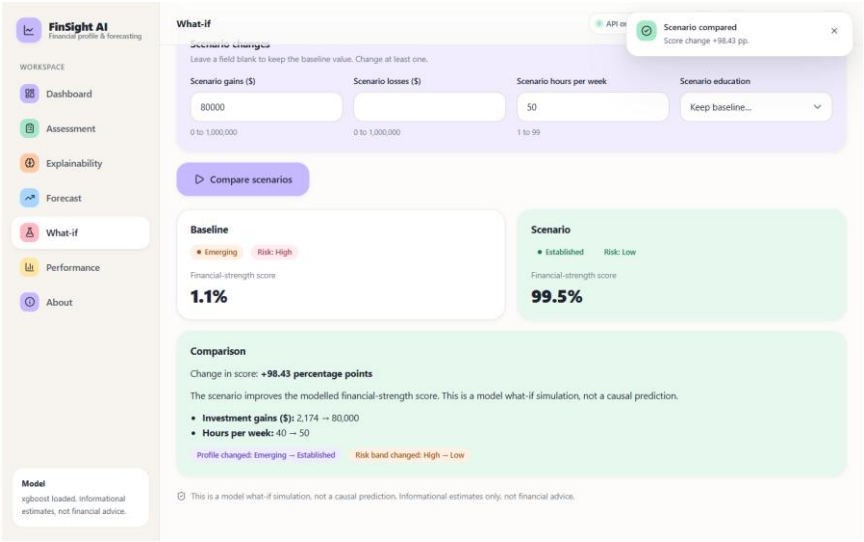



**Figure 26: What-if screen: scenario gains of 80,000 and 50 weekly hours compared with the baseline profile**

The what-if screen keeps the baseline from the last assessment and lets the user change gains, losses, hours or education. In the capture, raising gains to 80,000 and hours to 50 moves the same person from Emerging (1.1%) to Established (99.5%), a change of +98.43 percentage points, and the comparison lists the two changed fields. Every result is captioned as a model simulation, not a causal prediction, because a jump of this size is the model extrapolating into a region where the training data are sparse (Section 8.5).

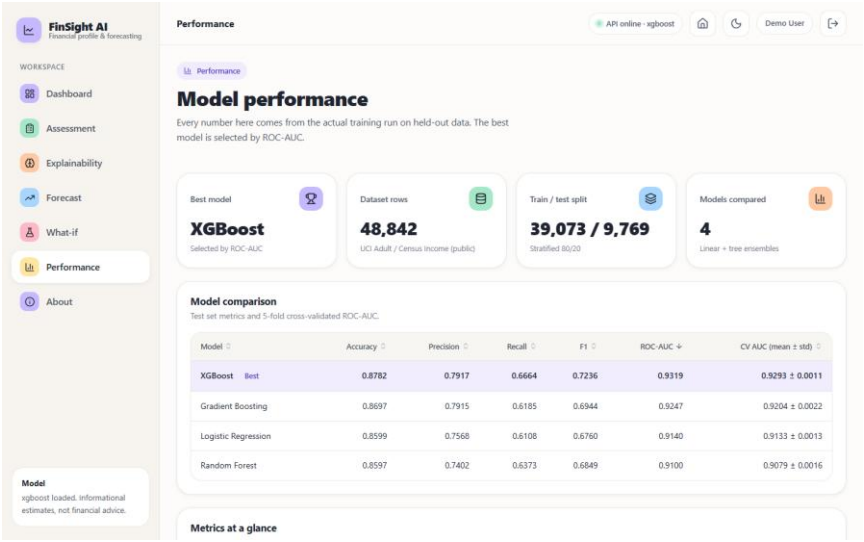



**Figure 27: Performance screen: model comparison table with the selected model highlighted, followed by the metrics, ROC and confusion-matrix views**

The performance screen lists the four models with accuracy, precision, recall, F1, ROC-AUC and the mean and standard deviation of cross-validated AUC, with the best model marked. Below the fold it shows the ROC curves, the confusion matrix, the validation protocol with the per-fold AUCs, and the correlation and logistic-regression statistics. The numbers on this screen are the numbers in Section 8.2.

## 8.2 Model performance

All numbers in this section are measured on the 9,769 records of the stratified test split (2,338 with income above 50K and 7,431 at or below), produced by training on the other 39,073 records. Classification uses the default threshold of 0.5 unless stated. A classifier that always answered 'at most 50K' would score 76.1% accuracy on this split, which is the bar the accuracy figures should be read against; ROC-AUC, which does not depend on the class ratio, is the primary metric.

**Table 39: Test-set metrics of the four classifiers**

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC | 5-fold CV AUC |
|---|---|---|---|---|---|---|
| Logistic Regression | 0.8599 | 0.7568 | 0.6108 | 0.6760 | 0.9140 | 0.9133 +/- 0.0013 |
| Random Forest | 0.8597 | 0.7402 | 0.6373 | 0.6849 | 0.9100 | 0.9079 +/- 0.0016 |
| Gradient Boosting | 0.8697 | 0.7915 | 0.6185 | 0.6944 | 0.9247 | 0.9204 +/- 0.0022 |
| XGBoost (selected) | 0.8782 | 0.7917 | 0.6664 | 0.7236 | 0.9319 | 0.9293 +/- 0.0011 |

XGBoost is best on every metric, with Gradient Boosting essentially tied on precision (0.791 against 0.792). Its ROC-AUC of 0.9319 is 0.0179 above Logistic Regression and 0.0072 above Gradient Boosting. The size of the gain over the linear baseline is modest: Logistic Regression already reaches 0.914, and it is slightly ahead of Random Forest (0.910) on AUC. With 48,842 rows and mostly categorical inputs, much of the signal is linear in the engineered features, which is why interpretable regression remains a strong comparator.

The ranking is stable. The cross-validated AUC, computed on the training split only, gives the same order (XGBoost 0.9293, Gradient Boosting 0.9204, Logistic Regression 0.9133, Random Forest 0.9079), the fold-to-fold standard deviation is at most 0.0022, and XGBoost beats Gradient Boosting in every one of the five folds. No formal significance test (such as the

DeLong test used by the base paper) was run, and the model was chosen using the test split, so the headline figure is slightly optimistic; the cross-validated figure is the cleaner estimate.

**Table 40: Confusion-matrix counts, specificity, PR-AUC and Brier score on the test split**

| Model | TN | FP | FN | TP | Specificity | PR-AUC | Brier |
|---|---|---|---|---|---|---|---|
| Logistic Regression | 6,972 | 459 | 910 | 1,428 | 0.9382 | 0.7867 | 0.0976 |
| Random Forest | 6,908 | 523 | 848 | 1,490 | 0.9296 | 0.7955 | 0.0981 |
| Gradient Boosting | 7,050 | 381 | 892 | 1,446 | 0.9487 | 0.8228 | 0.0903 |
| XGBoost | 7,021 | 410 | 780 | 1,558 | 0.9448 | 0.8381 | 0.0857 |

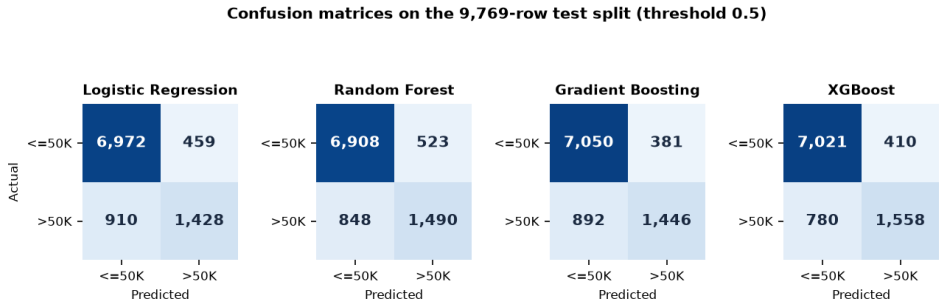



**Figure 28: Confusion matrices of the four models on the test split (rows: actual, columns: predicted)**

For the selected model, 7,021 of the 7,431 people at or below 50K are classified correctly (specificity 94.5%) and 1,558 of the 2,338 above 50K are found (recall 66.6%). The 1,190 errors split into 410 false positives and 780 false negatives: the model errs more often by missing a high-income person than by promoting a low-income one. It also has the best Brier score (0.0857) and PR-AUC (0.8381), so its probabilities as well as its labels are better than the alternatives'.

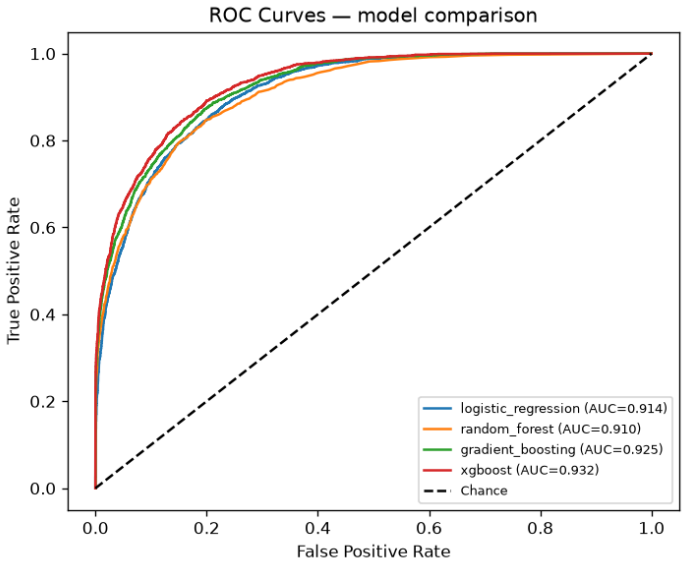



**Figure 29: ROC curves of the four models on the test split (generated by the training script)**

The ROC curves of the four models lie close together and well above the diagonal; the ordering by area matches the table. A threshold of 0.5 is a convention, not an optimum for this application, so the threshold analysis below is more informative than any single operating point.

**Table 41: Threshold analysis of the selected model (test split)**

| Threshold | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| 0.20 | 0.8212 | 0.5825 | 0.8922 | 0.7048 |
| 0.30 | 0.8529 | 0.6531 | 0.8221 | 0.7279 |
| 0.35 | 0.8637 | 0.6901 | 0.7810 | 0.7327 |
| 0.40 | 0.8698 | 0.7219 | 0.7417 | 0.7316 |
| 0.50 | 0.8782 | 0.7917 | 0.6664 | 0.7236 |
| 0.60 | 0.8764 | 0.8523 | 0.5851 | 0.6939 |
| 0.70 | 0.8650 | 0.8996 | 0.4906 | 0.6349 |
| 0.80 | 0.8483 | 0.9573 | 0.3832 | 0.5473 |

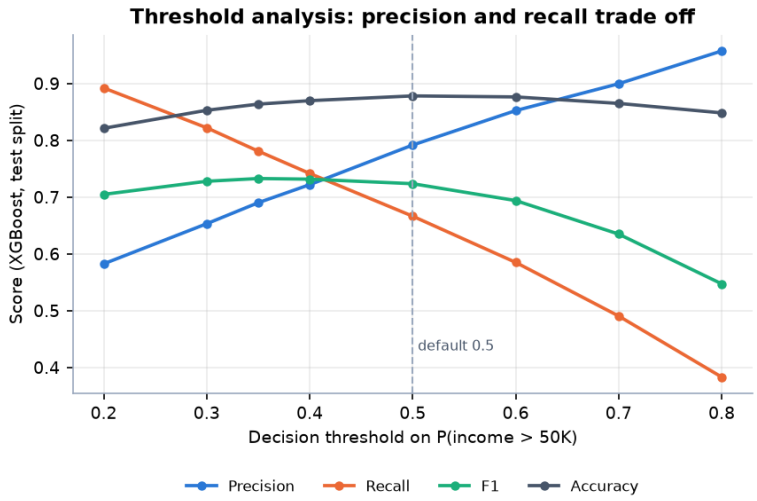



**Figure 30: Precision, recall, F1 and accuracy of the selected model as the decision threshold moves**

Lowering the threshold from 0.5 to 0.3 raises recall from 66.6% to 82.2% while precision falls from 79.2% to 65.3%. F1 peaks at a threshold of 0.35 (0.7327), slightly above its value at 0.5 (0.7236), and accuracy is highest near 0.5 (0.8782). At 0.8 precision reaches 95.7% but only 38.3% of high-income people are found. An institution that wants to avoid missing strong profiles would choose a lower threshold; one that wants few false alarms, a higher one. The application does not use a single threshold for its output: it uses the probability itself, mapped to three bands, as described next.

**Table 42: Calibration of the selected model: predicted probability against observed share above 50K (test split)**

| Probability bin | Rows | Mean predicted | Observed share |
|---|---|---|---|
| 0.0-0.1 | 5,336 | 0.0212 | 0.0221 |
| 0.1-0.2 | 852 | 0.1462 | 0.1573 |
| 0.2-0.3 | 638 | 0.2469 | 0.2571 |
| 0.3-0.4 | 541 | 0.3474 | 0.3475 |
| 0.4-0.5 | 434 | 0.4470 | 0.4055 |
| 0.5-0.6 | 363 | 0.5493 | 0.5234 |
| 0.6-0.7 | 330 | 0.6475 | 0.6697 |
| 0.7-0.8 | 339 | 0.7472 | 0.7404 |
| 0.8-0.9 | 238 | 0.8475 | 0.8697 |
| 0.9-1.0 | 698 | 0.9843 | 0.9871 |

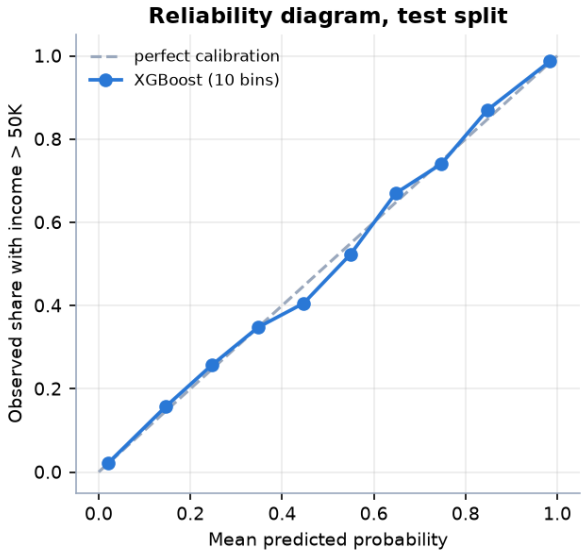



**Figure 31: Reliability diagram of the selected model on the test split**

The model is well calibrated: in every bin the mean predicted probability is close to the observed share, with the largest gap in the 0.4 to 0.5 bin (0.447 predicted, 0.406 observed). This matters for the application because the financial-strength score shown to the user is this probability; a score of 47% really does correspond to roughly four to five in ten comparable people earning above 50K in this data. Calibration was measured but not corrected; the raw XGBoost probabilities were used.

**Table 43: Profile bands on the test split**

| Profile band | Rule on probability p | Test rows | Actually earn above 50K |
|---|---|---|---|
| Emerging (High risk) | p < 0.35 | 7,123 | 7.2% |
| Developing (Moderate risk) | 0.35 <= p < 0.60 | 1,041 | 44.0% |
| Established (Low risk) | p >= 0.60 | 1,605 | 85.2% |

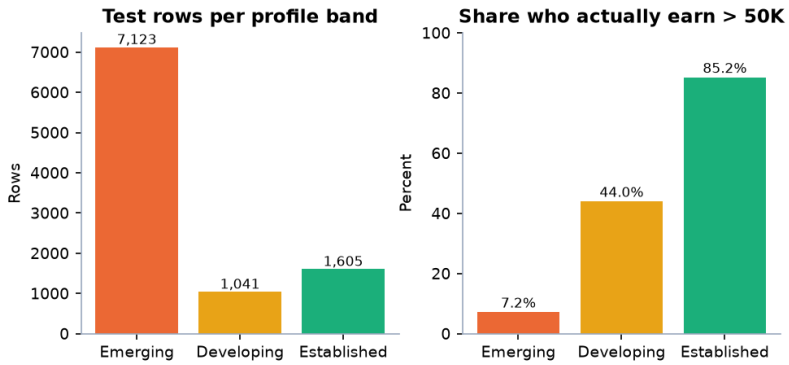



**Figure 32: Number of test rows per profile band and the share of each band that actually earns above 50K**

The bands separate the population usefully: only 7.2% of the 7,123 rows in the Emerging band actually earn above 50K, against 44% in Developing and 85.2% in Established. The band thresholds (0.35 and 0.60) are fixed in the application code (`profile_from_proba`). The labels are descriptive names for ranges of a probability; they are not credit grades.

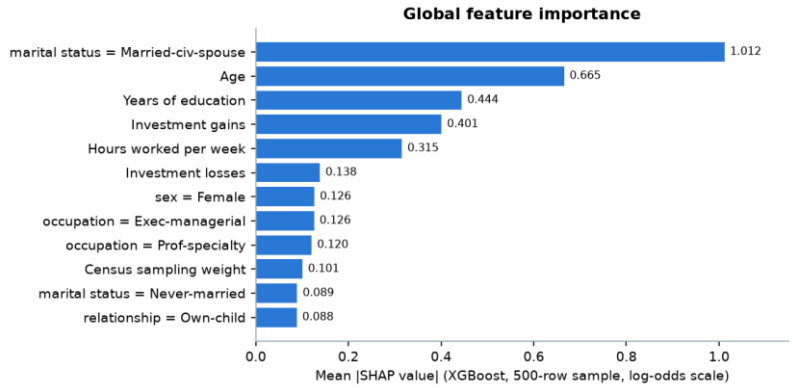



**Figure 33: Twelve largest global SHAP importances of the selected model**

**Table 44: Top ten global SHAP importances (XGBoost, 500-row sample, log-odds scale)**

| Rank | Feature | Mean \|SHAP\| |
|---|---|---|
| 1 | marital status = Married-civ-spouse | 1.0116 |
| 2 | Age | 0.6653 |
| 3 | Years of education | 0.4445 |
| 4 | Investment gains | 0.4007 |
| 5 | Hours worked per week | 0.3153 |
| 6 | Investment losses | 0.1385 |
| 7 | sex = Female | 0.1263 |
| 8 | occupation = Exec-managerial | 0.1256 |
| 9 | occupation = Prof-specialty | 0.1204 |
| 10 | Census sampling weight | 0.1007 |

Marital status (being a civilian-spouse household), age, education, investment gains and weekly hours account for most of the model's behaviour. Sex appears in the top ten (rank 7). The logistic-regression statistics computed on the training split point the same way: correlations with income above 50K are 0.335 for years of education, 0.231 for weekly hours, 0.229 for age and 0.212 for net capital, and the four logit coefficients are all positive with p-values reported as 0.0. As the base paper also stresses [1], these are associations learned from historical data, not causal effects.

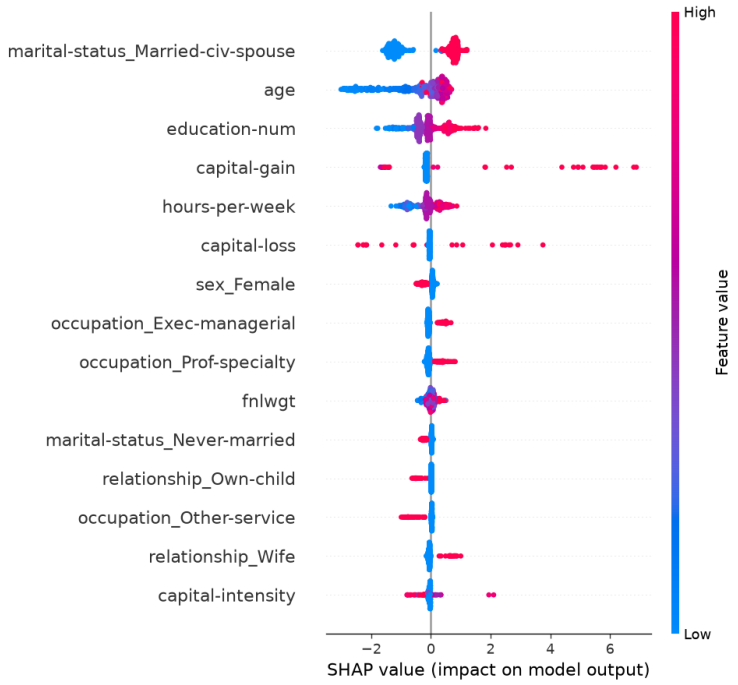



**Figure 34: SHAP beeswarm plot: effect of each feature value on the prediction (generated by the training script)**

### *Forecast*

**The forecast series is synthetic.** The Adult data have no time dimension, so the 12-month forecast uses a Household Financial Conditions Index generated by `scripts/forecast.py` (132 monthly values from January 2015 to December 2025: a linear trend, a yearly sinusoid, Gaussian noise and a mild downturn in March to August 2020). It is documented as illustrative in `data/README.md`. The numbers below therefore measure how well the Holt-Winters method recovers its own generating pattern; they are not evidence of accuracy on real economic data.

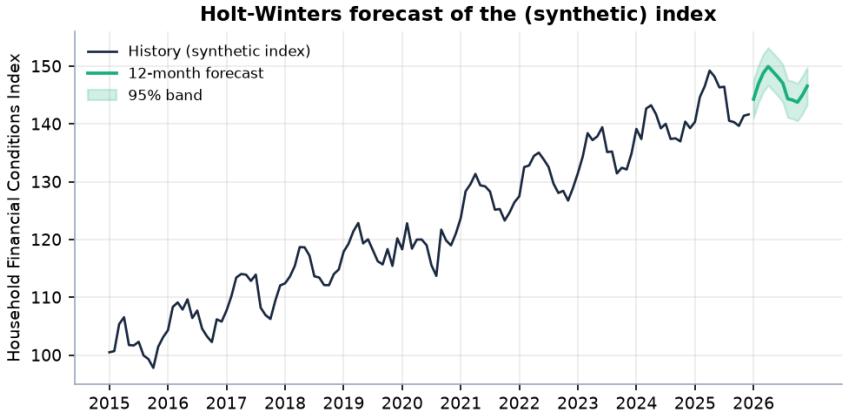



**Figure 35: Holt-Winters forecast of the synthetic index with its 95% band**

**Table 45: Back-test of the forecasting method on the synthetic series**

| Quantity | Value |
|---|---|
| Fitting window | 120 months (2015-01 to 2024-12) |
| Held-out window | 12 months (2025) |
| Mean absolute error | 1.429 index points |
| Root mean squared error | 1.600 |
| Mean absolute percentage error | 0.997% |

| Quantity | Value |
|---|---|
| Naive last-value forecast, MAE | 4.504 |
| Share of held-out months inside the 95% band | 100% (12 of 12) |

Fitted to 2015-2024, the model predicts the 12 months of 2025 with a mean absolute error of 1.43 index points (1.00%), about a third of the 4.50 of a naive forecast that repeats the last value, and all 12 held-out values fall inside the 95% band. Because the series is generated from a trend plus seasonality plus noise, a trend-and-seasonality smoother is almost guaranteed to do well; the result confirms that the code is correct, nothing more. The live forecast (fitted to all 132 months) moves from 141.7 in December 2025 to 146.6 in December 2026 (+3.46%), with a final-month band of 143.3 to 149.8.

## 8.3 Comparison with the base paper

The base paper [1] is the only quantitative reference for this project, and the comparison has to be read with care because almost everything differs. The table below sets the numbers side by side and the figure that follows plots them. **This is context, not a ranking.**

**Table 46: Base paper and this project compared (ROC-AUC)**

| Model | Base paper (hospital-level Group K-Fold) | This project, test split | This project, 5-fold CV |
|---|---|---|---|
| Logistic Regression | 0.776 | 0.914 | 0.913 |
| Random Forest | 0.831 | 0.910 | 0.908 |
| Gradient Boosting | 0.840 | 0.925 | 0.920 |
| XGBoost | 0.842 | 0.932 | 0.929 |

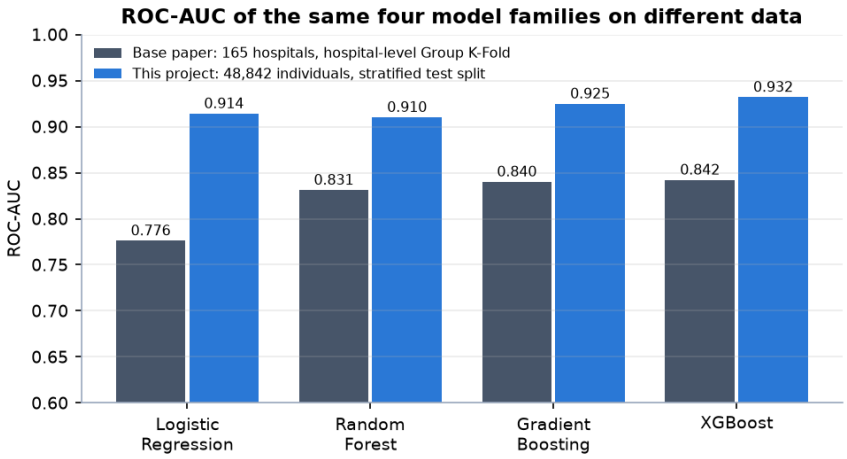



**Figure 36: ROC-AUC of the same four model families in the base paper and in this project (different data and protocols)**

**Table 47: What differs between the two studies**

| Aspect | Base paper | This project |
|---|---|---|
| Unit of analysis | Hospitals (organisations) | Individual people |
| Data | 165 Korean hospitals, 825 hospital-year rows, 2019-2023, 236 line items reduced to 11 features | UCI Adult / Census Income, 48,842 rows from 1994, 14 raw fields expanded to 26 columns |
| Target | Annual deficit (net income below zero), 41.2% positive | Income above 50K, 23.9% positive; mapped to a profile band, not a distress label |
| Validation | Hospital-level Group K-Fold (primary), temporal hold-out 2019-22 to 2023, stratified K-Fold for reference | Stratified 80/20 split and stratified 5-fold CV; one row per person, so no entity grouping and no temporal split |

| Aspect | Base paper | This project |
|---|---|---|
| Further analyses | Counterfactual subsidy analysis, panel fixed-effects regression, DeLong tests, Altman Z" baseline | Not reproduced; SHAP, a logistic-regression table, a what-if screen, calibration and threshold analysis instead |
| Deliverable | A framework and findings | A working web application around the same framework |

Three observations can be drawn. First, the ordering of the families is similar: gradient-boosted trees lead and the linear model trails, although in this project Random Forest falls below Logistic Regression while in the base paper it sits between boosting and the linear baseline. Second, the gap between boosting and the linear baseline is much smaller here (0.018 AUC) than in the base paper (0.066), consistent with a large, mostly categorical person-level data set in which main effects explain most of the signal. Third, the absolute AUCs (0.932 against 0.842) must not be compared as a measure of skill: they are computed on different populations and different targets, with a much larger sample and a less demanding protocol here. The base paper's own contrast between random and entity-grouped validation (0.860 to 0.831 for Random Forest) shows how much the protocol alone can move AUC. What this project adds to the base paper is not a better number but the extension of the approach to individuals and its delivery as an explainable application.

## 8.4 Sample inputs and outputs

Five profiles were sent to the running API (`POST /api/predict`) by `scripts/error_analysis.py`; the table lists the inputs and the output of the system, copied from `artifacts/sample_predictions.json`. Profile A is the first record of the Adult data set; the others were written to span the range of outputs.

**Table 48: Sample profiles and the system's output**

| ID | Inputs | Strength | Profile / risk |
|---|---|---|---|
| A | 39, State-gov, Bachelors, Never-married, Adm-clerical, Male, gains 2,174, losses 0, 40 h | 1.1% | Emerging / High |
| B | 52, Private, Masters, Married-civ-spouse, Exec-managerial, Male, gains 7,688, losses 0, 50 h | 99.9% | Established / Low |
| C | 34, Local-gov, Some-college, Married-civ-spouse, Sales, Female, gains 0, losses 0, 42 h | 46.8% | Developing / Moderate |
| D | 22, Private, HS-grad, Never-married, Other-service, Female, gains 0, losses 0, 20 h | 0.0% | Emerging / High |
| E | 45, Self-emp-inc, Doctorate, Divorced, Prof-specialty, Male, gains 0, losses 1,902, 55 h | 81.0% | Established / Low |

**Table 49: Strongest drivers reported for each sample (two strengthening, one weakening)**

| ID | SHAP contributions (log-odds) |
|---|---|
| A | Years of education (+0.545); Age (+0.184); Investment gains (-1.626) |
| B | Investment gains (+4.678); marital status = Married-civ-spouse (+1.004); Overall investment activity (-0.243) |
| C | marital status = Married-civ-spouse (+0.836); relationship = Wife (+0.553); Investment gains (-0.156) |
| D | occupation = Machine-op-inspct (+0.020); occupation = Farming-fishing (+0.012); Age (-1.558) |
| E | Years of education (+1.812); Investment losses (+1.125); marital status = Married-civ-spouse (-1.466) |

The five outputs behave sensibly. Profile B (52, Masters, married, executive, gains 7,688) is near certain to be Established because the capital gain contributes +4.678 on the log-odds scale; Profile D (22, part-time, high-school graduate) is far below the Emerging threshold, with age the main weakening factor; Profile E combines a doctorate and long hours with a divorced

status and investment losses and lands in Established with a mixed explanation (education +1.813, losses +1.125, a pattern learned from the training data and not advice to incur losses, against marital status -1.466). Profile C falls in the Developing band. The what-if comparison shown in Section 8.1, applied to Profile A, with gains of 80,000 and 50 weekly hours, moved the score from 1.1% to 99.5% (+98.43 percentage points).

## 8.5 Success and failure analysis

### *Where the model succeeds*

The model is right on 8,579 of 9,769 test records and is confident when it is right: its mean confidence (the probability it assigns to the class it predicts) is 0.903 on correct predictions against 0.701 on errors. 5,907 correct predictions have confidence of at least 0.9 while only 127 errors do, so a very confident prediction is wrong about once in 47 cases. The Emerging band is the strongest: 7,123 rows, of which 7.2% are actually above 50K. The Adult data also contain an awkward spike at an investment gain of 99,999 (a value that appears to be a censoring code): all 63 test rows with that value are above 50K, which is consistent with the model treating a very large gain as a strong positive signal.

### *Where the model fails*

Of the 1,190 errors, 349 are low-confidence (below 0.6) and 1,232 test rows lie in the borderline probability range of 0.35 to 0.65, which is where most mistakes concentrate. The most confident mistakes are listed below. They are mostly cases where a single dominant attribute, usually capital gain, contradicts the label: the model sees a large capital gain (or, in the third case, a reported investment loss, which it also associates with higher incomes) and says 'above 50K', while the person's recorded income is below; the false negatives are young, unmarried or part-time workers whose recorded income is nonetheless high. Both kinds are individually plausible and show a limit of an eleven-attribute census profile: it cannot see the factors that actually separate such people.

**Table 50: Most confident errors on the test split (FP: predicted above 50K, actually at most 50K; FN: the reverse)**

| Case | Record | P(above 50K) |
|---|---|---|
| FP1 | 55, Self-emp-not-inc, HS-grad, Divorced, Exec-managerial, Male, gains 34,095, losses 0, 60 h | 0.9944 |
| FP2 | 63, Self-emp-not-inc, Masters, Married-civ-spouse, Farming-fishing, Male, gains 41,310, losses 0, 50 h | 0.9786 |
| FP3 | 57, Private, Assoc-voc, Married-civ-spouse, Prof-specialty, Male, gains 0, losses 1,902, 45 h | 0.9726 |
| FN1 | 26, Private, Bachelors, Never-married, Adm-clerical, Female, gains 0, losses 0, 40 h | 0.0064 |
| FN2 | 23, Local-gov, Some-college, Never-married, Protective-serv, Male, gains 0, losses 0, 40 h | 0.0082 |
| FN3 | 67, Private, 11th, Married-civ-spouse, Other-service, Male, gains 0, losses 0, 9 h | 0.0123 |

### *Error rates by group*

Because the model uses sex, marital status and age, the error rates were broken down by these groups on the test split. The table gives, for each group, the share who actually earn above 50K, the share the model predicts to, and the recall, false-positive rate and precision of the model. This is a descriptive check, not a fairness audit: no fairness criterion was set in advance and no mitigation was tried.

**Table 51: Test-split outcomes by group (threshold 0.5)**

| Attribute | Group | Rows | Actually >50K | Predicted >50K | Recall | False-positive rate | Precision |
|---|---|---|---|---|---|---|---|

| Attribute | Group | Rows | Actually >50K | Predicted >50K | Recall | False-positive rate | Precision |
|---|---|---|---|---|---|---|---|
| Sex | Female | 3,289 | 11.2% | 8.3% | 59.1% | 1.9% | 79.5% |
| Sex | Male | 6,480 | 30.4% | 26.2% | 68.0% | 7.8% | 79.1% |
| Marital status | Married-civ-spouse | 4,428 | 45.2% | 40.7% | 70.5% | 16.1% | 78.3% |
| Marital status | All other statuses | 5,341 | 6.3% | 3.1% | 43.6% | 0.4% | 88.5% |
| Education tier | Secondary-or-below | 4,450 | 13.4% | 5.9% | 35.8% | 1.2% | 81.6% |
| Education tier | Higher-secondary | 2,841 | 20.2% | 16.8% | 61.6% | 5.5% | 74.0% |
| Education tier | Tertiary | 2,478 | 47.2% | 49.6% | 84.8% | 18.2% | 80.7% |
| Age band | 18-25 | 1,969 | 1.8% | 0.7% | 34.3% | 0.1% | 92.3% |
| Age band | 26-35 | 2,501 | 18.8% | 13.8% | 57.2% | 3.7% | 78.0% |
| Age band | 36-50 | 3,296 | 36.1% | 32.2% | 71.1% | 10.3% | 79.6% |
| Age band | 51-65 | 1,642 | 34.2% | 29.8% | 68.5% | 9.7% | 78.6% |
| Age band | 65+ | 361 | 22.7% | 16.1% | 57.3% | 3.9% | 81.0% |

The disparities are large and they follow the data. In the test split 11.2% of women and 30.4% of men earn above 50K; the model predicts 8.3% and 26.2%. Its precision is almost identical for the two groups (79.5% and 79.1%), but recall is lower for women (59.1% against 68.0%), so high-earning women are missed more often than high-earning men. Married civilian-spouse households have a recall of 70.5% but also a false-positive rate of 16.1%, while every other marital status has a recall of only 43.6% and a false-positive rate of 0.4%: the model leans heavily on being married. Recall also rises steeply with education tier (35.8% for secondary or below, 84.8% for tertiary) and with age band (from 34.3% at 18-25 to 71.1% at 36-50). In plain terms, the model reproduces a historical income pattern in which men, married people, graduates and the middle-aged more often earn above 50K, and it is least able to recognise a high earner who is young, unmarried, a woman or has little formal education. That is a property of the 1994 data and of the model's reliance on it, and it is the main reason the application is described throughout as informational and not as a basis for decisions about individuals.

***Extrapolation and the what-if screen***

The what-if results (+98.43 and +98.26 points in the examples above and in Chapter 7) are a different kind of failure. Raising investment gains from a few thousand to tens of thousands or a million moves the score to the top of the scale because in the training data almost everyone with large gains earns above 50K. The model has not learned that a person can raise their own gains; it has learned a correlation, and the what-if screen applies it as if it were an intervention. The interface labels the result as a simulation; the numbers should be read as 'people with such gains in this data are almost all high earners', never as 'what you would earn if you changed this'.

## 8.6 Limitations of the results

- **Different target from the objective.** The model predicts income above 50K. It is a financial-profile signal built from census attributes, not a credit-default or distress model, and the report makes no claim about repayment.
- **Dated, single-source data.** The training data are 1994 US census records. Accuracy figures say how well the model reproduces that data, not how it would perform on a current or non-US population.
- **Synthetic forecast.** The forecast component was validated only against its own generator. Real economic indicators were not used.

- **Selection on the test split and no tuning.** The best model was chosen by test ROC-AUC, hyperparameters were fixed, and no significance test was run; the cross-validated AUC (0.929) is the safer estimate.
- **Single random split.** One stratified split and one cross-validation were run with one random seed (42). The variability across splits was not measured beyond the fold standard deviations.
- **No fairness audit.** Section 8.5 shows large group differences in recall and in predicted positive rate. They were measured but not corrected.
- **Modest recall.** About one in three high-income people is missed at the default threshold; the threshold can be lowered at the cost of precision.
- **Uncertain generalisation of scenarios.** What-if scores far from the bulk of the training data are extrapolations.

## 8.7 Objectives versus outcomes

The six objectives are those recorded in the project's objectives document. The table states plainly what was achieved against each.

**Table 52: Objectives versus outcomes**

| No. | Objective | Status | Outcome |
|---|---|---|---|
| 1 | Collect a dataset combining demographic attributes with financial and transactional parameters. | Met in part | UCI Adult / Census Income, 48,842 individual records with age, education, occupation, location (native country), and capital gains and losses. It has no transaction-level data, so the 'transactional parameters' part is not met. |
| 2 | Preprocess the data, handle missing values and encode categorical demographic fields. | Met | '?' markers turned into missing values, median / 'Unknown' imputation and one-hot encoding inside each scikit-learn Pipeline, and 12 interpretable derived features. |
| 3 | Train models to classify financial profile and estimate credit or distress risk. | Met in part | Four classifiers trained; XGBoost selected. The target is income above 50K, mapped to Emerging / Developing / Established profiles with Low / Moderate / High risk bands. It is not a credit-default or distress label. |
| 4 | Build a forecasting component that projects future financial position. | Met in part | Holt-Winters 12-month forecast with a 95% band is built and served, but on a documented synthetic Household Financial Conditions Index; it does not forecast an individual's own finances. |
| 5 | Add an explainability layer showing which factors drove each assessment. | Met | SHAP global importance, live per-assessment SHAP contributions in plain language, and a labelled non-causal what-if comparison. |
| 6 | Evaluate using accuracy, precision, recall and ROC-AUC, and deploy through a web interface. | Met | Accuracy, precision, recall, F1 and ROC-AUC reported for all four models; the system is deployed as a Flask API with a React interface on ports 8409 and 5409. |

Three of the six objectives are met in full and three in part. The parts that were not met share one cause: the project used a public census data set instead of a data set that combines demographic and transactional or credit-history variables, and no such data set was available openly. The consequences are that the classifier answers 'how does this profile compare with higher-earning profiles' instead of 'how likely is distress or default', and that forecasting is demonstrated on an aggregate index, not on a person's finances. Within those limits, the system delivers what was promised: a working, explainable, evaluated and deployed pipeline from demographic input to profile, explanation and forecast.

# CHAPTER 9: CONCLUSION

## 9.1 Summary

The project set out to identify an individual's financial profile from demographic and financial attributes, to forecast economic conditions and to explain every result through a web application. It delivers a Flask API and a React interface around four classifiers trained on 48,842 Adult / Census Income records inside leakage-safe pipelines; XGBoost was selected by ROC-AUC. For each assessment the application returns a profile (Emerging, Developing or Established), a risk band, a financial-strength score, the SHAP factors that raised or lowered it, written insights and a 12-month forecast context, and it can compare a baseline with a changed scenario.

The work follows the base paper [1] in structure, but changes the unit of analysis from hospitals to people, makes the explanation a per-request service and adds the forecasting and interface layers. The two studies differ in data, target and validation, so the comparison is one of approach, not of scores.

## 9.2 Objectives achieved

**Table 53: Objectives and their status**

| No. | Objective | Status | Outcome |
|---|---|---|---|
| 1 | Collect a dataset combining demographic attributes with financial and transactional parameters. | Met in part | UCI Adult / Census Income, 48,842 individual records with age, education, occupation, location (native country), and capital gains and losses. It has no transaction-level data, so the 'transactional parameters' part is not met. |
| 2 | Preprocess the data, handle missing values and encode categorical demographic fields. | Met | '?' markers turned into missing values, median / 'Unknown' imputation and one-hot encoding inside each scikit-learn Pipeline, and 12 interpretable derived features. |
| 3 | Train models to classify financial profile and estimate credit or distress risk. | Met in part | Four classifiers trained; XGBoost selected. The target is income above 50K, mapped to Emerging / Developing / Established profiles with Low / Moderate / High risk bands. It is not a credit-default or distress label. |
| 4 | Build a forecasting component that projects future financial position. | Met in part | Holt-Winters 12-month forecast with a 95% band is built and served, but on a documented synthetic Household Financial Conditions Index; it does not forecast an individual's own finances. |
| 5 | Add an explainability layer showing which factors drove each assessment. | Met | SHAP global importance, live per-assessment SHAP contributions in plain language, and a labelled non-causal what-if comparison. |
| 6 | Evaluate using accuracy, precision, recall and ROC-AUC, and deploy through a web interface. | Met | Accuracy, precision, recall, F1 and ROC-AUC reported for all four models; the system is deployed as a Flask API with a React interface on ports 8409 and 5409. |

Three objectives (preprocessing, explainability, and evaluation with deployment) are met as stated. Three are met only in part and the report says so: the dataset has no transaction-level data, the risk output is an income-based band and not a credit or distress label, and the forecast is of an aggregate synthetic index and not of an individual's finances.

## 9.3 Key findings

- **XGBoost is the best of the four.** On 9,769 held-out records it reaches accuracy 87.8%, precision 79.2%, recall 66.6%, F1 0.724 and ROC-AUC 0.932; its five-fold cross-validated AUC is 0.929 with a standard deviation of 0.001.

- **Boosted trees beat the linear baseline, but not by a lot.** Logistic Regression reaches AUC 0.914 and F1 0.676; Random Forest has AUC 0.910 (below the linear model) but a higher recall (63.7% against 61.1%) and F1 (0.685). The ordering by AUC is XGBoost, Gradient Boosting, Logistic Regression, Random Forest, the same family ordering as in the base paper except for the position of Random Forest.
- **Recall is the weak point.** At 0.5 the model finds 66.6% of the people with income above 50K. Moving the threshold to 0.35 gives recall 78.1%, precision 69.0% and the best F1 (0.733); the choice is a business decision, not a statistical one.
- **Probabilities are well calibrated on this data.** The mean prediction tracks the observed rate in nearly every one of ten bins (for example 0.984 predicted against 0.987 observed in the top bin) and the Brier score is 0.086; the largest gap is in the 0.4-0.5 bin (0.447 against 0.406).
- **The profile bands carry information.** In the test split 7.2% of the Emerging band, 44.0% of the Developing band and 85.2% of the Established band actually earn more than 50K.
- **What drives the model.** The five largest mean absolute SHAP values belong to marital status = Married-civ-spouse, Age, Years of education, Investment gains and hours worked per week. These are associations learned from 1994 census data, not causes.
- **The forecast works as software, but its series is synthetic.** The 12-month Holt-Winters forecast projects a +3.46% move of the index; against its own generator the back-test error is MAPE 1.00%, which says nothing about real economies.
- **The system is fast and tested.** A prediction takes a median of 68.9 ms; 9 automated tests and 35 of 35 smoke checks pass.

## 9.4 Practical applications

Within its limits the system can serve as an educational tool and as a pattern for similar projects: it shows how a bank, a financial-literacy programme or a policy analyst could present a model-based profile together with the factors behind it, instead of a bare score. It is useful for exploring how a few inputs move a modelled profile, for teaching the difference between association and causation using the what-if screen, and as a template for wrapping a scikit-learn pipeline with a JSON API and a React interface. It is not suitable for lending, hiring, insurance or any decision about a real person.

## 9.5 Known limitations

The limitations that matter most are the following; the full list with consequences is in the scope and limitations document and in Chapter 1.

- The training data are 1994 US census records and the target is income, not creditworthiness.
- The forecast series is synthetic and the forecast says nothing about a person's own finances.
- Sex, race and marital status are model inputs and no fairness audit or mitigation was performed.
- Recall of the positive class is 66.6% at the default threshold; the best model was chosen on the test split without a tuned search or significance tests.

- The sample login, absence of a database and development secret key make the application suitable only for local use.

# CHAPTER 10: FUTURE SCOPE

The system is a complete prototype whose limits point to its natural extensions. The ideas below are grouped by what they would change.

## 10.1 Improvements

- **Better data.** Retrain on a recent household survey with credit-bureau-style attributes and an observed repayment or distress outcome, so that the target matches the claim.
- **Evaluation.** Add a separate validation set, a hyperparameter search, DeLong tests between models, bootstrap intervals on every metric and a temporal split once time-stamped data exist.
- **Calibration and thresholds.** Apply recalibration and let the user or institution choose the threshold; the sweep shows recall of 78.1% at a threshold of 0.35 for the price of lower precision (69.0%).
- **Explanations.** Group related one-hot features before ranking, report SHAP interaction values and add counterfactual recourse (the smallest change that would move a profile to the next band).
- **Robustness of what-if.** Limit scenarios to the range where the training data are dense and warn when a change moves the score by more than a set amount.

## 10.2 Additional features

- **Real macroeconomic inputs.** Add inflation, employment and benchmark-rate series and let the forecast respond to them, as the project abstract describes.
- **Individual forecasting.** Savings and expenditure projection from a user-supplied history, with scenario bands.
- **Saved assessments.** Optional, consented storage of past assessments with a trend view, behind real authentication.
- **Reports.** Exportable PDF of a profile with its explanation, and a comparison of two people or two dates.
- **Localisation.** Region-specific option lists and currencies, and plain-language text in more languages.

## 10.3 Scalability and deployment

- **Packaging.** Extend the Docker setup, which today serves the Flask pages only, to run the React build and the API together with a reverse proxy.
- **Serving.** Run the API under a production WSGI server, keep the model loaded per worker and cache the global SHAP artefacts; measure behaviour under concurrent load, which was not tested here.
- **Operations.** Version models and artefacts, log predictions without personal data, monitor input drift and retrain on a schedule.
- **Security.** Replace the demo gate with an identity provider, move secrets out of defaults, and add rate limiting and input-size limits.

## 10.4 Further research

- **Extending the framework.** Apply the four-stage framework [1] to other individual-level settings (small-business owners, students, households) and compare it with deep tabular models [23].
- **Privacy.** Combine demographic features with alternative data without sharing raw records, along the lines of privacy-preserving credit-risk learning [4].
- **Data quality.** Measure how missing values, noise and label errors affect profile models, following the robustness methodology of [3].
- **Responsible use.** Study fairness, explanation faithfulness and user understanding of SHAP sentences [40], [21], with a user study of the kind not carried out here.
- **Forecasting.** Compare Holt-Winters with state-space and neural models on real household-finance series, and quantify interval coverage honestly.

Explainable machine learning for financial distress has been developed for organisations and sovereigns, while work on individuals concentrates on accuracy and uses a narrow set of financial parameters. None of the surveyed works combines individual demographic profiling, a per-person explanation, a forecast of economic conditions and a usable web interface in one reproducible system. This project fills that gap on public data.

# REFERENCES

[1] Y. Kim and E. J. Sun, "An explainable machine learning framework for organizational financial distress detection: Evidence from Korean hospital panel data," IEEE Access, vol. 14, pp. 100337-100350, 2026, doi: 10.1109/ACCESS.2026.3709346.

[2] A. Abid, "Forecasting sovereign credit risk amidst a political crisis: A machine learning and deep learning approach," J. Risk Financial Manag., vol. 18, no. 6, Art. no. 300, Jun. 2025, doi: 10.3390/jrfm18060300.

[3] A. Maurino, "How data quality affects machine learning models for credit risk assessment," in Proc. Workshop AI Data Sci. Digit. Finance, held with ACM Int. Conf. AI Finance (ICAIF), Singapore, 2025.

[4] H. Zhang, J. Xu, J. He, and X. Fang, "Privacy-preserving credit risk prediction with alternative data," arXiv:2606.10333, Jun. 2026.

[5] E. I. Altman, "Financial ratios, discriminant analysis and the prediction of corporate bankruptcy," J. Finance, vol. 23, no. 4, pp. 589-609, Sep. 1968.

[6] J. A. Ohlson, "Financial ratios and the probabilistic prediction of bankruptcy," J. Accounting Res., vol. 18, no. 1, pp. 109-131, 1980.

[7] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001.

[8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189-1232, Oct. 2001.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, Aug. 2016, pp. 785-794.

[10] F. Barboza, H. Kimura, and E. Altman, "Machine learning models and bankruptcy prediction," Expert Syst. Appl., vol. 83, pp. 405-417, Oct. 2017.

[11] M. Zieba, S. K. Tomczak, and J. M. Tomczak, "Ensemble boosted trees with synthetic features generation in application to bankruptcy prediction," Expert Syst. Appl., vol. 58, pp. 93-101, Oct. 2016.

[12] M. Moscatelli, F. Parlapiano, S. Narizzano, and G. Viggiano, "Corporate default forecasting with machine learning," Expert Syst. Appl., vol. 161, Art. no. 113567, Dec. 2020.

[13] F. Mai, S. Tian, C. Lee, and L. Ma, "Deep learning models for bankruptcy prediction using textual disclosures," Eur. J. Oper. Res., vol. 272, no. 2, pp. 743-758, 2019.

[14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Adv. Neural Inf. Process. Syst., vol. 30, 2017, pp. 4765-4774.

[15] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Mach. Intell., vol. 2, no. 1, pp. 56-67, Jan. 2020.

[16] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2016, pp. 1135-1144.

[17] N. Bussmann, P. Giudici, D. Marinelli, and J. Papenbrock, "Explainable machine learning in credit risk management," Comput. Econ., vol. 57, no. 1, pp. 203-216, Jan. 2021.

[18] A. Gramegna and P. Giudici, "SHAP and LIME: An evaluation of discriminative power in credit risk," Frontiers Artif. Intell., vol. 4, Art. no. 752558, Sep. 2021.

[19] J. Cerneviciene and A. Kabasinskas, "Explainable artificial intelligence (XAI) in finance: A systematic literature review," Artif. Intell. Rev., vol. 57, no. 8, pp. 1-80, Jul. 2024.

[20] A. Barredo Arrieta et al., "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," Inf. Fusion, vol. 58, pp. 82-115, Jun. 2020.

[21] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," arXiv:1702.08608, 2017.

[22] C. Molnar, G. Casalicchio, and B. Bischl, "Interpretable machine learning - A brief history, state-of-the-art and challenges," in Proc. ECML PKDD Workshops, 2020, pp. 417-431.

[23] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in Proc. Adv. Neural Inf. Process. Syst., 2022, pp. 507-520.

[24] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[25] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York, NY, USA: Springer, 2009.

[26] M. Stone, "Cross-validatory choice and assessment of statistical predictions," J. Roy. Stat. Soc., Ser. B, vol. 36, no. 2, pp. 111-133, 1974.

[27] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[28] T. Fawcett, "An introduction to ROC analysis," Pattern Recognit. Lett., vol. 27, no. 8, pp. 861-874, Jun. 2006.

[29] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," Radiology, vol. 143, no. 1, pp. 29-36, 1982.

[30] E. R. DeLong, D. M. DeLong, and D. L. Clarke-Pearson, "Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach," Biometrics, vol. 44, no. 3, pp. 837-845, Sep. 1988.

[31] R. Kohavi, "Scaling up the accuracy of Naive-Bayes classifiers: A decision-tree hybrid," in Proc. 2nd Int. Conf. Knowl. Discovery Data Mining (KDD), 1996, pp. 202-207.

[32] UCI Machine Learning Repository, "Adult (Census Income) data set." [Online]. Available: https://archive.ics.uci.edu/dataset/20/census+income

[33] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," Manage. Sci., vol. 6, no. 3, pp. 324-342, Apr. 1960.

[34] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," Int. J. Forecasting, vol. 20, no. 1, pp. 5-10, 2004.

[35] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," Int. J. Forecasting, vol. 22, no. 4, pp. 679-688, 2006.

[36] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021.

[37] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. Int. Conf. Mach. Learn. (ICML), 2017, pp. 1321-1330.

[38] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in Proc. 22nd Int. Conf. Mach. Learn. (ICML), 2005, pp. 625-632.

[39] M. Hardt, E. Price, and N. Srebro, "Equality of opportunity in supervised learning," in Proc. Adv. Neural Inf. Process. Syst., vol. 29, 2016, pp. 3315-3323.

[40] N. Mehrabi, F. Morstatter, N. Saxena, K. Lerman, and A. Galstyan, "A survey on bias and fairness in machine learning," ACM Comput. Surv., vol. 54, no. 6, Art. no. 115, 2021.

[41] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in Proc. 9th Python Sci. Conf. (SciPy), 2010, pp. 92-96.

[42] Pallets Projects, "Flask: Web development, one drop at a time." [Online]. Available: https://flask.palletsprojects.com

[43] Meta Platforms, Inc., "React: The library for web and native user interfaces." [Online]. Available: https://react.dev

[44] Vite contributors, "Vite: Next generation frontend tooling." [Online]. Available: https://vite.dev

[45] "Credit and financial cycle synchronization: Impact on sovereign credit risk," 2025 (entry recorded in the batch literature survey; not independently verified).

[46] "Financial and credit risk research," 2024 (entry recorded in the batch literature survey; not independently verified).

[47] "Design of a multimodal credit risk forecasting system for financial text analysis," in Proc. 2nd Int. Symp. Integr. Circuit Design Integr. Syst., 2025 (entry recorded in the batch literature survey; not independently verified).

# APPENDIX A: IMPORTANT CODE

The listings below are extracted from the repository by the document build script, so they are the code that was tested and measured. File paths are relative to `8.ProjectCode`. Only the parts that carry the method are shown; routine plumbing (routes that render templates, plotting code) is left out.

## A.1 Input validation (app/utils/validation.py)

Every request, whether from the React form or the JSON API, passes through this function. It checks that each field is present, belongs to its allowed set or numeric range, and derives `education-num` from the education label.

```
def validate_assessment(data) -> tuple:
    """Validate assessment input. Returns (cleaned_dict, errors_list)."""
    errors = []
    cleaned = {}


    def req_cat(key, allowed, label):
        raw = str(_get(data, key, "")).strip()
        if not raw:
            errors.append(f"{label} is required.")
            return None
        if raw not in allowed:
            errors.append(f"{label} has an invalid value.")
            return None
        return raw


    def req_num(key, label, lo, hi, integer=False):
        raw = str(_get(data, key, "")).strip()
        if raw == "":
            errors.append(f"{label} is required.")
            return None
        try:
            v = int(raw) if integer else float(raw)
    # ... (listing shortened; full source in the repository)
```

## A.2 Cleaning and derived features (scripts/feature_engineering.py)

`add_derived_features` creates the interpretable columns that are used identically in training and at prediction time.

```
def add_derived_features(df: pd.DataFrame) -> pd.DataFrame:
    """Add interpretable derived financial-health features.


    Every derived column is computable from available raw fields only.
    """
    df = df.copy()
    age = pd.to_numeric(df.get("age"), errors="coerce")
    gain = pd.to_numeric(df.get("capital-gain"), errors="coerce").fillna(0)
    loss = pd.to_numeric(df.get("capital-loss"), errors="coerce").fillna(0)
    hours = pd.to_numeric(df.get("hours-per-week"), errors="coerce")
    edu_num = pd.to_numeric(df.get("education-num"), errors="coerce")


    df["net-capital"] = gain - loss
    df["capital-intensity"] = (gain + loss).clip(lower=0)


    # Capital gains/losses are highly skewed; log-scale them for linear models
    # (tree models are unaffected by monotonic transforms of single columns).
    df["log-capital-gain"] = np.log1p(gain.clip(lower=0))
    df["log-capital-loss"] = np.log1p(loss.clip(lower=0))


    # Earnings-effort proxy: hours worked per week, banded.
    df["hours-band"] = pd.cut(
    # ... (listing shortened; full source in the repository)
```

## A.3 Leakage-safe preprocessing (scripts/feature_engineering.py)

```
NUMERIC_FEATURES = [
    "age", "fnlwgt", "education-num", "capital-gain", "capital-loss",
    "hours-per-week", "net-capital", "capital-intensity",
    "log-capital-gain", "log-capital-loss",
]


CATEGORICAL_FEATURES = [
    "workclass", "education", "marital-status", "occupation",
    "relationship", "race", "sex", "native-country",
    "hours-band", "has-capital-gain", "has-capital-loss",
    "age-band", "education-tier", "region",
    "employment-stability", "capital-position",
]
```

```
def build_preprocessor(scale_numeric: bool) -> ColumnTransformer:
    """Build a leakage-safe ColumnTransformer.


    scale_numeric=True for the Logistic Regression pipeline;
    tree pipelines use scale_numeric=False.
    """
    if scale_numeric:
        num_pipe = Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ])
    else:
        num_pipe = Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
        ])
    cat_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
        ("onehot", _onehot()),
    ])
    return ColumnTransformer([
        ("num", num_pipe, NUMERIC_FEATURES),
        ("cat", cat_pipe, CATEGORICAL_FEATURES),
    # ... (listing shortened; full source in the repository)
```

## A.4 Candidate models (scripts/models.py)

```
def get_candidate_models():
    return {
        "logistic_regression": LogisticRegression(
            max_iter=2000, C=1.0, random_state=RANDOM_STATE
        ),
        "random_forest": RandomForestClassifier(
            n_estimators=300, max_depth=None, min_samples_split=2,
            n_jobs=-1, random_state=RANDOM_STATE,
        ),
        "gradient_boosting": GradientBoostingClassifier(random_state=RANDOM_STATE),
        "xgboost": XGBClassifier(
            n_estimators=400, max_depth=6, learning_rate=0.05,
            subsample=0.9, colsample_bytree=0.9, reg_lambda=1.0,
            eval_metric="logloss", n_jobs=-1, random_state=RANDOM_STATE,
        ),
    }
```

## A.5 Split, cross-validation and model selection (scripts/train.py)

```
    # Cross-sectional: one row per person -> stratified split + stratified K-fold.
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)


    candidates = get_candidate_models()
    results = {}
    roc_curves = {}
    cv_summary = {}
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)


    for name, clf in candidates.items():
        scale = (name == "logistic_regression")
        pipe = Pipeline([
```

```
            ("preprocess", build_preprocessor(scale_numeric=scale)),
            ("model", clf),
        ])
        cv_scores = cross_val_score(pipe, X_train, y_train, cv=skf, scoring="roc_auc", n_jobs=-1)
        pipe.fit(X_train, y_train)
        proba = pipe.predict_proba(X_test)[:, 1]
        pred = (proba >= 0.5).astype(int)
        m = evaluate_binary(y_test, pred, proba)
        m["cv_roc_auc_mean"] = round(float(cv_scores.mean()), 4)
    # ... (listing shortened; full source in the repository)
```

# A.6 Forecasting (scripts/forecast.py)

```
def build_index_series() -> pd.DataFrame:
    rng = np.random.default_rng(RANDOM_STATE)
    dates = pd.date_range("2015-01-01", "2025-12-01", freq="MS")
    t = np.arange(len(dates))
    trend = 100 + 0.35 * t
    seasonal = 4.0 * np.sin(2 * np.pi * (dates.month - 1) / 12)
    noise = rng.normal(0, 1.6, size=len(dates))
    shock = np.zeros(len(dates))
    # Mild, documented downturn mid-2020 (illustrative, not a causal claim).
    shock[(dates >= "2020-03-01") & (dates <= "2020-08-01")] = -7.0
    values = trend + seasonal + shock + noise
    return pd.DataFrame({"date": dates, "index_value": np.round(values, 2)})
```

```
def fit_forecast(df: pd.DataFrame, horizon: int = FORECAST_HORIZON) -> dict:
    series = df.set_index("date")["index_value"].asfreq("MS")
    model = ExponentialSmoothing(
        series, trend="add", seasonal="add", seasonal_periods=12,
        initialization_method="estimated",
    )
    fit = model.fit(optimized=True)
    fc = fit.forecast(horizon)
    resid_std = float(np.std(fit.resid.dropna())) if len(fit.resid.dropna()) else 2.0
    last_date = series.index[-1]
    future_dates = pd.date_range(
        last_date + pd.offsets.MonthBegin(1), periods=horizon, freq="MS"
    )
    fc_vals = [round(float(v), 2) for v in fc.values]
    lower = [round(float(v - 1.96 * resid_std), 2) for v in fc.values]
    upper = [round(float(v + 1.96 * resid_std), 2) for v in fc.values]
    direction = "upward" if fc_vals[-1] > float(series.values[-1]) else (
        "downward" if fc_vals[-1] < float(series.values[-1]) else "stable")
    pct_change = round(
        100 * (fc_vals[-1] - float(series.values[-1])) / float(series.values[-1]), 2)
    fitted = [round(float(v), 2) for v in fit.fittedvalues.values]
    return {
    # ... (listing shortened; full source in the repository)
```

# A.7 Profile mapping, prediction and live SHAP (app/services/predictor.py)

```
def profile_from_proba(p: float) -> tuple:
    """Map P(income>50K) to (profile, risk_level)."""
    if p >= 0.60:
        return "Established", "Low"
    if p >= 0.35:
        return "Developing", "Moderate"
    return "Emerging", "High"
```

```
def local_shap(pipe, frame: pd.DataFrame, max_items: int = 8) -> dict:
    """Compute live SHAP contributions for one row.


    Returns dict with positives/negatives lists and base info.
    Never raises: failures return an explanatory fallback.
    """
    out = {"positives": [], "negatives": [], "expected_value": None,
           "available": False, "note": ""}
    try:
        explainer = get_explainer()
        if explainer is None:
            out["note"] = "Local explanations are unavailable for this model type."
            return out
        pre = pipe.named_steps["preprocess"]
        Xt = pre.transform(frame)
        names = _load_feature_names()
```

```
        if names is None:
            try:
                _ensure_scripts_path()
                from feature_engineering import get_output_feature_names
                names = get_output_feature_names(pre)
            except Exception:
    # ... (listing shortened; full source in the repository)
```

```
def predict_profile(cleaned: dict) -> dict:
    """Full prediction pipeline for one validated input dict."""
    _ensure_scripts_path()
    from feature_engineering import clean_frame, add_derived_features
    pipe = get_model()
    frame = build_input_frame(cleaned)
    frame = add_derived_features(clean_frame(frame))
    try:
        proba = float(pipe.predict_proba(frame)[0, 1])
    except Exception as e:
        raise RuntimeError(f"Prediction failed: {e}")
    profile, risk = profile_from_proba(proba)
    shap_local = local_shap(pipe, frame)
    insights = build_insights(cleaned, proba, profile, risk, shap_local)
    return {
        "proba": round(proba, 4),
        "strength_pct": round(proba * 100, 1),
        "risk_pct": round((1 - proba) * 100, 1),
        "profile": profile,
        "risk": risk,
        "model": get_model_name(),
        "inputs": cleaned,
    # ... (listing shortened; full source in the repository)
```

# APPENDIX B: INSTALLATION AND USER GUIDE

## B.1 Requirements

**Table 54: Software and hardware needed to run FinSight AI**

| Item | Requirement |
|---|---|
| Operating system | Windows 10 or 11 (developed and measured on Windows 11 Pro); the Python and Node steps are the same on other systems |
| Python | 3.11 or newer (measured with 3.11.9) with pip |
| Node.js | 18 or newer (measured with 24.19.0) for the React front end |
| Python packages | pandas, numpy, scikit-learn, xgboost, shap, statsmodels, Flask, joblib, matplotlib, pytest, requests, fpdf2 (see `requirements.txt`) |
| Hardware | Any ordinary laptop; no GPU. Measured on an Intel Core i5-12500 with 15.7 GB of RAM |
| Disk and network | About 1.5 GB for the virtual environment and `node_modules`; internet only for the first install |

## B.2 Automatic start with run.bat

Double-click `run.bat` in `8.ProjectCode`. The script reads the ports from `.env` (`PORT=8409` for the Flask API and `FRONTEND_PORT=5409` for the React app) and then performs six steps:

1. Checks that Python and Node.js are on the PATH and prints their versions.
2. Creates the `.venv` virtual environment if it does not exist.
3. Activates it and runs `pip install -r requirements.txt`.
4. Trains the models with `python scripts\train.py` if `models\best_model.pkl` is missing (a few minutes; the persisted models and artifacts are already in the repository).
5. Runs `npm install` and `npm run build` inside `frontend` on the first run.
6. Starts the API in a second window (`python run.py`), opens http://localhost:5409 and serves the built front end with `npm run preview`.

## B.3 Manual start

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python scripts\train.py          :: only if models\best_model.pkl is missing
python run.py                    :: API on http://localhost:8409


cd frontend
npm install
npm run dev                      :: http://localhost:5409 (proxies /api to 8409)
```

For a production-style run use `npm run build` followed by `npm run preview`, which also serves port 5409. The original server-rendered Flask pages remain available at http://localhost:8409. `python scripts\report_metrics.py` re-derives the confusion matrices, threshold sweep, calibration bins and forecast back-test from the saved models.

## B.4 Sample login

The application opens at a sign-in screen for the `/app` area. Use the email **demo@finsight.ai** and the password **finsight123**, or press *Use demo credentials*. This is a browser-only demonstration gate stored in `localStorage`: it does not authenticate anyone and FinSight AI has no user accounts or stored personal data.

## B.5 User guide, screen by screen

**Table 55: Screens of the application and what to do on each**

| Screen (route) | What it shows | What to do |
|---|---|---|
| Landing (`/`) | Headline *Understand your financial profile, gently.*, a live forecast chart, sections *How it works*, *Features*, *Numbers* with real model figures, a light / dark switch | Press *Start your assessment*, *Explore the dashboard* or *Open the app*; use *Sign in* if not yet signed in |
| Sign in (`/login`) | *Sample login* card with the demo credentials, *Email*, *Password*, *Show password* | Press *Use demo credentials*, then *Sign in*; a guarded address redirects here first and returns afterwards |
| Dashboard (`/app`) | Cards *Serving model*, *Accuracy*, *ROC-AUC*, *F1 score*; *Top drivers in the trained model*; *Economic outlook*; *What you can do here* | Read the model snapshot; press *Start assessment*, *Open explainability* or *Explore the forecast* |
| Assessment (`/app/assess`) | *Financial assessment* form titled *Your information* with twelve fields: Age, Employment type, Education, Marital status, Occupation, Household relationship, Race, Sex, Country, Investment gains, Investment losses, Hours worked per week | Fill the fields or press *Use sample profile*; press *Analyse my profile*; *Clear* empties the form. Out-of-range values are rejected with a message beside the field |
| Result (`/app/result`) | *Your financial profile*: *Predicted profile* (Emerging / Developing / Established), risk band, *Financial-strength score*, *What shaped your score* (strengthening and weakening SHAP factors), *Actionable insights*, *Your inputs*, a mini forecast | Press *Try a what-if scenario*, *Why this result?* or *Edit inputs* |
| Explainability (`/app/explain`) | *Explainability (SHAP)*: *Global importance* bar chart, *SHAP summary (beeswarm)*, *Original SHAP bar chart*, searchable *Ranked features* table, *How to read this* | Search or sort the table; press *Get my local explanation* to run an assessment |
| Forecast (`/app/forecast`) | *Economic forecast*: cards *Horizon*, *Trend*, *Projected change*, *Residual std*; chart *Index history and forecast* with a 95% band and range tabs; *Interpretation*; *Forecast values* table | Choose *Full history*, *Last 5 years* or *Last 2 years*; page through the table |
| What-if (`/app/whatif`) | *What-if financial scenario*: *Baseline information* (pre-filled from the last assessment), *Scenario changes* (gains, losses, hours per week, education), baseline and scenario cards and a *Comparison* | Change at least one scenario field, leave others blank, press *Compare scenarios* |
| Performance (`/app/performance`) | *Model performance*: *Model comparison* table, *Metrics at a glance*, *Confusion matrix*, *ROC curves*, *Validation strategy*, *Cross-validation fold AUCs*, *Statistical analysis* | Sort the comparison table; read the validation protocol |
| About (`/app/about`) | *About FinSight AI*: *How it works* (six steps), *Technology*, *Responsible use* | Read the responsible-use statement |
| Not found | *Page not found* with *Go to the dashboard* and *Landing page* | Follow either button |

## B.6 Troubleshooting

**Table 56: Common problems and fixes**

| Symptom | Cause | Fix |
|---|---|---|
| run.bat says Python or Node.js was not found | Not installed or not on the PATH | Install Python 3.11+ and Node.js 18+, tick *Add python.exe to PATH*, reopen the window |
| Port 8409 or 5409 already in use | Another program holds the port | Change `PORT` or `FRONTEND_PORT` in `.env` and restart |
| Header pill shows *API offline*, or the form says *make sure the API is running (run.bat)* | The Flask window was closed or failed to start | Restart `python run.py`; open http://localhost:8409/health, which should answer `status: ok` |
| Result page asks for an assessment first | The last result and baseline are kept in the browser tab's session storage only and are cleared on sign-out | Press *Start assessment* and analyse a profile again |
| Version warnings about scikit-learn or XGBoost when the API starts | The `.pkl` files were written by an older library version | Retrain with `python scripts\train.py`; the numbers reproduce exactly |
| `shap` or `xgboost` fails to install | No wheel for the Python version | Use Python 3.11 and upgrade pip first |
| Screenshots script cannot find a browser | Chrome missing | `pip install playwright` then `playwright install chromium` |

# APPENDIX C: ADDITIONAL SCREENS

The screens below complement those shown in Chapter 8. All were captured at 1440 x 900 pixels (full-page captures are cropped to their upper part) from the running application with real model output; none is a mock-up.

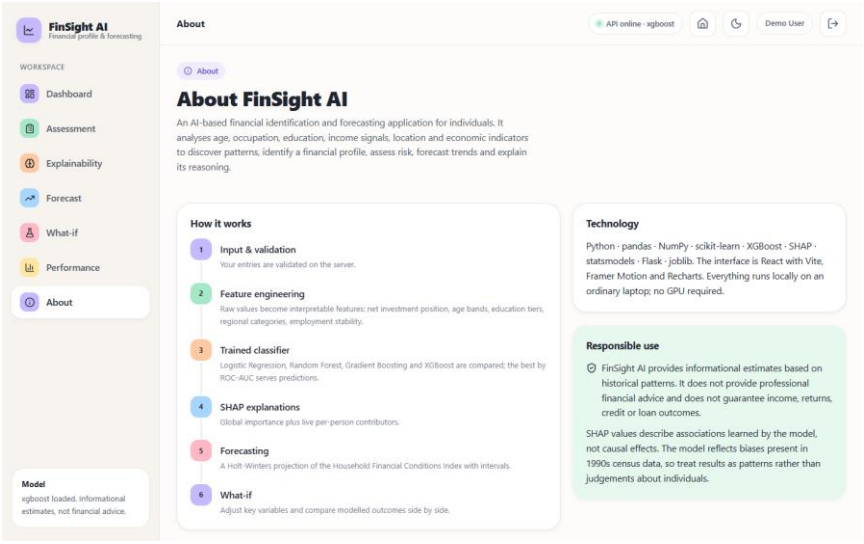



**Figure 37: About screen: the six-step description of how the system works, the technology list and the responsible-use statement**

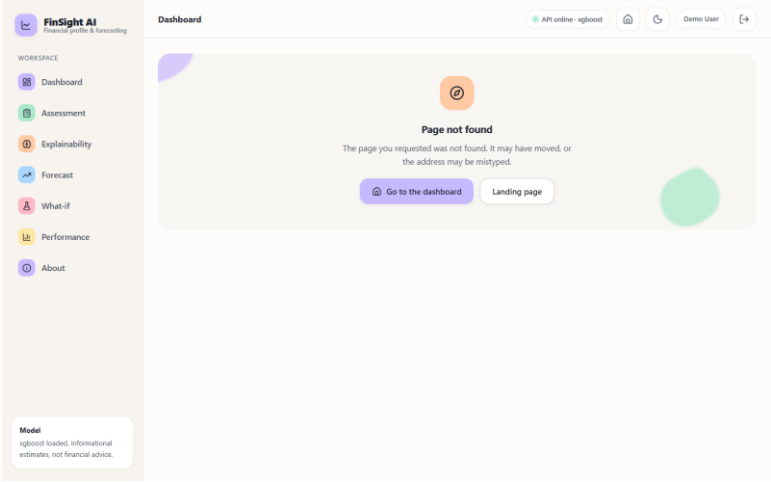



**Figure 38: Page-not-found screen shown for an unknown address inside the application, with buttons back to the dashboard and the landing page**

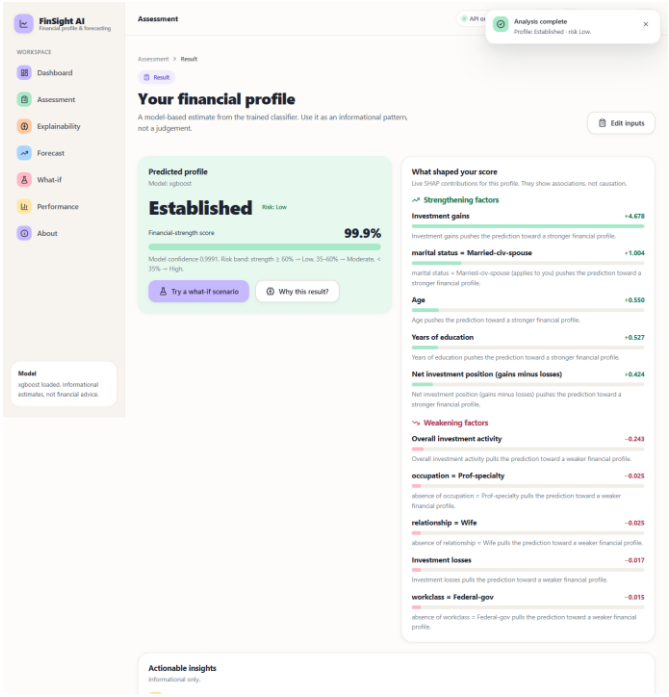



**Figure 39: Result for a 52-year-old married executive with a Master's degree, gains of 7,688 and 50 working hours: profile *Established*, risk *Low*, strength 99.9% (upper part of the page, with the strengthening factors led by investment gains)**

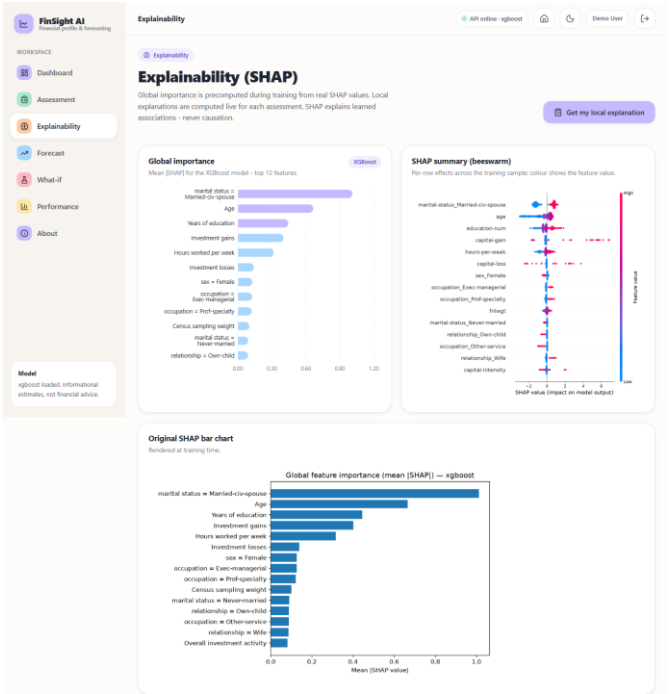



**Figure 40: Explainability screen (upper part): global importance bars for the top twelve features, the SHAP beeswarm summary and the original bar chart rendered at training time**

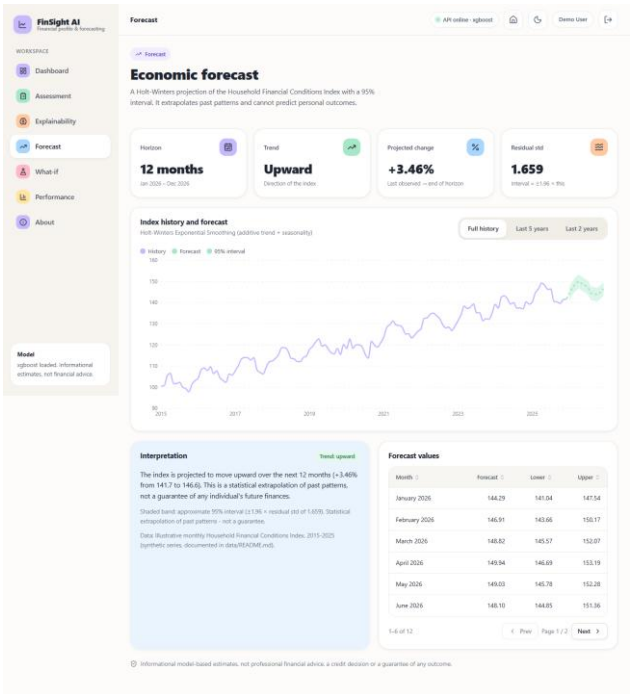



**Figure 41: Forecast screen: horizon, trend, projected change (+3.46%) and residual standard deviation (1.659); the history-and-forecast chart with its 95% band; the interpretation text that names the series as synthetic; and the first six forecast months**

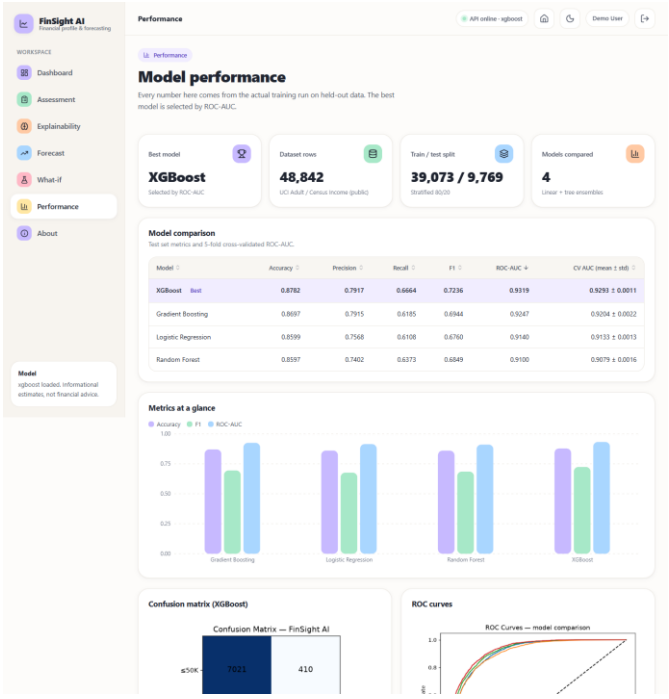



**Figure 42: Performance screen (upper part): best model, dataset rows, train / test split, the four-model comparison table, the metrics-at-a-glance bars and the XGBoost confusion matrix with the ROC curves**